Lattice-Based Post-Quantum Cryptography

Complete Lecture Notes

Dr. Faisal Aslam

Lessons 1–4

Contents

Lesson 1: Mathematical Foundations for Lattice-Based Cryptography

Groups, Rings, Fields, Modular Arithmetic, Vector Spaces, Basis, and Lattices

Purpose of this lecture. Every technical term used later in the thesis proposal — "lattice," "modulus," "ring $\mathbb{Z}_q[x]/(x^n + 1)$" — rests on a small set of algebraic building blocks. This lecture builds those blocks, one at a time, from the ground up: *Group* → *Subgroup* → *Ring* → *Field* → *Modular Arithmetic* → *Vector Space* → *Basis* → *Lattice*. Nothing here requires prior abstract algebra; every definition is followed immediately by concrete, worked examples.

1.1 Why Start With Algebra?

Lattice-based cryptography is, at its core, about doing arithmetic on *vectors* whose coordinates live in a *finite, wraparound number system*, arranged according to strict algebraic rules. Before we can even state what a lattice is, we need precise answers to questions like:

- What does it mean for a set of numbers to have a well-behaved notion of "add" and "multiply"? (**Groups, Rings, Fields**)
- What happens when arithmetic "wraps around," like a clock? (**Modulus**)
- What does it mean to build every point in a space out of a small number of building-block directions? (**Vector spaces, Basis**)
- What happens when we only allow *whole-number* combinations of those building blocks, instead of any real-number combination? (**Lattice**)

This lecture answers each of these in order. By the end, the definition of a lattice (Section 1.8) should feel like the natural, inevitable next step, not an isolated new idea.

1.2 Sets and Binary Operations (Quick Refresher)

A **set** is just a collection of objects (numbers, vectors, symbols — anything). A **binary operation** on a set S is a rule that takes two elements of S and combines them into a third element, e.g., ordinary addition $+$ takes two numbers and produces their sum. We write $a * b$ for a general binary operation.

An operation is **closed** on S if combining two elements of S always gives another element of S (never something outside S). This sounds obvious but fails more often than you'd expect — see Example 1 below.

Example: Closure Can Fail

Let $S = \{1, 2, 3, 4, 5\}$ (just these five numbers) and let $*$ be ordinary addition. Then $2 * 4 = 6$, but $6 \notin S$. So ordinary addition is *not* closed on S. This is exactly the kind of problem modular arithmetic (Section 1.5) is built to fix.

1.3 Groups

Definition: Group

A set G together with a binary operation $*$ is called a **group**, written $(G, *)$, if all four of the following hold:
(G1) **Closure:** for all $a, b \in G$, $a * b \in G$.
(G2) **Associativity:** for all $a, b, c \in G$, $(a * b) * c = a * (b * c)$.
(G3) **Identity:** there exists an element $e \in G$ such that $a * e = e * a = a$ for all $a \in G$.
(G4) **Inverses:** for every $a \in G$, there exists $a^{-1} \in G$ such that $a * a^{-1} = a^{-1} * a = e$.
If, in addition, $a * b = b * a$ for all $a, b \in G$ (the order doesn't matter), the group is called **abelian** (commutative).

Example: Groups You Already Know

- $(\mathbb{Z}, +)$ — the integers under addition. Identity is 0; the inverse of a is $-a$. This is an abelian group.
- $(\mathbb{R} \setminus \{0\}, \times)$ — nonzero real numbers under multiplication. Identity is 1; the inverse of a is $1/a$. Also abelian. (We had to remove 0 because 0 has no multiplicative inverse.)
- $(\mathbb{Z}, \times)$ — integers under multiplication — is **not** a group: 2 has no inverse in $\mathbb{Z}$ (we'd need $1/2$, which isn't an integer). This fails axiom (G4).

Example: Groups Hiding in Less Obvious Places: Matrices and Polynomials

The three examples above were all sets of plain numbers. Groups show up in far less numeric-looking settings too — the axioms (G1)–(G4) don't care what the elements *are*, only that closure, associativity, identity, and inverses all hold. Four examples worth having in your back pocket, each a little "fancier" than the last:

- 3×3 **real matrices under addition,** $(M_{3\times 3}(\mathbb{R}), +)$**, is a group.** Adding two 3×3 matrices gives another 3×3 matrix (closure); matrix addition is associative; the identity is the all-zeros matrix $\mathbf{0}$; the inverse of A is $-A$ (negate every entry). It's even abelian, since $A + B = B + A$ entrywise. Nothing about this is different in spirit from $(\mathbb{Z}, +)$ — just 9 numbers moving in lockstep instead of 1.
- **The same set under multiplication is *not* a group.** Try $(M_{3\times 3}(\mathbb{R}), \times)$ (ordinary matrix multiplication): closure holds (multiplying two 3×3 matrices gives another 3×3 matrix) and multiplication is associative, and there's an identity, I_3. But axiom (G4) fails: a **singular** matrix (one with $\det = 0$, e.g. the all-zeros matrix, or $\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$) has no inverse matrix at all — there is no B with $AB = BA = I_3$. So the full set of 3×3 matrices under multiplication is not a group, for exactly the same structural reason $(\mathbb{Z}, \times)$ wasn't: some elements simply lack an inverse. (Also worth noting for later: even restricting to the invertible matrices, this group would *not* be abelian in general — $AB \neq BA$ for most matrix pairs — unlike every example so far.)
- **Invertible matrices under multiplication, on the other hand, *do* form a group.** Restrict to only the invertible 3×3 matrices, $GL_3(\mathbb{R}) = \{A \in M_{3\times 3}(\mathbb{R}) : \det(A) \neq 0\}$. Now every element has an inverse *by construction* (that's what "invertible" means), the product of two invertible matrices is again invertible (closure restored), and I_3 is still the identity. This is called the **general linear group**, and it's the standard fix for the multiplication problem above: throw away the elements that were breaking (G4), and check that what's left is still closed.
- **Unitary matrices under multiplication form a group too — a more specialized cousin of** GL_3**.** A (complex) $n \times n$ matrix U is **unitary** if $U^*U = I$, where U^* is the conjugate transpose (Lecture 1, Section 1.7.2's Hermitian inner product remark is exactly the conjugate this definition needs). The set of $n \times n$ unitary matrices, under multiplication, is a group: the product of two unitary matrices is unitary (check: $(U_1U_2)^*(U_1U_2) = U_2^*U_1^*U_1U_2 = U_2^*IU_2 = U_2^*U_2 = I$), I itself is unitary, and every unitary U has an inverse — namely $U^{-1} = U^*$, which is itself unitary. This group, $U(n)$, sits *inside* $GL_n(\mathbb{C})$ as a much smaller, highly-structured subgroup (Section 1.3.1's subgroup idea, one level up): every unitary matrix is invertible, but very few invertible matrices are unitary.
- **Degree-**$< n$ **polynomials under addition form a group; under multiplication they do not.** Let $P_n = \{a_0 + a_1x + \cdots + a_{n-1}x^{n-1} : a_i \in \mathbb{R}\}$ (real polynomials of degree strictly less than n). Under addition, $(P_n, +)$ is a group exactly like matrices under addition above: closed (adding two such polynomials keeps the degree below n), associative, identity is the zero polynomial, and the inverse of $p(x)$ is $-p(x)$. Under multiplication, it immediately fails on *two* separate grounds: closure fails (multiplying two degree-$(n-1)$ polynomials can produce degree up to $2n-2$, which is no longer in P_n at all), and even ignoring that, most individual polynomials have no multiplicative inverse within polynomials (e.g., there is no polynomial $q(x)$ with $x \cdot q(x) = 1$ for every x). This is exactly the polynomial-ring preview from Section 1.4 below: $(P_n, +)$ being a group but $(P_n, \times)$ not being one is precisely why we need the weaker notion of a **ring** (two operations, only one of which needs full group structure) to describe polynomials properly, rather than expecting both operations to behave like a group.

Remark: Why Cryptography Cares About Groups

Many classical cryptographic schemes (Diffie–Hellman, ElGamal, elliptic-curve cryptography) are built directly on top of group structure — the security relies on certain problems (like "given g and g^a, find a") being hard inside a specific group. Lattice-based cryptography instead builds on richer structures (rings and modules, coming next), but the group is always the first layer underneath.

Exercise: Try This Before Next Lecture

Is $(\mathbb{Z}_5 \setminus \{0\}, \times \bmod 5) = (\{1,2,3,4\}, \times \bmod 5)$ a group? Check all four axioms directly by building the multiplication table. (You'll need Section 1.6 for what "$\bmod 5$" means if it's new to you — come back to this after reading that section.)

1.3.1 Subgroups

There is one more group-related idea we need before we can properly define a lattice: a group living *inside* another group.

Definition: Subgroup

Let $(G, *)$ be a group. A subset $H \subseteq G$ is a **subgroup** of G if H is itself a group under the same operation $*$. In practice, this is easiest to check with a simple test: H is a subgroup of G if and only if (1) H is nonempty (usually: it contains the identity e), (2) H is closed under $*$ (for $a, b \in H$, $a * b \in H$), and (3) H is closed under inverses (for $a \in H$, $a^{-1} \in H$). Associativity is automatic, since it already held for all of G.

Example: Subgroups You Can Picture

- For any integer n, the set $n\mathbb{Z} = \{\ldots, -2n, -n, 0, n, 2n, \ldots\}$ (all multiples of n) is a subgroup of $(\mathbb{Z}, +)$: it contains 0, adding two multiples of n gives another multiple of n, and $-(kn)$ is also a multiple of n. For example, $2\mathbb{Z}$ is "all even integers."
- The natural numbers $\mathbb{N} = \{0, 1, 2, \ldots\}$ are **not** a subgroup of $(\mathbb{Z}, +)$: $\mathbb{N}$ is closed under addition, but it fails closure under inverses — $1 \in \mathbb{N}$, but $-1 \notin \mathbb{N}$.

Looking Ahead: Why This Matters in Two Sections

Keep the example $n\mathbb{Z} \subseteq \mathbb{Z}$ in mind. In Section 1.8, we will define a lattice as a special kind of subgroup of $\mathbb{R}^n$ — and $n\mathbb{Z}$ is exactly a simple, one-dimensional lattice in disguise: an evenly-spaced, discrete set of points, closed under addition and negation, sitting inside the larger group $\mathbb{Z}$ (or $\mathbb{R}$).

1.3.2 Cosets and Quotient Groups: Where $\mathbb{Z}_n$ Really Comes From

Section 1.6 will introduce $\mathbb{Z}_n$ informally, as "remainders with wraparound arithmetic." That's the right intuition, but it's worth seeing the fully rigorous construction now, because it is a direct, satisfying payoff of the subgroup idea — and it is the same construction ("take a group, collapse it by a subgroup") that shows up again later when we build the ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ that Kyber and Dilithium actually use.

Remark: The One-Sentence Idea, Before Any Symbols

A coset is a "shifted copy" of a subgroup. You take every element of H and slide it over by adding a fixed amount a. That's all $a + H$ is: H, translated — nothing more exotic than that.

Definition: Coset

Let H be a subgroup of an abelian group $(G, +)$. For $a \in G$, the **coset** $a+H$ is the set $\{a+h : h \in H\}$ — "shift H by a." Two elements $a, b \in G$ turn out to produce the *same* coset, $a+H = b+H$, exactly when $a - b \in H$. This gives a clean way to sort all of G into non-overlapping "buckets," each one a shifted copy of H.

Example: Example 1 — The Simplest Possible Coset: Evens and Odds

Let $G = \mathbb{Z}$ and $H = 2\mathbb{Z} = \{\ldots, -4, -2, 0, 2, 4, \ldots\}$ (the even numbers; check it's a subgroup exactly as in the "Subgroups You Can Picture" example: contains 0, sum of two evens is even, negative of an even is even). Now form cosets by picking different a's:

- $0 + H = \{\ldots, -4, -2, 0, 2, 4, \ldots\}$ — same set as H itself (the evens)
- $1 + H = \{\ldots, -3, -1, 1, 3, 5, \ldots\}$ — the odds
- $2 + H = \{\ldots, -2, 0, 2, 4, 6, \ldots\}$ — same set as $0 + H$ again!
- $3 + H = \{\ldots, -1, 1, 3, 5, 7, \ldots\}$ — same set as $1 + H$ again!

No matter which integer you pick as a, you only ever land on **one of two** possible sets: "the evens" or "the odds." Every integer belongs to exactly one of these two buckets — that's the whole idea of cosets, in the simplest case there is.

Why did $0+H$ and $2+H$ turn out to be the same set? This is exactly the rule from the defbox above: $a + H = b + H$ iff $a - b \in H$. Check: $2 - 0 = 2$, and 2 is even, so $2 \in H$ — that's *why* they collapsed to the same coset. Meanwhile $1 - 0 = 1$, which is odd, not in H — so $1 + H$ is genuinely a different coset from $0 + H$.

Example: Example 2 — A "Clock" Version: The Cosets of $5\mathbb{Z}$ in $\mathbb{Z}$

Same idea as Example 1, one notch more general: let $H = 5\mathbb{Z} = \{\ldots, -10, -5, 0, 5, 10, \ldots\}$. There are exactly **5** distinct cosets:

Coset	What's in it	"Bucket"
$0 + 5\mathbb{Z}$	$\ldots, -5, 0, 5, 10, \ldots$	remainder 0
$1 + 5\mathbb{Z}$	$\ldots, -4, 1, 6, 11, \ldots$	remainder 1
$2 + 5\mathbb{Z}$	$\ldots, -3, 2, 7, 12, \ldots$	remainder 2
$3 + 5\mathbb{Z}$	$\ldots, -2, 3, 8, 13, \ldots$	remainder 3
$4 + 5\mathbb{Z}$	$\ldots, -1, 4, 9, 14, \ldots$	remainder 4

You have secretly been computing cosets your whole life: **a coset of $n\mathbb{Z}$ is just "everything with the same remainder when divided by n."** Every integer sorts into exactly one bucket, and there are exactly n buckets. This is precisely $\mathbb{Z}_n$ (Section 1.6), and it's exactly the same pattern as the $3\mathbb{Z}$ case used later in this lecture — there is nothing special about 5 here, only that it makes the "n buckets" pattern easy to see with more than 2 or 3 of them.

Example: Example 3 — A Picture You Can Draw: Lines in the Plane

This one is worth sitting with, because it connects straight to the lattice tilings of Section 1.8.5. Let $G = \mathbb{R}^2$ (the plane, under vector addition) and let H be a line through the origin — say $H = \{(t, t) : t \in \mathbb{R}\}$, the line $y = x$ (check it's a subgroup: contains $(0, 0)$; sum of two points on the line is on the line; negation of a point on the line is on the line).
Pick a point $a = (1, 0)$, not on the line, and form the coset:

$$(1, 0) + H = \{(1 + t,\ t) : t \in \mathbb{R}\}.$$

This is the line $y = x - 1$ — **the same line H, just shifted to pass through $(1, 0)$ instead of the origin.** Pick a different point on that same shifted line, say $a' = (3, 2)$: then $a' + H$ is *also* the line $y = x - 1$ — same coset, different name. Check the rule: $a - a' = (1, 0) - (3, 2) = (-2, -2)$, and $(-2, -2) = (t, t)$ with $t = -2$, so $(-2, -2) \in H$ — confirming $(1, 0) + H = (3, 2) + H$.
The picture: the cosets of H are exactly all the lines parallel to H. The whole plane $\mathbb{R}^2$ gets sliced into infinitely many parallel lines, and every point of the plane lies on exactly one of them. This "slicing the whole space into non-overlapping parallel copies" picture is *exactly* the fundamental-domain tiling picture of Section 1.8.5 — cosets and lattice tilings are the same idea, one dimension apart in this example.

Example: The Cosets of $3\mathbb{Z}$ in $\mathbb{Z}$ (the Version Used Later in This Lecture)

Take $G = \mathbb{Z}$ and $H = 3\mathbb{Z} = \{\ldots, -3, 0, 3, 6, \ldots\}$ (a subgroup, per the "Subgroups You Can Picture" example in Section 1.3.1). The cosets are:
- $0 + 3\mathbb{Z} = \{\ldots, -3, 0, 3, 6, \ldots\}$ (multiples of 3)
- $1 + 3\mathbb{Z} = \{\ldots, -2, 1, 4, 7, \ldots\}$ (remainder 1 when divided by 3)
- $2 + 3\mathbb{Z} = \{\ldots, -1, 2, 5, 8, \ldots\}$ (remainder 2 when divided by 3)

Every integer falls into exactly one of these three buckets — and which bucket it falls into is precisely its remainder mod 3. There is no fourth distinct coset: $3 + 3\mathbb{Z} = 0 + 3\mathbb{Z}$ (check: $3 - 0 = 3 \in 3\mathbb{Z}$), so the pattern repeats.

Remark: The Two Facts Worth Holding Onto

1. **Same-coset test:** $a + H = b + H$ if and only if $a - b \in H$ — this is how you check whether two things you've written down are secretly the same coset (used explicitly in all three examples above).
2. **No partial overlap:** two cosets are either *identical* or *completely disjoint* — never overlapping-but-not-equal. That's what lets cosets cleanly partition G into non-overlapping buckets, with nothing left over and nothing double-counted.

Definition: Quotient Group

The set of all distinct cosets of H in G, with addition defined by $(a+H)+(b+H) := (a+b)+H$, is itself a group — the **quotient group**, written G/H ("G mod H"). Its identity element is the coset $0 + H = H$ itself.

Remark: $\mathbb{Z}_n$ Is Not Just *Like* $\mathbb{Z}/n\mathbb{Z}$ — It *Is* $\mathbb{Z}/n\mathbb{Z}$

Compare the coset example above to Section 1.6: the three cosets of $3\mathbb{Z}$ in $\mathbb{Z}$ correspond *exactly* to the three elements $\{0, 1, 2\}$ of $\mathbb{Z}_3$, and coset addition matches addition mod 3 exactly. This is not a coincidence or an analogy — it is a theorem: $\mathbb{Z}_n$, as defined informally in Section 1.6, is precisely the quotient group $\mathbb{Z}/n\mathbb{Z}$. "Wraparound arithmetic" and "sorting integers into cosets of $n\mathbb{Z}$" are two descriptions of the exact same object. This is the rigorous justification for a claim we made without proof in Section 1.6: that $\mathbb{Z}_n$ really is a well-defined group (in fact ring) under addition and multiplication with wraparound — it inherits this structure directly from being a quotient of $\mathbb{Z}$.

Exercise: Practice

Let $H = 4\mathbb{Z} \subseteq \mathbb{Z}$. List all distinct cosets of H in $\mathbb{Z}$, and confirm there are exactly 4 of them, matching $\mathbb{Z}_4 = \{0, 1, 2, 3\}$. Which coset does -5 belong to?

1.4 Rings

A ring is what you get when you ask a set to support *two* binary operations that play two structurally different roles — one behaves like addition (fully invertible), the other behaves like multiplication (weaker, no inverses required). We always *label* these two operations "$+$" and "$\times$," but — and this is worth being precise about, since it's easy to misread — **the labels do not mean the operations have to literally be school-arithmetic addition and multiplication**. They can be almost any two operations at all, as long as together they satisfy the axioms below. "$+$" and "$\times$" are names for the *roles* the two operations play, not a requirement about what the operations must concretely do.

Definition: Ring

A set R with two binary operations $+$ and $\times$ is a **ring** $(R, +, \times)$ if:

(R1) $(R, +)$ is an abelian group (with identity element usually written 0).

(R2) $\times$ is associative: $(a \times b) \times c = a \times (b \times c)$.

(R3) **Distributivity of $\times$ over $+$:** $\times$ distributes over $+$, not the other way around — i.e., "multiplying a sum" can be split into "sum of the multiplications," but there is no requirement (and generally no meaning) for $+$ to distribute over $\times$. This comes in two versions, since $\times$ need not be commutative (see the note below), so both the left and right versions must be stated separately:

$$\underbrace{a \times (b + c) = (a \times b) + (a \times c)}_{\text{left distributivity}}, \qquad \underbrace{(a + b) \times c = (a \times c) + (b \times c)}_{\text{right distributivity}}.$$

Note: a ring does *not* need multiplicative inverses, and multiplication need not be commutative in general (though in every ring we use in this thesis, it will be — these are called **commutative rings**). If there is an element $1 \in R$ with $a \times 1 = 1 \times a = a$ for all a, we say R is a **ring with unity**.

Remark: Why Two Distributive Laws, and Why Only Over $+$

When $\times$ is commutative (as in every ring used in this course), left and right distributivity say the same thing in different order, so either one implies the other. But the *axioms* state both, because a ring is not required to be commutative in general (Section 1.3's matrix example under multiplication is a case where order matters). What never appears, in any ring, is a law like $a + (b \times c) = (a + b) \times (a + c)$ — $+$ never needs to distribute over $\times$. Distributivity is a one-way bridge: it tells you how $\times$ interacts with $+$, not the reverse.

Remark: "$+$" and "$\times$" Are Role Labels, Not a Restriction to Arithmetic

Compare this to groups (Section 1.3): a group has *one* operation, and we wrote it as a generic $*$ precisely to emphasize that it could be anything (addition, multiplication, matrix multiplication, function composition — whatever satisfies the axioms). A ring needs *two* operations, and it turns out we always call them "$+$" and "$\times$" — but this is a naming convention for the two *roles*, not a claim that the operations must be ordinary numeric addition and multiplication. Concretely:

- $n \times n$ **matrices:** "$+$" is ordinary matrix addition (this forms an abelian group), and "$\times$" is matrix *multiplication* (associative, distributes over $+$) — but matrix multiplication is **not** commutative in general, which is fine, since a ring's axioms above don't require it (only *commutative* rings require it, and this one wouldn't qualify as one).
- **Boolean rings** (used in digital logic): "$+$" is XOR and "$\times$" is AND on $\{0, 1\}$ — nothing to do with school arithmetic at all, yet every ring axiom above holds.

So whenever you meet a new ring in this course, the first question to ask is not "is $+$ really addition and is $\times$ really multiplication?" but rather: *what are the two operations here, and do they satisfy axioms (R1)–(R3)?* The labels "$+$"/"$\times$" just tell you which operation is playing which role once you've checked that.

Example: Rings You Already Know

- $(\mathbb{Z}, +, \times)$ — the integers form a commutative ring with unity 1. Notice 2 still has no multiplicative inverse, and that's fine — rings don't require one.
- **Polynomial rings**, e.g. $\mathbb{Z}[x]$: the set of *all* polynomials with integer coefficients, **of every degree at once** — no upper bound on degree at all. It contains 7 (degree 0), $3x - 1$ (degree 1), $3x^2 - x + 7$ (degree 2), and so on, all mixed together in one set, with ordinary polynomial addition and multiplication. This is a ring for exactly the same reason $\mathbb{Z}$ is: closure holds because adding or multiplying two polynomials, of *any* finite degrees, always produces another polynomial of *some* finite degree — still inside $\mathbb{Z}[x]$, whatever that degree turns out to be.

  Contrast this with P_n from Section 1.3's "Groups Hiding in Less Obvious Places" example, the set of polynomials of degree *strictly less than* a fixed n. There, closure under addition holds (adding two degree-$(n-1)$ polynomials keeps the degree below n), but closure under multiplication fails (the product can reach degree $2n-2$, escaping P_n) — which is precisely why $(P_n, +)$ is only a group, never a ring. $\mathbb{Z}[x]$ avoids this problem by simply not capping the degree in the first place, so there is no ceiling for a product to break through.

Looking Ahead: The Ring That Kyber and Dilithium Actually Use

Both Kyber and Dilithium do their arithmetic inside a specific ring of the form

$$R_q = \mathbb{Z}_q[x]/(x^n + 1),$$

read as: "polynomials with coefficients taken modulo q (Section 1.6), where we also reduce powers of x using the rule $x^n \equiv -1$." This looks intimidating right now, but it is built from exactly the pieces in this lecture: a polynomial ring (this section), with coefficients from $\mathbb{Z}_q$ (Section 1.6), further "folded" by a fixed rule. We will unpack this fully in a later lecture — for now, just notice that "ring" is not an abstract curiosity; it's the literal object the cryptography is built on.

1.4.1 Zero Divisors: A Structural Reason Some Rings Aren't Fields

We will soon see (Section 1.5) that $\mathbb{Z}_n$ is a field exactly when n is prime. Section 1.6.1 will justify this computationally, via gcd. Here is the deeper, structural reason *why* — worth seeing now, while we're still inside general ring theory.

Definition: Zero Divisor

In a ring R, a nonzero element a is a **zero divisor** if there exists some *nonzero* $b \in R$ with $a \times b = 0$. A commutative ring with unity that has *no* zero divisors (other than needing a or b to be 0) is called an **integral domain**.

Example: Zero Divisors in $\mathbb{Z}_6$, Absent in $\mathbb{Z}_5$

In $\mathbb{Z}_6$: $2 \times 3 = 6 \equiv 0 \pmod 6$, but neither 2 nor 3 is 0. So 2 and 3 are both zero divisors — $\mathbb{Z}_6$ is *not* an integral domain. Contrast this with $\mathbb{Z}_5$: checking every pair of nonzero elements $(1, 2, 3, 4)$, no product is ever $\equiv 0 \pmod 5$ (e.g., $2 \times 3 = 6 \equiv 1$, $4 \times 4 = 16 \equiv 1$, etc.) — $\mathbb{Z}_5$ has no zero divisors.

Remark: Why This Explains the Prime/Composite Split

Fact: every field is automatically an integral domain (if $a \neq 0$ and $ab = 0$, multiply both sides by a^{-1} to get $b = 0$ — so no nonzero b can pair with a nonzero a to give 0). Contrapositive: *if a ring has a zero divisor, it cannot be a field.* Now, $\mathbb{Z}_n$ has a zero divisor exactly when n is composite: write $n = jk$ with $1 < j, k < n$; then $j \times k = n \equiv 0 \pmod{n}$, with neither j nor k equal to 0 in $\mathbb{Z}_n$. So every composite modulus produces zero divisors, which immediately rules out being a field — a purely structural explanation, with no gcd computation needed, of the same fact Section 1.6.1 reaches computationally.

Exercise: Practice

Find a pair of zero divisors in $\mathbb{Z}_{12}$ (there are several valid answers). Then explain, using the Fact above, why $\mathbb{Z}_{12}$ cannot be a field — without computing any inverses directly.

1.5 Fields

Definition: Field

A **field** is a set F with two binary operations $+$ and $\times$ such that:
(F1) $(F, +)$ is an abelian group, over *all* of F (identity element 0).
(F2) $(F \setminus \{0\}, \times)$ is an abelian group, over F **with** 0 **removed** (identity element 1).
(F3) **Distributivity of $\times$ over $+$:** $a \times (b + c) = (a \times b) + (a \times c)$. (Only this one direction needs stating here, unlike the Ring definition above: (F2) already makes $\times$ commutative on $F \setminus \{0\}$, and $0 \times x = x \times 0 = 0$ always, so left and right distributivity coincide automatically — there's no separate "right" version to write down.)
Informally: a field is a set where you can add, subtract, multiply, and divide (by anything except 0) and always stay inside the set. In short — **a field is two abelian groups sharing one set, glued together by distributivity.**

Remark: Why 0 Has to Be Excluded, and It's Not Optional

In *any* structure with a distributive law, $0 \times x = 0$ for every x (this follows from distributivity itself, not from an extra rule: $0 \times x = (0 + 0) \times x = 0 \times x + 0 \times x$, so subtracting $0 \times x$ from both sides gives $0 = 0 \times x$). So 0 could only have a multiplicative inverse if $0 \times x = 1$ for some x — but the left side is always 0, never 1 (as long as $1 \neq 0$, which we always assume). So 0 is permanently excluded from having an inverse, in every field, without exception; it isn't a special case someone chose, it's forced by the arithmetic.

Definition: Subtraction and Division Are *Derived*, Not Extra

Neither subtraction nor division is a new, independent operation requiring its own axiom — both are just notation for "combine with an inverse," one level for each operation:

$$a - b := a + (-b), \qquad \frac{a}{b} := a \times b^{-1} \ \ (b \neq 0),$$

where $-b$ is guaranteed to exist by $(F, +)$ being a group (F1), and b^{-1} is guaranteed to exist, for every *nonzero* b, by $(F \setminus \{0\}, \times)$ being a group (F2). So "a field supports division" is not an extra requirement on top of the definition — it falls straight out of (F2) once every nonzero element is already promised an inverse. This is exactly why (F2) excludes 0: since 0 can never have an inverse (see the remark above), "dividing by 0" can never be defined this way, no matter what field you're in.

Example: Division in $\mathbb{Z}_5$, Worked Out

What is $4 \div 3$ in $\mathbb{Z}_5$? First find $3^{-1} \bmod 5$: $3 \times 2 = 6 \equiv 1 \pmod 5$, so $3^{-1} = 2$. Then $4 \div 3 := 4 \times 3^{-1} = 4 \times 2 = 8 \equiv 3 \pmod 5$. Check: $3 \times 3 = 9 \equiv 4 \pmod 5$ — correct, exactly as "$4 \div 3 = 3$" should verify.

Remark: The One-Line Takeaway

Subtraction is the additive inverse; division is the multiplicative inverse. Same trick, applied once per operation — $a - b$ means "add a and b's additive inverse," a/b means "multiply a by b's multiplicative inverse." The one asymmetry between them traces directly back to (F1) vs. (F2): *every* element of F has an additive inverse, since $(F, +)$ is a group over *all* of F — so subtraction always works. But *only nonzero* elements have a multiplicative inverse, since $(F \setminus \{0\}, \times)$ is a group only after removing 0 — so division by b only ever works when $b \neq 0$.

Fact: Every Field Is a Ring

Every field F (Definition above) is automatically a commutative ring with unity — in fact, an **integral domain** (Section 1.4.1). *Why:* (F1) is exactly ring axiom (R1). Associativity and distributivity of $\times$ (ring axioms (R2)–(R3)) are inherited directly from $\times$ being a group operation on $F \setminus \{0\}$ together with (F3). So a field satisfies every ring axiom, plus two extra guarantees stacked on top: $\times$ is commutative, and every *nonzero* element is invertible. This is why some textbooks instead *define* a field as "a commutative ring with unity in which every nonzero element has a multiplicative inverse" — that phrasing and the definition above pick out exactly the same objects; it's just built in the reverse order (starting from "ring" and adding a condition, instead of starting from two groups and gluing them together). Finally, a field is automatically an integral domain too: if $ab = 0$ with $a \neq 0$, multiplying both sides by a^{-1} forces $b = a^{-1}(ab) = a^{-1} \cdot 0 = 0$, so there can be no zero divisors.

Example: Fields and Non-Fields

- $\mathbb{Q}$ (rationals), $\mathbb{R}$ (reals), $\mathbb{C}$ (complex numbers) are all fields — you can divide by any nonzero element and stay inside the set.
- $\mathbb{Z}$ is **not** a field: $2 \in \mathbb{Z}$ has no multiplicative inverse in $\mathbb{Z}$ (again, $1/2 \notin \mathbb{Z}$). $\mathbb{Z}$ is a ring, but not a field.
- $\mathbb{Z}_5 = \{0, 1, 2, 3, 4\}$ under addition and multiplication mod 5 *is* a field — every nonzero element has an inverse (you checked this, or will check this, in the Section 1.3 subgroup/group exercise). Fields built this way, from $\mathbb{Z}_p$ with p prime, are called **finite fields** or **Galois fields**, written $\mathbb{F}_p$ or $GF(p)$.

Remark: The One-Sentence Summary So Far

Group $\subset$ Ring $\subset$ Field, in the sense that a field is a ring with extra structure (division), and a ring is built from a group with extra structure (a second operation). Each step adds a capability: groups let you undo one operation (inverses); rings add a second, distributive operation; fields guarantee *both* operations are fully invertible (except division by 0).

1.6 The Modulus: Arithmetic That Wraps Around

Definition: Congruence Modulo n

For a fixed positive integer n (the **modulus**), two integers a and b are **congruent modulo** n, written

$$a \equiv b \pmod{n},$$

if n divides $(a - b)$ evenly — equivalently, a and b leave the same remainder when divided by n. The set of possible remainders, $\{0, 1, 2, \ldots, n-1\}$, together with addition and multiplication performed "with wraparound," is written $\mathbb{Z}_n$ (or $\mathbb{Z}/n\mathbb{Z}$).

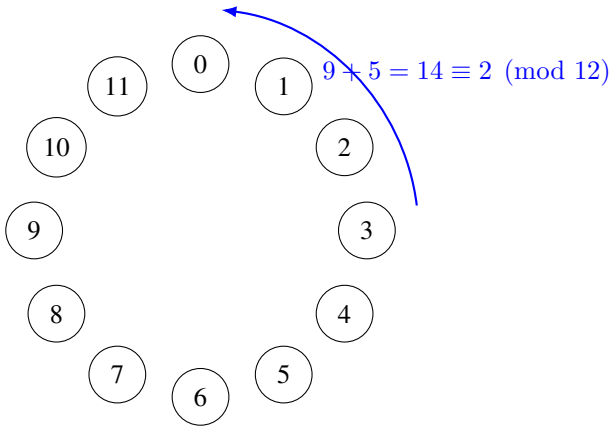


Figure 1.1: The clock analogy for modular arithmetic: $\mathbb{Z}_{12}$ works exactly like a 12-hour clock face. 9 o'clock plus 5 hours wraps around to 2 o'clock, i.e., $9 + 5 = 14 \equiv 2 \pmod{12}$.

Example: Computing Modulo

$17 \bmod 5$: divide 17 by 5 to get 3 remainder 2, so $17 \equiv 2 \pmod{5}$. Similarly, $23 \equiv 3 \pmod{10}$, and $-1 \equiv n - 1 \pmod{n}$ (e.g., $-1 \equiv 4 \pmod{5}$) — negative numbers wrap around too.

Remark: When Is $\mathbb{Z}_n$ a Ring? When Is It a Field?

$\mathbb{Z}_n$ is *always* a commutative ring with unity, for any $n \geq 2$. But it is a *field* only when $n = p$ is **prime**. If n is composite (e.g., $n = 6$), some nonzero elements fail to have multiplicative inverses — e.g., in $\mathbb{Z}_6$, there is no x with $2x \equiv 1 \pmod{6}$, since 2 shares a common factor with 6. This is exactly why cryptographic schemes that need a field (rather than just a ring) are built with a prime modulus q. (Section 1.4.1 gave the deeper structural reason: composite n always produces zero divisors, and a ring with zero divisors can never be a field.)

1.6.1 Composite Moduli Aren't Fields — But Their Invertible Elements Still Form a Group

The remark above closes the door firmly: $\mathbb{Z}_n$ for composite n is never a field, no matter how you look at it — some nonzero elements simply have no multiplicative inverse, and that's disqualifying, permanently. But it's natural to ask a follow-up question: *which* elements of $\mathbb{Z}_n$ *do* have an inverse, and how many of them are there? That subset turns out to be extremely well-behaved — just not a field, and not all of $\mathbb{Z}_n$.

Definition: Euler's Totient Function

For a positive integer n, **Euler's totient function** $\varphi(n)$ counts how many integers in $\{1, 2, \ldots, n\}$ are **coprime** to n (i.e., $\gcd(a, n) = 1$). Equivalently — recalling the "Fact: When Does an Inverse Exist?" remark box in the Extended Euclidean Algorithm subsection just below, $a \in \mathbb{Z}_n$ has a multiplicative inverse iff $\gcd(a, n) = 1$ — $\varphi(n)$ is exactly **the number of elements of $\mathbb{Z}_n$ that have a multiplicative inverse**.

Definition: The Group of Units $\mathbb{Z}_n^*$

The set of invertible elements of $\mathbb{Z}_n$,

$$\mathbb{Z}_n^* \;=\; \{a \in \mathbb{Z}_n : \gcd(a, n) = 1\},$$

is called the **group of units** modulo n. Under multiplication mod n, $(\mathbb{Z}_n^*, \times)$ is an abelian group (Section 1.3): it's closed (the product of two units is a unit — if a, b each have inverses, so does ab, namely $b^{-1}a^{-1}$), it has identity 1, and by definition every element already has an inverse. By construction, $|\mathbb{Z}_n^*| = \varphi(n)$.

Remark: This Is *Not* a Field — Read the Claim Carefully

It's tempting to summarize this as "$\mathbb{Z}_n$ is secretly a field with only $\varphi(n)$ elements," but that phrasing is not correct, and it's worth being precise about why. A field needs *every* nonzero element invertible, checked against the field's own addition (Section 1.5, axiom F1: $(F, +)$ a group over *all* of F). $\mathbb{Z}_n^*$ is not closed under $+$ at all — e.g., in $\mathbb{Z}_{12}^* = \{1, 5, 7, 11\}$, $5{+}7 = 12 \equiv 0 \pmod{12}$, which isn't even in $\mathbb{Z}_{12}^*$ (it's not coprime to 12). So $\mathbb{Z}_n^*$ is a perfectly good *group under multiplication alone* — it is **not** a smaller field sitting inside $\mathbb{Z}_n$; it doesn't have an addition operation of its own at all. The correct statement is: composite $\mathbb{Z}_n$ is a ring, not a field, but its invertible elements form a multiplicative group of order $\varphi(n)$.

Example: Computing $\varphi(12)$ and $\mathbb{Z}_{12}^*$ by Hand

List 1 through 12 and check gcd with $12 = 2^2 \times 3$ for each:

a	$\gcd(a, 12)$	coprime to 12?
1	1	yes
2	2	no
3	3	no
4	4	no
5	1	yes
6	6	no
7	1	yes
8	4	no
9	3	no
10	2	no
11	1	yes
12	12	no (this is 0 in $\mathbb{Z}_{12}$)

So $\mathbb{Z}_{12}^* = \{1, 5, 7, 11\}$ and $\varphi(12) = 4$. Check closure under multiplication mod 12: $5 \times 7 = 35 \equiv 11$, $5 \times 11 = 55 \equiv 7$, $7 \times 11 = 77 \equiv 5$, $5 \times 5 = 25 \equiv 1$, $7 \times 7 = 49 \equiv 1$, $11 \times 11 = 121 \equiv 1$ — every product of two elements of $\{1, 5, 7, 11\}$ lands back in $\{1, 5, 7, 11\}$, exactly as the "group of units" defbox above promises. Notice $\varphi(12) = 4 < 12$: only 4 of the 12 elements of $\mathbb{Z}_{12}$ are invertible, and the other 8 (including all the zero divisors found in the Section 1.4.1 example, 2 and 3) are permanently excluded from $\mathbb{Z}_{12}^*$.

Remark: A Formula, and the Prime Sanity Check

For a prime p, every one of $1, \ldots, p-1$ is automatically coprime to p (since p's only divisors are 1 and p itself), so $\varphi(p) = p - 1$ — matching exactly the earlier fact that *all* nonzero elements of $\mathbb{Z}_p$ are invertible when p is prime. In other words, $\mathbb{Z}_p^* = \mathbb{Z}_p \setminus \{0\}$ precisely when p is prime — the group of units swallows the entire nonzero ring, which is exactly the extra guarantee that makes $\mathbb{Z}_p$ a field (Section 1.5) and $\mathbb{Z}_{12}$ merely a ring. For composite n, $\varphi(n)$ is computed from the prime factorization of n (e.g., $\varphi(12) = \varphi(2^2)\varphi(3) = 2 \times 2 = 4$, matching the hand count above), but the formula itself isn't needed for this course — what matters is the concept: $\varphi(n)$ measures exactly how far $\mathbb{Z}_n$ falls short of being a field.

Looking Ahead: Why This Matters Later: RSA

Euler's totient function is the load-bearing quantity behind RSA (the classical, non-lattice-based scheme this thesis's cryptography ultimately aims to replace): RSA's security and correctness both hinge on $\varphi(n)$ for a composite $n = pq$ (product of two large primes), computed via $\varphi(pq) = (p-1)(q-1)$, and Euler's theorem, $a^{\varphi(n)} \equiv 1 \pmod{n}$ for $a \in \mathbb{Z}_n^*$, which generalizes Fermat's Little Theorem from prime moduli to composite ones. This isn't needed anywhere else in this lecture series, but it's worth recognizing the totient function on sight — it's one of the few classical-cryptography ideas that resurfaces constantly in the literature this thesis will cite.

Exercise: Practice

Compute $\varphi(10)$ by listing $\mathbb{Z}_{10}^*$ directly (check $\gcd(a, 10) = 1$ for $a = 1, \ldots, 10$). Then verify closure under multiplication mod 10 for at least two pairs of elements from your list, the way the $\varphi(12)$ example above did.

1.6.2 Finding Inverses Without Guessing: The Extended Euclidean Algorithm

So far, the only way we've found a multiplicative inverse mod n is trial and error (Exercise: "find x with $3x \equiv 1 \pmod{7}$" — you could just try $x = 1, 2, 3, \ldots$ until one works). That's fine for small numbers, but real cryptographic moduli are enormous (Kyber's modulus is $q = 3329$; RSA moduli have hundreds of digits) — trial and error is completely impractical. We need an actual algorithm.

Definition: Greatest Common Divisor (GCD)

For integers a, b (not both zero), $\gcd(a, b)$ is the largest positive integer that divides both a and b. Two integers are **coprime** (or relatively prime) if $\gcd(a, b) = 1$.

Remark: Fact: When Does an Inverse Exist?

An element $a \in \mathbb{Z}_n$ has a multiplicative inverse if and only if $\gcd(a, n) = 1$. This is why every nonzero element of $\mathbb{Z}_p$ (for prime p) has an inverse: any a with $1 \leq a \leq p - 1$ automatically satisfies $\gcd(a, p) = 1$, since p's only divisors are 1 and p itself.

The classical **Euclidean Algorithm** computes $\gcd(a, b)$ by repeated division with remainder: $\gcd(a, b) = \gcd(b, a \bmod b)$, stopping when the remainder hits 0. The **Extended** Euclidean Algorithm does this same process but also tracks, at *every single step*, how the current remainder can be written as a combination $a \cdot x + b \cdot y$ (integers x, y) — and when the algorithm finishes with $\gcd(a, b) = 1$, the row where the remainder equals 1 hands you the inverse directly. The cleanest way to track this — much easier to follow than plain back-substitution — is a running **table** with one row per step.

Definition: The Extended Euclidean Algorithm, as a Table

To find $\gcd(a, b)$ together with integers x, y satisfying $a \cdot x + b \cdot y = \gcd(a, b)$, build a table with columns q (quotient), r (remainder), x, y, starting with two fixed initial rows and then one new row per division step:

step	q	r	x	y
-1	—	a	1	0
0	—	b	0	1
1	$q_1 = \lfloor r_{-1}/r_0 \rfloor$	$r_1 = r_{-1} - q_1 r_0$	$x_1 = x_{-1} - q_1 x_0$	$y_1 = y_{-1} - q_1 y_0$
2	$q_2 = \lfloor r_0/r_1 \rfloor$	$r_2 = r_0 - q_2 r_1$	$x_2 = x_0 - q_2 x_1$	$y_2 = y_0 - q_2 y_1$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Every single row satisfies the **invariant** $r_i = a \cdot x_i + b \cdot y_i$ by construction — this is the whole mechanism, and it's worth checking explicitly on at least one row the first time you use this table, to convince yourself it isn't magic. Stop once a row produces $r_i = 0$; the *previous* row's r is $\gcd(a, b)$, and that row's x, y are exactly the combination you want.

Example: Table Method, Short Version: $\gcd(7, 3)$ and $3^{-1} \bmod 7$

step	q	r	x	y	check: $7x + 3y$
-1	—	7	1	0	$7(1) + 3(0) = 7$ ✓
0	—	3	0	1	$7(0) + 3(1) = 3$ ✓
1	$\lfloor 7/3 \rfloor = 2$	$7 - 2(3) = 1$	$1 - 2(0) = 1$	$0 - 2(1) = -2$	$7(1) + 3(-2) = 1$ ✓
2	$\lfloor 3/1 \rfloor = 3$	$3 - 3(1) = 0$	$0 - 3(1) = -3$	$1 - 3(-2) = 7$	(stop: $r = 0$)

The row *before* r hit 0 is step 1: $r_1 = 1 = \gcd(7, 3)$, with $x_1 = 1, y_1 = -2$. So $7(1)+3(-2) = 1$, i.e., $3 \times (-2) \equiv 1 \pmod 7$, giving $3^{-1} \equiv -2 \equiv 5 \pmod 7$ — matching what trial and error would also find, but now via a method with a clear, mechanical stopping point.

Example: Table Method, Longer Version: $15^{-1} \bmod 26$

Here the table earns its keep — more steps than the tiny example above, but the process is identical, row by row.

step	q	r	x	y
-1	—	26	1	0
0	—	15	0	1
1	$\lfloor 26/15 \rfloor = 1$	$26 - 1(15) = 11$	$1 - 1(0) = 1$	$0 - 1(1) = -1$
2	$\lfloor 15/11 \rfloor = 1$	$15 - 1(11) = 4$	$0 - 1(1) = -1$	$1 - 1(-1) = 2$
3	$\lfloor 11/4 \rfloor = 2$	$11 - 2(4) = 3$	$1 - 2(-1) = 3$	$-1 - 2(2) = -5$
4	$\lfloor 4/3 \rfloor = 1$	$4 - 1(3) = 1$	$-1 - 1(3) = -4$	$2 - 1(-5) = 7$
5	$\lfloor 3/1 \rfloor = 3$	$3 - 3(1) = 0$	$3 - 3(-4) = 15$	$-5 - 3(7) = -26$

The row before r hit 0 is step 4: $r_4 = 1 = \gcd(26, 15)$, with $x_4 = -4, y_4 = 7$. **Check the invariant directly:** $26(-4) + 15(7) = -104 + 105 = 1$ ✓. Reading this modulo 26: $15 \times 7 \equiv 1 \pmod{26}$, so $15^{-1} \equiv 7 \pmod{26}$. **Final check:** $15 \times 7 = 105 = 4 \times 26 + 1 \equiv 1 \pmod{26}$. Correct.

Exercise: Practice

Use the table method above (not back-substitution, and not trial and error) to find $5^{-1} \bmod 11$. Build the full table — steps $-1, 0, 1, 2, \ldots$ — and stop at the correct row. Then check your answer by direct multiplication.

Looking Ahead: Why Cryptography Loves the Modulus

Modular arithmetic gives us two things cryptography needs badly: (1) a **finite** number system (so keys and computations have a fixed, bounded size, no matter how much you compute), and (2) built-in **one-wayness** in places — e.g., given only $a \bmod n$, you cannot tell which of infinitely many integers a actually was. The Learning-With-Errors (LWE) problem from the proposal document hides its secret inside equations that live in exactly this kind of finite, modular number system (there, the modulus is usually called q).

Exercise: Practice

(a) Compute $23 \times 15 \bmod 7$. (b) List all elements of $\mathbb{Z}_8$ that *do not* have a multiplicative inverse, and explain why (hint: compare each element to 8). (c) Confirm $\mathbb{Z}_7$ is a field by checking that 3 has a multiplicative inverse mod 7 (find x such that $3x \equiv 1 \pmod 7$).

1.7 Vector Spaces, Basis, and Dimension

Definition: Vector Space

A **vector space** over a field F is a set V (whose elements are called **vectors**) equipped with an addition operation $V \times V \to V$ and a scalar multiplication operation $F \times V \to V$, satisfying, for all $u, v, w \in V$ and all $a, b \in F$:

(V1) **Closure of addition:** $u + v \in V$.
(V2) **Associativity of addition:** $(u + v) + w = u + (v + w)$.
(V3) **Commutativity of addition:** $u + v = v + u$.
(V4) **Additive identity:** there exists $\vec{0} \in V$ with $v + \vec{0} = v$ for all v.
(V5) **Additive inverses:** for every $v \in V$, there exists $-v \in V$ with $v + (-v) = \vec{0}$.
(V6) **Compatibility of scalar multiplication with field multiplication:** $a(bv) = (ab)v$.
(V7) **Identity element of scalar multiplication:** $1v = v$, where 1 is the multiplicative identity of F.
(V8) **Distributivity, two directions:** $a(u + v) = au + av$ *and* $(a + b)v = av + bv$.

Axioms (V1)–(V5) say exactly "$(V, +)$ is an abelian group" (Section 1.3) — nothing new there. Axioms (V6)–(V8) are the genuinely new content: they say scalar multiplication by field elements behaves consistently with the field's own addition and multiplication (Sections 4–5). The most important example for us: $\mathbb{R}^n$, the set of all n-tuples of real numbers, is a vector space over $\mathbb{R}$ — and it's a good exercise to mentally check each of (V1)–(V8) holds there before moving on.

Definition: Linear Combination, Span, Linear Independence

Given vectors $v_1, \ldots, v_k \in V$ and scalars $c_1, \ldots, c_k \in F$, the vector $c_1v_1 + c_2v_2 + \cdots + c_kv_k$ is called a **linear combination** of $v_1, \ldots, v_k$. The set of *all* linear combinations of a set of vectors is called their **span**. The vectors $v_1, \ldots, v_k$ are **linearly independent** if the only way to write $0 = c_1v_1 + \cdots + c_kv_k$ is with every $c_i = 0$ — informally, no vector in the set is "redundant" (expressible using the others).

1.7.1 Testing Linear Independence: Row Operations and Row Echelon Form

The definition above is a yes/no test stated abstractly in terms of the c_i's. For concrete vectors in $\mathbb{R}^n$, it is also a completely mechanical computation — and the standard tool for carrying it out, **Gaussian elimination**, is worth setting up properly here, since we will lean on it again once bases and lattices enter the picture.

Definition: Elementary Row Operations

Given a matrix, the following three operations on its rows are called **elementary row operations**:

(E1) **Row swap:** interchange two rows, $R_i \leftrightarrow R_j$.

(E2) **Row scaling:** multiply a row by a nonzero scalar, $R_i \to c\,R_i$ for some $c \neq 0$.

(E3) **Row addition (replacement):** add a scalar multiple of one row to another, $R_i \to R_i + c\,R_j$ (for $i \neq j$).

Each operation is **reversible** by another operation of the same type: a swap undoes itself; scaling by c is undone by scaling by $1/c$; adding cR_j is undone by adding $-cR_j$. This reversibility is exactly why row operations never destroy information — every row of the new matrix is some combination of the old rows, and (because the operation can be undone) every old row is likewise recoverable as a combination of the new ones.

Fact: Row Operations Preserve the Span — and Hence Independence — of the Rows

Let $v_1, \ldots, v_k$ be the rows of a matrix A. Applying any sequence of elementary row operations produces a new matrix A' whose rows $v_1', \ldots, v_k'$ **span exactly the same space** as $v_1, \ldots, v_k$ did — because, by the remark above, each new row is some linear combination of the old rows, and each old row is recoverable as a combination of the new ones. In particular: $v_1, \ldots, v_k$ are linearly independent **if and only if** $v_1', \ldots, v_k'$ are. Row operations are therefore "free" for this purpose — we can simplify a set of vectors as aggressively as we like without ever changing the yes/no answer to the independence question, exactly the same way row operations leave $\gcd$, solution sets, and (as we'll see in Section 1.8.5) a lattice's determinant unchanged under the right kind of transformation.

Definition: Row Echelon Form

A matrix is in **row echelon form (REF)** if:

(1) Every all-zero row (if any) lies below every nonzero row.

(2) In each nonzero row, its first nonzero entry — the **leading entry** or **pivot** — lies strictly to the right of the leading entry of the row above it (so the pivots form a descending "staircase").

Any matrix can be brought to row echelon form using finitely many elementary row operations — systematically eliminating entries below each pivot, top to bottom, is exactly **Gaussian elimination**. The number of nonzero rows in the resulting echelon form is called the **rank** of the matrix; by the Fact above, this number doesn't depend on which particular sequence of row operations you used to get there.

Remark: Turning This Into an Independence Test

Stack $v_1, \ldots, v_k \in \mathbb{R}^n$ as the k rows of a $k \times n$ matrix A, and row-reduce A to echelon form A'. Since row operations never change independence, $v_1, \ldots, v_k$ are linearly independent **exactly when A' has no zero row** — equivalently, when $\operatorname{rank}(A) = k$ (every row produced its own pivot). If instead a zero row appears, the original vectors are dependent — and the specific combination of row operations that produced that zero row *is* a witness: it records exactly which combination of the original rows sums to $\vec{0}$. The worked example below shows a clean way to read that combination off directly.

Example: Row Reduction in Action: Testing $v_1 = (1, 2, 3)$, $v_2 = (2, 3, 4)$, $v_3 = (1, 1, 1)$

Stack the vectors as rows, and — to make the eventual dependency relation easy to read off — augment the matrix on the right with the 3×3 identity, one column per original vector:

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 2 & 3 & 4 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 \end{array}\right] \begin{array}{l} \leftarrow v_1 \\ \leftarrow v_2 \\ \leftarrow v_3 \end{array}$$

The idea: whatever combination of rows we apply to the left block, we apply *identically* to the right block — so the right block always records "this row currently equals (this combination) of v_1, v_2, v_3."

Step 1: eliminate below the first pivot. Use R_1 (pivot in column 1) to clear column 1 in R_2 and R_3:

$$R_2 \to R_2 - 2R_1, \qquad R_3 \to R_3 - R_1.$$

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & -1 & -2 & -2 & 1 & 0 \\ 0 & -1 & -2 & -1 & 0 & 1 \end{array}\right]$$

Step 2: eliminate below the second pivot. R_2's leading entry is now in column 2; use it to clear column 2 in R_3:

$$R_3 \to R_3 - R_2.$$

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & -1 & -2 & -2 & 1 & 0 \\ 0 & 0 & 0 & 1 & -1 & 1 \end{array}\right]$$

This is row echelon form: two pivots (row 1 in column 1, row 2 in column 2), and one zero row — so $\text{rank} = 2 < 3 = k$, meaning v_1, v_2, v_3 **are linearly dependent**.

Reading off the dependency. Row 3's right-hand block, $(1, -1, 1)$, records exactly which combination of the *original* rows produced that zero row: $1 \cdot v_1 + (-1) \cdot v_2 + 1 \cdot v_3$. Check directly: $v_1 - v_2 + v_3 = (1, 2, 3) - (2, 3, 4) + (1, 1, 1) = (0, 0, 0)$ ✓. So $c_1 = 1,\ c_2 = -1,\ c_3 = 1$ is a non-trivial solution — equivalently, $v_2 = v_1 + v_3$, which you can also confirm directly.

Remark: Why More Vectors Than Dimensions Forces Dependence

If $k > n$ (more vectors than the ambient dimension), the vectors are *automatically* dependent — no computation needed. Setting up $c_1 v_1 + \cdots + c_k v_k = \vec{0}$ gives n equations (one per coordinate) in $k > n$ unknowns $c_1, \ldots, c_k$; row-reducing the coefficient matrix can produce at most n pivots (one per row), so at least $k - n$ of the unknowns are left as **free variables** — and a homogeneous system with a free variable always has a nontrivial solution (just set the free variable to 1 and solve for the rest). We will meet this exact fact again, in geometric disguise, once we reach lattices in Section 1.8.

Definition: Basis and Dimension

A set of vectors $B = \{b_1, \ldots, b_k\}$ is a **basis** of a vector space V if (1) B is linearly independent, and (2) B **spans** V (every vector in V can be written as a linear combination of B). Every basis of a given vector space has the same number of elements k, called the **dimension** of V.

Example: Basis of $\mathbb{R}^2$

The vectors $e_1 = (1,0)$ and $e_2 = (0,1)$ form the "standard" basis of $\mathbb{R}^2$: any vector (a,b) is exactly $a \cdot e_1 + b \cdot e_2$. But this is far from the *only* basis — $\{(1,1),(1,-1)\}$ is also a valid basis of $\mathbb{R}^2$ (check: they're linearly independent, and any (a,b) can be written as a combination of them). A vector space almost always has infinitely many possible bases; some are more convenient to work with than others. **Keep this fact in mind — it becomes very important in Section 1.8.**

Remark: A Quick Shortcut When $k = n$

When there are exactly as many vectors as dimensions, there's a fast special-case test that avoids row reduction entirely: stack $v_1, \ldots, v_n$ as the columns (or rows) of a square matrix A and compute $\det(A)$. Nonzero determinant $\Rightarrow$ independent; zero determinant $\Rightarrow$ dependent. This is a quick yes/no check, but — unlike the row-reduction method above — it doesn't by itself hand you the explicit nontrivial combination when the answer is "dependent"; for that, fall back on row reduction (optionally with the identity-augmentation trick above).

Exercise: Practice: Testing Linear Independence

For each set of vectors in $\mathbb{R}^3$ below, determine whether the set is linearly independent. If a set turns out to be **dependent**, don't just say so — exhibit an explicit **non-trivial** solution (scalars c_1, c_2, c_3, not all zero) to $c_1v_1 + c_2v_2 + c_3v_3 = \vec{0}$, and check it by substitution. Use row reduction (with the identity-augmentation trick from the worked example above) at least once, even where the answer is also visible by inspection.

(a) $v_1 = (2,4,6),\ v_2 = (1,2,3),\ v_3 = (0,1,0)$.
(b) $v_1 = (1,0,0),\ v_2 = (1,1,0),\ v_3 = (1,1,1)$.
(c) $v_1 = (1,2,1),\ v_2 = (2,1,3),\ v_3 = (4,5,5)$.

(Hint for part (c): unlike part (a), the dependency here isn't visible by inspection — row-reduce the augmented matrix $[\, v_1; v_2; v_3 \mid I_3 \,]$ exactly as in the worked example above.)

1.7.2 Inner Products, Norms, and Orthogonality

To make "good basis" (short, nearly-perpendicular vectors) versus "bad basis" (long, nearly-parallel vectors) precise rather than just intuitive, we need a way to measure length and angle.

Definition: Inner Product and Norm

For $x = (x_1, \ldots, x_n), y = (y_1, \ldots, y_n) \in \mathbb{R}^n$, the (standard) **inner product** is $\langle x, y \rangle = \sum_{i=1}^{n} x_i y_i$. The **norm** (length) of x is $\|x\| = \sqrt{\langle x, x \rangle}$. Two vectors are **orthogonal** if $\langle x, y \rangle = 0$ (the angle between them is $90°$). A basis $\{b_1, \ldots, b_k\}$ is an **orthogonal basis** if every pair b_i, b_j $(i \neq j)$ is orthogonal.

Example: Computing Norms and Checking Orthogonality

For $x = (3,4)$: $\|x\| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$. For $x = (1,1)$ and $y = (1,-1)$: $\langle x, y \rangle = (1)(1) + (1)(-1) = 0$, so these two vectors (the alternate basis of $\mathbb{R}^2$ from the previous example) are orthogonal — a nicer basis, geometrically, than it might have looked at first glance.

Remark: Extending the Inner Product to Complex Vectors

The formula $\langle x, y\rangle = \sum_i x_i y_i$ above is only correct for *real* vectors, $x, y \in \mathbb{R}^n$. It breaks down over $\mathbb{C}^n$: take a single complex coordinate, $x = (i)$. The formula gives $\langle x, x\rangle = i \cdot i = i^2 = -1$, so $\|x\| = \sqrt{-1}$ — not a real number at all, let alone a sensible "length." A norm must be a non-negative real number, so this naive formula simply stops being a norm once complex entries are allowed.

The standard fix is to place a **complex conjugate** on one argument,

$$\langle x, y\rangle = \sum_{i=1}^{n} x_i \overline{y_i} \qquad \text{(the } \textit{Hermitian} \text{ inner product)},$$

where $\overline{a + bi} = a - bi$. Now $\langle x, x\rangle = \sum_i x_i \overline{x_i} = \sum_i |x_i|^2$ — a sum of non-negative real numbers, always real and always ≥ 0, exactly what a norm needs. And whenever every entry happens to be real, $\overline{y_i} = y_i$, so this formula collapses back *exactly* to the plain dot product in the defbox above — the conjugate only ever does work when the entries are genuinely complex, and is invisible (a no-op) on real vectors. This is why the definition above was stated over $\mathbb{R}^n$ specifically: every lattice in Lectures 1–4 lives in $\mathbb{R}^n$ or $\mathbb{Z}^n$, so the plain real formula is all we need for now — the moment complex vectors enter the picture, however, conjugating becomes required, not optional.

Looking Ahead: Where Complex Vectors Actually Show Up (Preview of Falcon, Lecture 4)

This isn't a hypothetical fix-it-just-in-case. Falcon's signing step ("fast Fourier sampling," Lecture 4) embeds a ring element of $R_q = \mathbb{Z}_q[x]/(x^n+1)$ into $\mathbb{C}^n$ using the n complex roots of x^n+1 — the **canonical embedding** — because in that embedding, the otherwise-awkward polynomial multiplication in R_q becomes simple coordinate-wise multiplication of complex numbers, which is exactly what makes fast Fourier sampling fast. Every norm and inner product computed inside that embedding *is* the Hermitian version defined above, not the plain real dot product. Keep this box in mind when Falcon's sampler resurfaces in Session 5: part of why its floating-point implementation is so delicate to secure is that it's performing real, finite-precision arithmetic on quantities that are conceptually complex-number computations underneath.

Looking Ahead: Quantifying "Good" vs. "Bad" Bases

With norms in hand, we can finally give a real, measurable meaning to the "good basis / bad basis" idea flagged repeatedly in this lecture: a good basis has vectors with small norm $\|b_i\|$ that are close to orthogonal; a bad basis for the *same* lattice can have arbitrarily large norms and near-parallel vectors, despite generating an identical set of points. (Here L denotes the **lattice** generated by the basis $\{b_1, \ldots, b_n\}$ — all integer combinations of the b_i's, i.e. the grid of points they produce; we've been using this good-basis/bad-basis idea informally for a few pages now, and Section 1.8 below is where L finally gets a full, formal definition. For now, just read L as "the lattice this basis describes.") One common numerical measure of "how good" a basis is is its **orthogonality defect**, $\dfrac{\prod_i \|b_i\|}{\det(L)}$, where $\det(L)$ is a single positive number — the lattice's fundamental-cell volume — that depends only on L itself, not on which basis is used to describe it (defined precisely in Section 1.8.5). A good basis has vectors close in length to $\det(L)^{1/n}$ and close to orthogonal, which drives this ratio down toward its minimum possible value of 1; a bad basis inflates the numerator $\prod_i \|b_i\|$ without changing $\det(L)$ at all, so the ratio grows the more skewed the basis is.

1.7.3 Gram–Schmidt Orthogonalization

Given *any* basis, can we systematically produce an orthogonal one (spanning the same space, though generally *not* the same lattice — more on that distinction below)? Yes — and the entire idea rests on one simple geometric move: **taking a vector and subtracting off however much of it points along some other direction**, leaving only the part that's genuinely new. Let's make that move precise before writing down the full process.

Definition: Vector Projection

The **projection** of a vector v onto a (nonzero) vector u is

$$\mathrm{proj}_u(v) \;=\; \frac{\langle v, u\rangle}{\langle u, u\rangle}\, u.$$

Geometrically, $\mathrm{proj}_u(v)$ is the "shadow" v casts on the line through u — the closest point to v that lies on that line, i.e. the component of v pointing in the u direction. The leftover piece, $v - \mathrm{proj}_u(v)$, is exactly the component of v perpendicular to u: check that $\langle v - \mathrm{proj}_u(v),\, u\rangle = \langle v, u\rangle - \frac{\langle v,u\rangle}{\langle u,u\rangle}\langle u, u\rangle = 0$, confirming orthogonality directly from the definition.

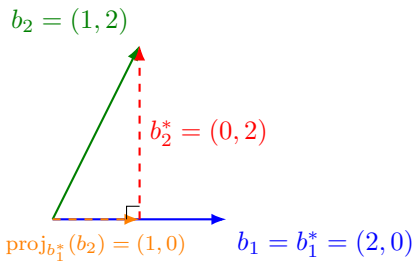


Figure 1.2: The geometry behind one Gram–Schmidt step (numbers match the worked example below). The green vector b_2 decomposes into an orange piece — its projection onto b_1^*, i.e. "how much of b_2 points along b_1^*" — and a red perpendicular leftover, $b_2^* = b_2 - \mathrm{proj}_{b_1^*}(b_2)$. That red leftover, translated back to the origin, *is* the new orthogonal basis vector. Every step of Gram–Schmidt is this same picture, repeated once per previous direction.

Definition: Gram–Schmidt Process

Given linearly independent $b_1, \ldots, b_k$, define $b_1^* = b_1$, and for $i = 2, \ldots, k$:

$$b_i^* = b_i - \sum_{j=1}^{i-1} \mu_{i,j}\, b_j^*, \qquad \text{where } \mu_{i,j} = \frac{\langle b_i, b_j^*\rangle}{\langle b_j^*, b_j^*\rangle}.$$

Each term $\mu_{i,j}\, b_j^*$ is precisely $\mathrm{proj}_{b_j^*}(b_i)$ from the defbox above — so this formula is nothing more than **"take b_i, and subtract off its projection onto every previously-built orthogonal direction, one at a time."** Each b_i^* is b_i with the "shadow" it casts on all previous b_j^* subtracted off, leaving only the genuinely new, orthogonal direction. The result $\{b_1^*, \ldots, b_k^*\}$ is an orthogonal basis of the same vector space (though, importantly, its integer combinations generally do *not* produce the same lattice as the original b_i's — Gram–Schmidt vectors are a vector-space tool, not automatically a lattice basis).

Example: Gram–Schmidt on a 2×2 Example

Let $b_1 = (2, 0)$, $b_2 = (1, 2)$ (the same vectors as Figure 1.2). Then $b_1^* = b_1 = (2, 0)$. Compute $\mu_{2,1} = \frac{\langle b_2, b_1^* \rangle}{\langle b_1^*, b_1^* \rangle} = \frac{(1)(2)+(2)(0)}{4} = \frac{2}{4} = 0.5$ — this is exactly $\text{proj}_{b_1^*}(b_2) = 0.5\,(2, 0) = (1, 0)$, the orange vector in the figure. So $b_2^* = b_2 - \text{proj}_{b_1^*}(b_2) = (1, 2) - (1, 0) = (0, 2)$ — the red leftover. Check: $\langle b_1^*, b_2^* \rangle = (2)(0) + (0)(2) = 0$ — orthogonal, as required.

Looking Ahead: Where Gram–Schmidt Leads

This process reappears, in a modified "integer-preserving" form, inside the **LLL algorithm** — the classic algorithm for turning a bad lattice basis into a reasonably good one, and a core tool used both to *attack* weak lattice-based schemes and to reason about the security of well-parametrized ones. We'll meet it properly in a later lecture; for now, just remember that Gram–Schmidt is the geometric heart of it.

Exercise: Practice

Run Gram–Schmidt on $b_1 = (1, 1)$, $b_2 = (0, 2)$: compute b_1^*, $\mu_{2,1}$, and b_2^*, and verify $\langle b_1^*, b_2^* \rangle = 0$.

1.8 Lattices: A Formal Definition

Everything so far has allowed *any* scalar from the field F (e.g., any real number) when forming linear combinations. Informally, a lattice is what happens when we impose one dramatic restriction: **only integer combinations are allowed**, which turns a solid, continuous vector space into a discrete grid of isolated points. That informal picture (Figure 1.4 below) is correct and worth keeping in your head. But the definition we will actually use — the one used in serious treatments of the subject, such as Chris Peikert's graduate lecture notes *Lattices in Cryptography*[1] — starts from the **subgroup** idea from Section 1.3.1, not from a basis. We will show the two views agree.

1.8.1 Discrete Sets

Definition: Discrete Subset

A subset $S \subseteq \mathbb{R}^n$ is **discrete** if every point of S is *isolated*: for every $x \in S$, there exists some $\varepsilon > 0$ such that the open ball $B(x, \varepsilon)$ (all points within distance ε of x) contains *no* point of S other than x itself. Informally: you can always draw a small enough circle around any point of S that catches nothing else from S.

Remark: A Shortcut for Subgroups

Checking every single point of a set for isolation sounds like a lot of work. But if H is a *subgroup* of $(\mathbb{R}^n, +)$ (Section 1.3.1), there's a shortcut: it suffices to check isolation *at the origin only* — H is discrete if and only if there is some $\varepsilon > 0$ with $B(\vec{0}, \varepsilon) \cap H = \{\vec{0}\}$. (This is a statement about how much *checking* is needed, not about how many points end up isolated: as the argument below shows, once the origin passes this test, *every* point of H turns out to be isolated too — not just the origin.) *Why this works:* suppose $\vec{0}$ is isolated with radius ε, and suppose some other point $y \in H$ were within ε of some $x \in H$. Since H is a subgroup, $y - x \in H$ as well, and $y - x$ is within ε of $\vec{0}$ — so by our assumption, $y - x = \vec{0}$, i.e., $y = x$. So checking one point (the origin) is enough to certify the whole subgroup is discrete, thanks to closure under subtraction.

[1] C. Peikert, *Lattices in Cryptography*, lecture notes, https://github.com/cpeikert/LatticesInCryptography.

1.8.2 The Definition

Definition: Lattice

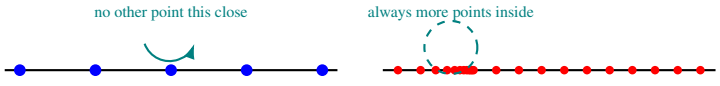
A **lattice** is a discrete additive subgroup L of $\mathbb{R}^n$. That is, $L \subseteq \mathbb{R}^n$ is a lattice if:

(L1) L is a subgroup of $(\mathbb{R}^n, +)$ (Section 1.3.1): it contains $\vec{0}$, and is closed under addition and negation.

(L2) L is discrete: by the shortcut above, there exists $\varepsilon > 0$ such that $B(\vec{0}, \varepsilon) \cap L = \{\vec{0}\}$.

Example: $\mathbb{Q}$ Is a Subgroup, But *Not* a Lattice

Consider $\mathbb{Q} \subseteq \mathbb{R}$ (the rational numbers) under addition. This *is* a subgroup: it contains 0, the sum of two rationals is rational, and $-a$ is rational whenever a is. So condition (L1) holds. But condition (L2) fails badly: the rationals are **dense** in $\mathbb{R}$, meaning every open ball around 0, no matter how tiny, contains infinitely many other rational numbers. There is no $\varepsilon > 0$ that isolates 0. So $\mathbb{Q}$ is a perfectly good subgroup that is **not** a lattice — this is exactly why condition (L2) has to be part of the definition; subgroup alone is not a strong enough requirement.



$\mathbb{Z}$ — discrete: points have gaps $\mathbb{Q}$ — dense: no gaps, ever

Figure 1.3: Discreteness, visualized. Left: in $\mathbb{Z}$, every point has a gap of size 1 around it with nothing else nearby — discrete. Right: in $\mathbb{Q}$, no matter how small a circle you draw around a point, there are always infinitely many more rational points crammed inside it — dense, and therefore *not* discrete.

Example: $\mathbb{Z}$ and $\mathbb{Z}^n$ Are Lattices

$\mathbb{Z} \subseteq \mathbb{R}$ is a subgroup (trivially — and recall from Section 1.3.1 that $n\mathbb{Z}$ is a subgroup of $\mathbb{Z}$ for any n; here we're just looking at $\mathbb{Z}$ itself sitting inside $\mathbb{R}$), and it is discrete: taking $\varepsilon = 0.5$, $B(0, 0.5) \cap \mathbb{Z} = \{0\}$, since the nearest other integers (± 1) are a full unit away. So $\mathbb{Z}$ is a lattice in $\mathbb{R}$ — the simplest possible example. The same argument, coordinate by coordinate, shows $\mathbb{Z}^n \subseteq \mathbb{R}^n$ (all integer-coordinate points) is a lattice in $\mathbb{R}^n$; it is often called **the integer lattice**.

1.8.3 The Shortest Vector and the Minimum Distance $\lambda_1(L)$

Before going further, it's worth naming one basic quantity attached to every lattice: how close together its points actually are. We will use this constantly for the rest of the lecture (and the rest of the course), so it earns a formal definition rather than staying an informal aside.

Definition: Shortest Vector and Minimum Distance $\lambda_1(L)$

Let $L \subseteq \mathbb{R}^n$ be a lattice. A **shortest vector** of L is a nonzero vector $v \in L$ whose norm is minimal among all nonzero lattice vectors. The length of a shortest vector is called the lattice's **minimum distance**, or **first successive minimum**, and is defined by

$$\lambda_1(L) = \min_{v \in L \setminus \{\vec{0}\}} \|v\|.$$

The subscript "1" is intentional. More generally, the **successive minima**

$$\lambda_1(L) \leq \lambda_2(L) \leq \cdots \leq \lambda_n(L)$$

describe the minimum lengths needed to obtain $1, 2, \ldots, n$ linearly independent lattice vectors, respectively. In particular, $\lambda_i(L)$ is the smallest radius r such that the ball

$$B_n(\vec{0}, r) = \{x \in \mathbb{R}^n : \|x\| \leq r\}$$

contains i linearly independent vectors of L.
Thus, $\lambda_1(L)$ is simply the length of a shortest nonzero lattice vector. In this course, we will only need $\lambda_1(L)$ until the optional **Minkowski's Second Theorem** box near the end of this section.

Remark: Why the Minimum Is Actually Attained (Not Just an Infimum)

For a general infinite set, "the smallest nonzero norm" might not literally be achieved by any single element (imagine values getting closer and closer to some limit without ever reaching it). This cannot happen for a lattice, precisely *because* L is discrete (Section 1.8.1): discreteness guarantees a gap $\varepsilon > 0$ around the origin with no other lattice points inside it, so only *finitely many* lattice points can lie in any bounded region — and the minimum of a norm over a finite set is always achieved. This is also exactly the first step of the "harder direction" proof sketch below (Section 1.8.4): $\lambda_1(L)$ isn't just a number we can talk about, it's a specific vector we can (in principle) point to.

Remark: Why $\lambda_1(L)$ Matters

Computing $\lambda_1(L)$ exactly — or even just estimating it well — for a lattice given only a (possibly bad) basis is called the **Shortest Vector Problem (SVP)**, and it is believed to be extremely hard once the dimension n is large. This hardness is one of the two or three core hard problems that lattice-based cryptography's security ultimately rests on; we will state SVP formally, and meet its hardness in practice, once we reach the LWE problem in the next lecture.

1.8.4 Recovering the Basis Picture

The subgroup definition is more general and more standard, but the "grid generated by a basis" picture you may already have in your head is not wrong — it is a theorem, not a definition, and it's worth seeing why.

Fact: Basis Representation of Lattices

Let $L \subseteq \mathbb{R}^n$. Then L is a lattice (in the sense of the subgroup definition above, i.e., a discrete additive subgroup) **if and only if** there exist linearly independent vectors $b_1, \ldots, b_k \in \mathbb{R}^n$ (with $k \leq n$) such that

$$L = \mathcal{L}(b_1, \ldots, b_k) = \left\{ \sum_{i=1}^{k} a_i b_i \;\middle|\; a_i \in \mathbb{Z} \right\}.$$

The vectors $b_1, \ldots, b_k$ are called a **basis** of L; the number k is the **rank** of L. If $k = n$ (the basis spans all of $\mathbb{R}^n$), L is called **full-rank** — this is the common case in cryptography.

Remark: Proof Idea (Sketch Only)

Easy direction ($\Leftarrow$): given linearly independent $b_1, \ldots, b_k$, the set of integer combinations is clearly a subgroup (adding two integer combinations, or negating one, gives another integer combination). Discreteness holds because the b_i point in genuinely independent directions: you cannot make $\sum a_i b_i$ arbitrarily close to $\vec{0}$ without every a_i being exactly 0 — linear independence rules out any "near-cancellation." (Making this fully rigorous uses the fact that the linear map $(a_1, \ldots, a_k) \mapsto \sum a_i b_i$ has a bounded inverse on its image.)
Harder direction ($\Rightarrow$): given only that L is a discrete subgroup, we must *construct* a basis. The idea is greedy and inductive: pick b_1 to be a shortest nonzero vector in L — exactly $\lambda_1(L)$ from the defbox above, which exists precisely *because* L is discrete (in a non-discrete set like $\mathbb{Q}$, there is no shortest nonzero element, since you can always find a smaller one). Then look at the part of L not lying on the line through b_1, and pick b_2 to be a shortest vector there, and so on. A careful argument (using discreteness at each step, and projecting onto the space orthogonal to the vectors already chosen) shows this process terminates with a valid basis after $k \leq n$ steps. The full details are carried out rigorously in Peikert's notes; we take the result as given here.

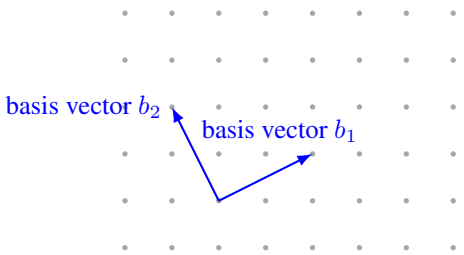


Figure 1.4: A 2-dimensional, full-rank lattice generated by a basis $\{b_1, b_2\}$: every grid dot is reachable by some *integer* combination $a_1 b_1 + a_2 b_2$ (with $a_1, a_2 \in \mathbb{Z}$), starting from any chosen origin dot. This is the same object as the subgroup definition above — just viewed through its basis instead of as an abstract subgroup.

Remark: Same Lattice, Different Basis

A single lattice has *many* different valid bases (just as a vector space does — recall the "Basis of $\mathbb{R}^2$" example in Section 1.7). Two bases generate the exact same set of points if and only if one can be converted to the other using an integer matrix with determinant ± 1 (a **unimodular** transformation) — we won't need the details of this fact yet, but the consequence is critical: some bases consist of short, nearly-perpendicular vectors (a "good" basis, easy to compute with), while other, equally valid bases for the *same* lattice consist of long, nearly-parallel vectors (a "bad" basis, hard to compute with). **This gap between good and bad bases for the same lattice is the single most important idea in lattice-based cryptography**: the secret key is typically a good (short) basis, while the public key describes the same lattice using a bad (long, scrambled-looking) basis. Recovering the good basis from the bad one is believed to be extremely hard once the dimension n is large enough.

Example: A Tiny Lattice by Hand

Let $b_1 = (2, 0)$ and $b_2 = (0, 3)$ in $\mathbb{R}^2$. The lattice $\mathcal{L}(b_1, b_2)$ consists of every point $(2a_1, 3a_2)$ for integers a_1, a_2 — so it includes $(0, 0)$, $(2, 0)$, $(-2, 0)$, $(0, 3)$, $(2, 3)$, $(4, -3)$, and so on, but *not* $(1, 1)$ or $(1, 0)$, since those cannot be written as $(2a_1, 3a_2)$ for integer a_1, a_2. As a sanity check against the subgroup definition directly: is this set a subgroup? Yes — closed under addition and negation, contains $(0, 0)$. Is it discrete? Yes — the nearest point to the origin is at distance 2, so any $\varepsilon < 2$ isolates $\vec{0}$. (Equivalently: $\lambda_1(L) = 2$, achieved by b_1 itself.)

1.8.5 The Determinant (Volume) of a Lattice

One more piece of structure will matter a great deal in later lectures: every full-rank lattice has a well-defined "volume," independent of which basis is used to describe it.

Definition: Fundamental Domain and Determinant

Let $L \subseteq \mathbb{R}^n$ be a full-rank lattice with basis $b_1, \ldots, b_n$, and let B be the matrix whose columns are $b_1, \ldots, b_n$. The **fundamental parallelepiped** associated with this basis is

$$\mathcal{P}(B) = \left\{ \sum_{i=1}^{n} t_i b_i \;\middle|\; 0 \le t_i < 1 \right\}.$$

Here, each coefficient t_i can be *any real number* in the interval $[0, 1)$. Thus, $\mathcal{P}(B)$ consists of all fractional linear combinations of the basis vectors, not merely combinations using coefficients 0 or 1. The **determinant** of the lattice L, denoted by $\det(L)$, is the volume of its fundamental parallelepiped:

$$\det(L) = \mathrm{vol}(\mathcal{P}(B)) = |\det(B)|.$$

The fundamental parallelepiped can be viewed as a basic "tile" for the entire space. Copies of $\mathcal{P}(B)$, shifted by every lattice point $\ell \in L$, tile all of $\mathbb{R}^n$ *exactly*: every point of $\mathbb{R}^n$ lies in *one and only one* translated copy, with no overlap at all. (This is precisely why the definition uses the half-open condition $0 \le t_i < 1$ rather than $0 \le t_i \le 1$: it guarantees each point has a unique representative $p \in \mathcal{P}(B)$, avoiding double-counting along shared faces.) Formally,

$$\mathbb{R}^n = \bigsqcup_{\ell \in L} (\ell + \mathcal{P}(B)),$$

where $\bigsqcup$ denotes a disjoint union. Here, **shifting** (or **translating**) $\mathcal{P}(B)$ by a lattice point ℓ simply means adding ℓ to every point in the parallelepiped:

$$\ell + \mathcal{P}(B) = \{\ell + x \mid x \in \mathcal{P}(B)\}.$$

The shape and volume do not change; only the location changes. **Example: the lattice $\mathbb{Z}^2$.** Consider the lattice

$$L = \mathbb{Z}^2$$

with basis

$$b_1 = (1, 0), \qquad b_2 = (0, 1).$$

According to the definition,

$$\mathcal{P}(B) = \{t_1 b_1 + t_2 b_2 \mid 0 \le t_1, t_2 < 1\}.$$

Substituting the basis vectors gives

$$\begin{aligned} t_1 b_1 + t_2 b_2 &= t_1(1, 0) + t_2(0, 1) \\ &= (t_1, 0) + (0, t_2) \\ &= (t_1, t_2). \end{aligned}$$

Therefore,

$$\mathcal{P}(B) = \{(t_1, t_2) \mid 0 \le t_1, t_2 < 1\} = [0, 1) \times [0, 1).$$

Notice that t_1 and t_2 are not restricted to the values 0 and 1. They can take any real values between 0 and 1. For example,

$$0.3b_1 + 0.7b_2 = (0.3, 0.7) \in \mathcal{P}(B).$$

Thus, $\mathcal{P}(B)$ contains every point in the unit square $[0, 1) \times [0, 1)$. Now consider the lattice point

$$\ell = (2, 1).$$

Shifting the fundamental parallelepiped by ℓ gives

$$\ell + \mathcal{P}(B) = [2, 3) \times [1, 2).$$

For example,

$$(0, 0) \mapsto (2, 1), \qquad (0.3, 0.7) \mapsto (2.3, 1.7).$$

Thus, shifting by $(2, 1)$ simply moves the entire unit square two units to the right and one unit upward. Notice that $\mathcal{P}(B) = [0, 1) \times [0, 1)$ and $\ell + \mathcal{P}(B) = [2, 3) \times [1, 2)$ are entirely different sets of points: they do not share any point in common, illustrating the disjointness of the tiling above. More generally, every point $x \in \mathbb{R}^n$ can be written *uniquely* as

$$x = \ell + p, \qquad \ell \in L, \quad p \in \mathcal{P}(B).$$

The lattice point ℓ determines the translated copy of the fundamental parallelepiped, while p determines the position of x within that copy. Since translation does not change volume, every translated copy $\ell + \mathcal{P}(B)$ has the same volume as $\mathcal{P}(B)$. Thus, the determinant of the lattice can be interpreted geometrically as the volume of one fundamental "cell" of the lattice:

$$\boxed{\det(L) = |\det(B)|.}$$

Remark: Why $\det(L)$ Doesn't Depend on Which Basis You Pick

Recall from the "Same Lattice, Different Basis" remark above: any two bases of the same lattice are related by a unimodular matrix U (integer entries, $\det(U) = \pm 1$), i.e., $B' = UB$. Then $\det(B') = \det(U)\det(B) = \pm\det(B)$, so $|\det(B')| = |\det(B)|$. This confirms $\det(L)$ is a genuine property of the lattice itself, not an artifact of which basis happened to be used to compute it — exactly the kind of basis-independence you should expect from something called "the volume of the lattice."

Example: Computing $\det(L)$ for the Tiny Lattice

For $b_1 = (2,0)$, $b_2 = (0,3)$ from the example above, $B = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$, so $\det(L) = |\det(B)| = |2 \times 3 - 0 \times 0| = 6$. Geometrically: the fundamental parallelepiped is just the 2×3 rectangle $[0,2) \times [0,3)$, and copies of this rectangle exactly tile the plane, anchored at every lattice point — matching the picture in Figure 1.4's style, just for this specific (axis-aligned) lattice.

1.8.6 Unimodular Equivalence: When Do Two Bases Give the Same Lattice?

Example: Same Lattice, or Different? A Worked Test Using the Unimodular Criterion

Let $B = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}$ (columns $b_1 = (2,1), b_2 = (1,1)$) and $B' = \begin{pmatrix} 1 & 3 \\ 0 & 1 \end{pmatrix}$ (columns $b_1' = (1,0), b_2' = (3,1)$) be two candidate bases in $\mathbb{R}^2$. Do they generate the same lattice?

Confirming case. Suppose instead $B' = \begin{pmatrix} 3 & 1 \\ 2 & 1 \end{pmatrix}$. Since B is invertible ($\det B = 2 - 1 = 1 \neq 0$), solve $U = B^{-1}B'$: $B^{-1} = \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix}$ (check: $BB^{-1} = I$). Then

$$U = B^{-1}B' = \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix} \begin{pmatrix} 3 & 1 \\ 2 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}.$$

U has integer entries and $\det(U) = 1 \times 1 - 0 \times 1 = 1$, so $|\det(U)| = 1$ — U **is unimodular**. Since $B' = BU$ with U unimodular, B and B' generate **the same lattice**.

Contrasting case. Now go back to the original $B' = \begin{pmatrix} 1 & 3 \\ 0 & 1 \end{pmatrix}$. Compute $U = B^{-1}B'$:

$$U = \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix} \begin{pmatrix} 1 & 3 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ -1 & -1 \end{pmatrix}.$$

Still integer entries, but $\det(U) = (1)(-1) - (2)(-1) = -1 + 2 = 1$ — wait, this is also unimodular; let's instead deliberately pick a genuinely different lattice to illustrate the failing case: $B'' = \begin{pmatrix} 1 & 3 \\ 0 & 2 \end{pmatrix}$. Then

$$U = B^{-1}B'' = \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix} \begin{pmatrix} 1 & 3 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix},$$

with $\det(U) = (1)(1) - (1)(-1) = 2$. Since $|\det(U)| = 2 \neq 1$, U is **not** unimodular — B and B'' generate *different* lattices, and in fact $\mathcal{L}(B'')$ turns out to be a proper sublattice of $\mathcal{L}(B)$ of **index** 2 (every point of $\mathcal{L}(B'')$ is a point of $\mathcal{L}(B)$, but only half of $\mathcal{L}(B)$'s points are recovered this way) — a direct consequence of $|\det(U)|$ measuring exactly this kind of "index" between the two lattices, matching $\det(L) = |\det(B)|$ scaling by the same factor: $\det(\mathcal{L}(B'')) = |\det(U)| \cdot \det(\mathcal{L}(B)) = 2\det(\mathcal{L}(B))$.

Remark: Discrete Implies Countable, But Not Conversely

It's worth clarifying a subtle point about the discreteness condition (L2) in the Lattice definition: **every discrete subset of $\mathbb{R}^n$ is automatically countable**, but the converse is false — being countable does not force a set to be discrete. *Why discrete $\Rightarrow$ countable:* around each point of a discrete set, by definition, sits a small ball containing no other point of the set; these disjoint balls can be shrunk enough to have rational-coordinate centers and rational radii, and there are only countably many such balls in total (since $\mathbb{Q}^n$ is countable) — so there can be at most countably many points of the discrete set, one per ball. *Why the converse fails:* $\mathbb{Q} \subseteq \mathbb{R}$ is a textbook counterexample — it is countable (the rationals can be listed $q_1, q_2, q_3, \ldots$), yet, as the exbox above on "$\mathbb{Q}$ Is a Subgroup, But Not a Lattice" already showed, $\mathbb{Q}$ is *dense* in $\mathbb{R}$, not discrete: every open ball around any point contains infinitely many other rational points. So countability is a necessary, but nowhere near sufficient, condition for discreteness — exactly why the Lattice definition needs the stronger, explicitly geometric condition (L2), rather than settling for the weaker "countable" in its place.

1.8.7 Minkowski's First Theorem: Volume Forces a Short Vector to Exist

We now have the two basic numbers attached to any lattice: $\lambda_1(L)$ (how short its shortest vector is) and $\det(L)$ (how much volume, on average, surrounds each lattice point). Minkowski's First Theorem is the single most important fact connecting them — and it flips the usual question around. So far, "finding $\lambda_1(L)$" has meant: given a specific basis, go and search for a short vector. Minkowski's theorem asks instead: *without searching at all*, can we predict, from $\det(L)$ alone, an upper bound on how long $\lambda_1(L)$ is *allowed* to be? Surprisingly, yes — and, remarkably, the argument is pure geometry, with no algebra and no knowledge of any particular basis required.

The theorem is about regions $K \subseteq \mathbb{R}^n$ with two specific shape properties, **symmetric** and **convex**, both used informally already in this course but worth pinning down precisely before they appear inside a theorem statement.

Definition: Symmetric Region (Origin-Symmetric / Centrally Symmetric)

A region $K \subseteq \mathbb{R}^n$ is **symmetric** (short for *origin-symmetric*, sometimes called *centrally symmetric* or *point-symmetric*) if

$$K = -K, \qquad \text{i.e.,} \qquad x \in K \implies -x \in K.$$

In words: whenever a point x belongs to K, its reflection through the origin, $-x$, must belong to K too. Informally, K "looks the same" if you rotate it $180°$ about the origin — a disk or square *centered at the origin* is symmetric in this sense, but the same disk or square shifted off-center is not (its reflection through the origin lands somewhere else entirely).

Definition: Convex Region

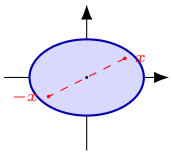
A region $K \subseteq \mathbb{R}^n$ is **convex** if, for every pair of points $x, y \in K$, the entire straight line segment joining them,

$$\{\, tx + (1-t)y \;:\; t \in [0,1] \,\},$$

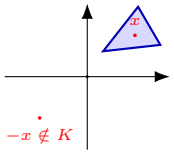
also lies inside K. Informally: a convex region has no "dents," "notches," or "holes" — you can never draw a straight line between two points of K that has to leave K and come back. Disks, squares, triangles, and ellipses are all convex; rings (annuli), crescents, and star shapes generally are not.

Remark: The Two Properties Are Independent

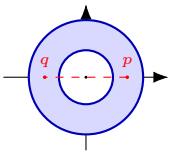
Symmetric and convex describe two *different* things about a region's shape — one is about "does it look the same reflected through the origin," the other is about "does it have any dents or holes" — so a region can have either property without the other, both, or neither. Minkowski's theorem needs *both at once*: Figure 1.5 below shows one example of each of the four possible combinations.



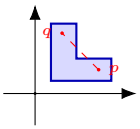
(a) Symmetric *and* convex — e.g. a disk or ellipse centered at the origin.



(b) Convex, but *not* symmetric — $x \in K$ but $-x \notin K$.



(c) Symmetric, but *not* convex — the ring is unchanged by $x \mapsto -x$, but the segment pq cuts through the empty hole.



(d) Neither symmetric nor convex — an off-center, notched ("L-shaped") region.

Figure 1.5: The four combinations of symmetric / not, convex / not. In (a), every point's reflection $-x$ is also in K, and every segment between two points of K stays inside K — this is the shape Minkowski's theorem needs. In (b), the triangle sits entirely in the upper-right, so a point $x \in K$ has its reflection $-x$ landing in the empty lower-left — symmetry fails, even though the triangle itself has no dents. In (c), the ring *is* symmetric (it looks the same rotated $180°$), but it fails convexity: the straight segment between the two marked points p, q passes through the hollow center, which is not part of the ring. In (d), the region is both off-center (so $x \mapsto -x$ maps points outside K) and has an inward notch (so some segments between points of K leave K), failing both properties at once.

Remark: The Idea in One Sentence, Before Any Symbols

If a symmetric, convex region around the origin is made large enough relative to $\det(L)$, it becomes *geometrically impossible* for that region to avoid every nonzero lattice point — so one must be hiding inside it, whether or not you know where.

Definition: Minkowski's First Theorem

Let $L \subseteq \mathbb{R}^n$ be a full-rank lattice, and let $K \subseteq \mathbb{R}^n$ be a **symmetric** ($K = -K$, i.e. $x \in K \Rightarrow -x \in K$) **convex** region. If

$$\mathrm{Vol}(K) \;>\; 2^n \det(L),$$

then K contains a nonzero lattice point, i.e. $K \cap (L \setminus \{\vec{0}\}) \neq \varnothing$.

Remark: Reading the Theorem in Plain Language, Term by Term

- $\det(L)$, as in Section 1.8.5, is "how much empty space, on average, surrounds each lattice point."
- $2^n \det(L)$ is a fixed volume threshold, scaled up from $\det(L)$ by a factor that grows with the dimension n (we will see exactly where this factor comes from in the proof below).
- **Symmetric and convex** are conditions on the *shape* of K — a disk or an ellipsoid centered at the origin both qualify; an oddly-shaped or off-center region generally does not.
- **The conclusion** is purely existential: once K's volume clears the threshold, K is *guaranteed* to contain some nonzero point of L — but the theorem does not say which point, or how to find it. That "existence without construction" flavor is typical of the deep facts underlying lattice cryptography, and it's worth getting comfortable with that style of guarantee now.

Remark: A Loose End First: Does K Always Contain the Origin?

Before turning to the proof, it's worth settling a question the theorem's statement doesn't address directly: could a symmetric convex region fail to contain the origin at all? No — and this is worth proving explicitly, because it removes a genuine ambiguity later.

Claim: any nonempty symmetric convex set K contains $\vec{0}$. Proof: since K is nonempty, pick any point $x \in K$. By symmetry ($K = -K$), $-x \in K$ too. By convexity, the line segment joining x and $-x$ lies entirely in K — in particular its midpoint, $\frac{1}{2}(x + (-x)) = \vec{0}$, is in K. ■

So $\vec{0} \in K$ is automatic, for *every* symmetric convex K, regardless of its size — it has nothing to do with the volume threshold $2^n \det(L)$ in the theorem. The origin is never at risk of being excluded, so the theorem's volume condition can't be about keeping the origin in; it must be doing something else. That something else is the theorem's real content: guaranteeing a *second*, *nonzero* lattice point, which is not automatic, and genuinely does depend on K being large enough.

A Fully Formal Proof, via Blichfeldt's Lemma

Remark: Proof Roadmap

The proof splits cleanly into two independent pieces, and it helps to see the two-step strategy before working through either one in detail:

1. **Blichfeldt's Lemma** (a pure volume/pigeonhole argument): *any* region with volume bigger than $\det(L)$ — symmetric, convex, or neither, it doesn't matter — must contain two distinct points whose difference is a nonzero lattice vector. This step never looks at the shape of the region at all.
2. **Bringing back symmetry and convexity**: apply Blichfeldt's Lemma not to K itself, but to the *half-sized* copy $S = \frac{1}{2}K$. The difference of the two points Blichfeldt hands us then turns out, *because* K is symmetric and convex, to land back inside K itself — which is exactly the nonzero lattice point in K that the theorem promises. This is also where the factor of 2^n in the theorem's statement comes from: shrinking K down to $S = \frac{1}{2}K$ divides its volume by 2^n, so S only clears Blichfeldt's threshold ($\mathrm{Vol}(S) > \det(L)$) once $\mathrm{Vol}(K) > 2^n \det(L)$ to begin with.

Fact: Blichfeldt's Lemma

Let $L \subseteq \mathbb{R}^n$ be a full-rank lattice with fundamental domain $\mathcal{P} = \mathcal{P}(B)$ (Section 1.8.5), so $\mathrm{Vol}(\mathcal{P}) = \det(L)$. Let $S \subseteq \mathbb{R}^n$ be any measurable set with $\mathrm{Vol}(S) > \det(L)$. Then there exist two **distinct** points $p, q \in S$ such that $p - q \in L \setminus \{\vec{0}\}$.

Remark: Proof of Blichfeldt's Lemma

Step 1: Cut S into pieces, one per lattice point, and fold every piece back to the origin. For each lattice point $v \in L$, consider the piece of S that lands in the copy of the fundamental domain sitting at v, i.e. $S \cap (v + \mathcal{P})$, and **fold it back** to the origin's copy by subtracting v:

$$S_v \;=\; \big(S \cap (v + \mathcal{P})\big) - v \;\subseteq\; \mathcal{P}.$$

Step 2: The folded pieces' volumes must add up to more than $\mathrm{Vol}(\mathcal{P})$. Because the translates $\{v + \mathcal{P} : v \in L\}$ tile $\mathbb{R}^n$ with no gaps or overlaps (Section 1.8.5), every point of S lies in *exactly one* $v + \mathcal{P}$, so the sets $S \cap (v + \mathcal{P})$, as v ranges over L, partition S exactly. Folding back (a rigid shift) doesn't change volume, so

$$\sum_{v \in L} \mathrm{Vol}(S_v) \;=\; \sum_{v \in L} \mathrm{Vol}\big(S \cap (v + \mathcal{P})\big) \;=\; \mathrm{Vol}(S) \;>\; \det(L) \;=\; \mathrm{Vol}(\mathcal{P}).$$

Step 3: Too much volume packed into one region forces an overlap (pigeonhole). All the S_v's live inside the single region $\mathcal{P}$. If they were all pairwise disjoint, the sum of their volumes could be at most $\mathrm{Vol}(\mathcal{P})$ — but Step 2 just showed that sum is *strictly greater* than $\mathrm{Vol}(\mathcal{P})$. Contradiction. So some two of them, S_{v_1} and S_{v_2} with $v_1 \neq v_2$, must overlap: there is a point $y \in S_{v_1} \cap S_{v_2}$.

Step 4: Unfold the overlap to recover two points of S whose difference is a lattice vector. By definition of S_v, $y = p - v_1$ for some $p \in S \cap (v_1 + \mathcal{P})$, and $y = q - v_2$ for some $q \in S \cap (v_2 + \mathcal{P})$. Both $p, q \in S$. Since $p - v_1 = q - v_2$, we get

$$p - q \;=\; v_1 - v_2 \;\in\; L \setminus \{\vec{0}\}$$

(nonzero because $v_1 \neq v_2$, and the difference of two lattice points is again a lattice point, by L's subgroup closure from Section 1.3.1). This is exactly the pair the lemma promised. ■

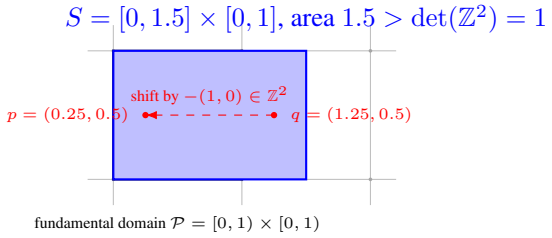


Figure 1.6: Blichfeldt's Lemma in action for $L = \mathbb{Z}^2$. S (blue) has area 1.5, exceeding $\det(L) = 1$, so it cannot fit inside one fundamental domain without "spilling over." The spillover — the sliver of S with $x \in [1, 1.5]$ — folds back (shift by $-(1, 0)$) into the same unit square as the rest of S, and lands on top of it: the two marked points $p, q \in S$ fold to the *same* location, and $p - q = (-1, 0)$ is a nonzero lattice vector, exactly as the lemma guarantees.

Example: Blichfeldt's Lemma, Checked by Hand

Take $L = \mathbb{Z}^2$ ($\det(L) = 1$) and $S = [0, 1.5] \times [0, 1]$, area $1.5 > 1$. Following the proof: the piece of S in the fundamental domain $\mathcal{P} = [0, 1) \times [0, 1)$ itself is all of $[0, 1) \times [0, 1)$ (area 1); the piece of S in the neighboring copy $v + \mathcal{P}$ for $v = (1, 0)$ is $[1, 1.5] \times [0, 1)$ (area 0.5), which folds back to $[0, 0.5) \times [0, 1)$ after subtracting v. These two folded pieces — area 1 and area 0.5, total $1.5 > 1 = \mathrm{Vol}(\mathcal{P})$ — cannot both fit disjointly inside $\mathcal{P}$, so they overlap on $[0, 0.5) \times [0, 1)$. Picking any point there, e.g. $(0.25, 0.5)$: this equals $p - (0, 0) = p$ for $p = (0.25, 0.5) \in S$, and also equals $q - (1, 0)$ for $q = (1.25, 0.5) \in S$. Indeed $p, q \in S$ (check: $0.25, 1.25 \in [0, 1.5]$, $0.5 \in [0, 1]$, both confirmed), and $p - q = (-1, 0) \in \mathbb{Z}^2 \setminus \{\vec{0}\}$ — exactly as guaranteed, and matching Figure 1.6.

Remark: Deriving Minkowski's Theorem from Blichfeldt's Lemma

Now bring back symmetry and convexity to finish the proof, following Step 2 of the roadmap above. Let K be symmetric and convex with $\mathrm{Vol}(K) > 2^n \det(L)$.

Set up the shrunk region. Define $S = \frac{1}{2}K = \{\frac{1}{2}x : x \in K\}$. Volume scales with the n-th power of a linear scale factor, so $\mathrm{Vol}(S) = \mathrm{Vol}(K)/2^n > \det(L)$ — exactly the hypothesis Blichfeldt's Lemma needs.

Apply Blichfeldt's Lemma to S. There exist distinct $p, q \in S$ with $p - q \in L \setminus \{\vec{0}\}$. Since $p, q \in S = \frac{1}{2}K$, write $p = \frac{1}{2}x$ and $q = \frac{1}{2}y$ for (unique) $x, y \in K$. Then

$$p - q \;=\; \tfrac{1}{2}(x - y) \;=\; \tfrac{1}{2}\big(x + (-y)\big).$$

Show this difference lands back inside K. Since K is symmetric, $-y \in K$. Since K is convex, the midpoint of x and $-y$ — which is exactly $\frac{1}{2}(x + (-y)) = p - q$ — lies in K. So $p - q$ is simultaneously:

- a nonzero lattice vector (from Blichfeldt's Lemma), and
- a point of K (from symmetry and convexity, just shown).

Therefore $p - q \in K \cap (L \setminus \{\vec{0}\})$ — a nonzero lattice point inside K, exactly as the theorem claims. ■

Remark: Tying the Two Halves Together

Notice how cleanly the labor divides. Blichfeldt's Lemma is the *engine*: pure volume-counting, no geometry of K's shape involved at all — it would work verbatim for *any* measurable S, symmetric or not, convex or not, as long as it's simply big enough. Symmetry and convexity only enter in the very last step, to upgrade the raw difference $p - q$ (which Blichfeldt hands you for free) into a point that's *also* guaranteed to lie inside K itself, not just somewhere in $\mathbb{R}^n$. This is worth remembering: it's the reason Minkowski's theorem needs *both* hypotheses (symmetric *and* convex) and not just "big enough" — drop either one, and the last step's midpoint trick breaks, even though Blichfeldt's Lemma itself still holds regardless.

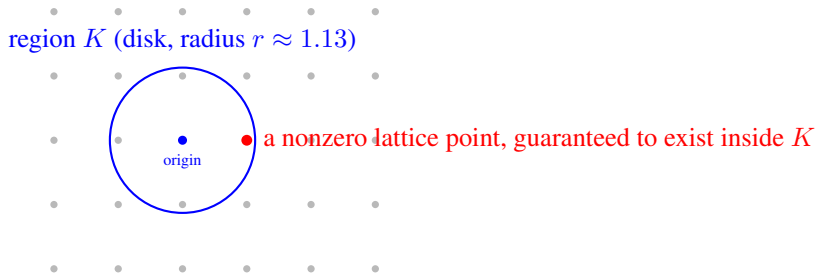


Figure 1.7: Minkowski's First Theorem for $L = \mathbb{Z}^2$ ($\det(L) = 1$): once the disk's area exceeds $2^2 \cdot \det(L) = 4$ (radius $r \gtrsim 1.13$), the theorem guarantees a nonzero lattice point lies inside it — and indeed $(1, 0)$ (shown here relative to the chosen origin) is inside, at distance exactly $1 < 1.13$.

Example: Minkowski's Bound on the Simplest Lattice, $L = \mathbb{Z}^2$

Take $L = \mathbb{Z}^2$, so $\det(L) = 1$ (Section 1.8.5). Let K be a disk of radius r centered at the origin — symmetric and convex, exactly as the theorem requires. The theorem applies once $\mathrm{Vol}(K) = \pi r^2 > 2^2 \cdot \det(L) = 4$, i.e. once

$$r > \frac{2}{\sqrt{\pi}} \approx 1.128.$$

So Minkowski's theorem *guarantees*, without any further work, that a disk of radius 1.13 around the origin must contain a nonzero lattice point. Checking directly: we already know $\lambda_1(\mathbb{Z}^2) = 1$ (the point $(1, 0)$), and indeed $1 < 1.128$ — consistent with, and in fact comfortably inside, the guarantee. Notice the theorem didn't need to know that $(1, 0)$ was the answer; it only needed the disk to be big enough.

Example: The Same Bound on a Very Different-Looking Lattice

Now take a stretched lattice with basis $b_1 = (5, 0)$, $b_2 = (0, 1)$, so $\det(L) = |5 \times 1 - 0 \times 0| = 5$ (Section 1.8.5). The disk threshold becomes $\pi r^2 > 4 \times 5 = 20$, i.e. $r > \sqrt{20/\pi} \approx 2.523$. Minkowski's theorem guarantees a nonzero lattice point within distance 2.523 of the origin. The actual shortest vector here is $(0, 1)$, of length 1 — again comfortably inside the guaranteed radius, but this time the guarantee is quite *loose*: the theorem promised "within 2.523" and reality delivered "within 1." This is expected and normal — Minkowski's theorem is a **worst-case guarantee**, not an exact formula; it never overpromises (the bound always holds), but it can certainly underdeliver on precision for any one specific lattice. Its real value is as a *universal* ceiling that holds for *every* lattice of a given determinant, not as a precise estimate for one.

Remark: From the Disk Bound to the General Formula

Redo the disk calculation above in n dimensions instead of 2: the volume of a radius-r ball in $\mathbb{R}^n$ shrinks (relative to r^n) as n grows, and pushing the same threshold argument through (we omit the n-dimensional volume formula itself, which involves the Gamma function) yields the general bound already stated in Lecture 1's introduction to this topic:

$$\lambda_1(L) \;\leq\; \sqrt{n} \cdot \det(L)^{1/n} \qquad \text{(up to a small constant factor).}$$

In plain terms: a lattice can never "hide" an unexpectedly long shortest vector behind a small determinant — volume and minimum distance are locked together by geometry alone, regardless of which basis you're handed to describe the lattice. This is part of *why* SVP is only believed hard in the specific regime cryptographers deliberately choose (large dimension n, carefully scaled $\det(L)$), rather than for arbitrary lattices — a preview of the parameter-selection discussion we'll return to once we look at how Kyber and Dilithium choose their concrete dimensions and moduli.

Exercise: Practice

Let $L = \mathbb{Z}^3$ (so $\det(L) = 1$). Using the volume of a ball in $\mathbb{R}^3$, $\text{Vol} = \frac{4}{3}\pi r^3$, find the radius r at which Minkowski's theorem first guarantees a nonzero lattice point inside the ball. Compare it to the true value $\lambda_1(\mathbb{Z}^3) = 1$ (e.g. the point $(1, 0, 0)$) — is the guarantee looser or tighter, relative to the truth, than it was for $\mathbb{Z}^2$ in the worked example above?

Exercise: Practice: Same Lattice, or Different?

For each pair of bases below (columns are the basis vectors), form $U = B^{-1}B'$, check whether U has integer entries with $|\det(U)| = 1$, and conclude whether the two bases generate the same lattice. Cross-check by also comparing $|\det(B)|$ and $|\det(B')|$ directly.

(a) $B = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, $B' = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$.

(b) $B = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}$, $B' = \begin{pmatrix} 1 & 3 \\ 1 & 4 \end{pmatrix}$.

(c) $B = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, $B' = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$.

Looking Ahead: Optional / Advanced: Minkowski's Second Theorem

The First Theorem only talks about $\lambda_1(L)$, the *single* shortest vector. It says nothing about the second-shortest independent direction, the third, and so on. **Minkowski's Second Theorem** fills that gap: recall the **successive minima** $\lambda_1(L) \leq \lambda_2(L) \leq \cdots \leq \lambda_n(L)$ flagged in the defbox above, where $\lambda_i(L)$ is the smallest radius r such that the ball of radius r contains i *linearly independent* lattice vectors. The theorem bounds their *product*:

$$\frac{2^n}{n!}\det(L) \;\leq\; \lambda_1(L)\cdots\lambda_n(L) \;\leq\; 2^n \det(L).$$

This is not core material — everything in Sessions 1–6 (understanding LWE, the algorithms, and the physical attacks in Section 2 of the proposal) only ever leans on the First Theorem's intuition: "small determinant ⇒ short vector must exist." The Second Theorem becomes genuinely useful only once the research moves toward analyzing *trapdoor* lattices and Gaussian sampling in depth — which is exactly what underlies Falcon's signing procedure (flagged for Session 5). A trapdoor basis needs *all* of its vectors short, not just the shortest one, for Gaussian sampling to be both efficient and secure; that "all vectors short together" requirement is precisely what the successive minima $\lambda_1, \ldots, \lambda_n$ capture, and the Second Theorem is the tool that ties them back to $\det(L)$. This box is worth returning to *if and only if* the Phase 2 research direction (Section 4 of the proposal) ends up involving Falcon's sampler or trapdoor generation in detail — otherwise, the First Theorem alone is sufficient background.

Exercise: Practice

For the lattice generated by $b_1 = (1, 2)$, $b_2 = (3, 1)$ (from the earlier homework exercise): compute $\det(L)$ directly from the basis matrix. Then pick any *other* valid basis for the same lattice (e.g., $b_1' = b_1$, $b_2' = b_2 - b_1$) and confirm you get the same $\det(L)$.

Looking Ahead: Where This Leads Next Lecture

With Groups → Subgroups → Rings → Fields → Modular arithmetic → Vector spaces → Basis → Lattice now in place, the next lecture can finally state the **Learning With Errors (LWE)** problem precisely: a secret vector, hidden inside noisy linear equations over $\mathbb{Z}_q$ (a ring, Section 1.6), which is exactly the kind of "bad basis" hardness described above, dressed up in a specific, standardized cryptographic form. We will also unpack the ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ flagged in Section 1.4, since **module lattices** (lattices whose coordinates come from a ring like R_q instead of plain integers) are exactly what Kyber and Dilithium use in practice.

1.9 Summary Table

Structure	**Requires**	**Example**
Group	1 operation, inverses exist	$(\mathbb{Z}, +)$
Subgroup	A subset that is itself a group	$2\mathbb{Z} \subseteq \mathbb{Z}$
Coset / Quotient group	G sorted into shifted copies of H	$\mathbb{Z}/3\mathbb{Z} = \{0, 1, 2\}$
Ring	2 operations, no division needed	$(\mathbb{Z}, +, \times)$, $\mathbb{Z}[x]$
Integral domain	Ring, no zero divisors	$\mathbb{Z}$, $\mathbb{Z}_5$ (not $\mathbb{Z}_6$)
Field	Ring + division by nonzero elements	$\mathbb{Q}, \mathbb{R}, \mathbb{Z}_p$ (p prime)
Modulus	Wraparound arithmetic	$\mathbb{Z}_n = \{0, \ldots, n-1\}$
Group of units $\mathbb{Z}_n^*$	Invertible elements of $\mathbb{Z}_n$, size $\varphi(n)$	$\mathbb{Z}_{12}^* = \{1, 5, 7, 11\}$
Vector space	Field + vectors + scalar mult.	$\mathbb{R}^n$
Basis	Independent, spanning vector set	$\{(1, 0), (0, 1)\}$ in $\mathbb{R}^2$
Orthogonal basis	Basis, all pairs perpendicular	Gram–Schmidt output
Lattice	Discrete additive subgroup of $\mathbb{R}^n$	$\mathbb{Z}^n$; grid of points
$\lambda_1(L)$	Length of the shortest nonzero lattice vector	$\lambda_1(\mathbb{Z}^2) = 1$
$\det(L)$	Volume of the fundamental domain	$\lvert\det(B)\rvert$, any basis B

1.10 Full List of Exercises (Assign as Homework Before Lecture 2)

Exercise: Homework Set

1. Verify $(\{1, 2, 3, 4\}, \times \bmod 5)$ is a group by writing out its full multiplication table (Section 1.3 exercise, above).
2. Is $3\mathbb{Z} = \{\ldots, -6, -3, 0, 3, 6, \ldots\}$ a subgroup of $(\mathbb{Z}, +)$? Justify using the subgroup test (Section 1.3.1). Is $\{0, 3, 6, 9, \ldots\}$ (only the non-negative multiples of 3) a subgroup of $(\mathbb{Z}, +)$? Why or why not?
3. List all distinct cosets of $4\mathbb{Z}$ in $\mathbb{Z}$, confirm there are exactly 4, and identify which coset -5 belongs to (Section 1.3.2 exercise, above).
4. Find a pair of zero divisors in $\mathbb{Z}_{12}$, and use the zero-divisor fact (Section 1.4.1) to explain why $\mathbb{Z}_{12}$ cannot be a field, without computing any inverses.
5. Compute $23 \times 15 \bmod 7$; list the non-invertible elements of $\mathbb{Z}_8$; use the Extended Euclidean Algorithm (Section 1.6.1) to find the inverse of $3 \bmod 7$ — confirm it matches the worked example.
6. Use the Extended Euclidean Algorithm to find $5^{-1} \bmod 11$ (Section 1.6.1 exercise, above), showing all steps.
7. Is $\mathbb{Z}_9$ a field? Why or why not? Answer it two ways: (a) using $\gcd$ (Section 1.6), and (b) using zero divisors (Section 1.4.1).
8. Show that $\{(1, 1), (1, -1)\}$ is a valid basis of $\mathbb{R}^2$ by (a) checking linear independence and (b) writing the vector $(4, 2)$ as a linear combination of them.
9. Run Gram–Schmidt on $b_1 = (1, 1)$, $b_2 = (0, 2)$ (Section 1.7.3 exercise, above), and verify the result is orthogonal.
10. For the lattice generated by $b_1 = (1, 2)$ and $b_2 = (3, 1)$: (a) is the point $(7, 5)$ in the lattice? (b) compute $\det(L)$ from this basis; (c) pick a different valid basis for the same lattice and confirm $\det(L)$ is unchanged (Section 1.8.5 exercise, above).
11. Let $L = \{(x, y) \in \mathbb{Z}^2 : x + y \text{ is even}\}$. (a) Show L is a subgroup of $(\mathbb{R}^2, +)$. (b) Show L is discrete (find an ε that isolates $\vec{0}$). (c) Since L is therefore a lattice, find a basis $\{b_1, b_2\}$ for it, and compute $\det(L)$.
12. In your own words (3–4 sentences): why does the definition of a lattice need *both* "subgroup" and "discrete"? Use the $\mathbb{Q}$ example from Section 1.8.2 to explain what goes wrong if you drop the discreteness requirement.
13. In your own words (3–4 sentences), explain why a "good basis" and a "bad basis" for the same lattice can make the same mathematical object feel completely different to work with computationally, now that you can measure this with norms (Section 1.7.2).

Lesson 2: From Lattices to Learning With Errors

SVP, CVP, SIS, LWE, Ring-LWE, and Module-LWE

Purpose of this lecture. Lecture 1 built the algebraic toolkit — *Group* → *Subgroup* → *Ring* → *Field* → *Modular Arithmetic* → *Vector Space* → *Basis* → *Lattice* → *Determinant* — and ended by naming the Shortest Vector Problem (SVP) as one of the hard problems lattice cryptography rests on. This lecture finishes the job: we state SVP and CVP formally, meet the **Short Integer Solution (SIS)** and **Learning With Errors (LWE)** problems precisely, and then generalize LWE to **Ring-LWE** and **Module-LWE** — the exact hard problems underneath Kyber (ML-KEM) and Dilithium (ML-DSA). By the end, the public key of a real post-quantum scheme should look to you like "just an LWE sample," not a mysterious block of numbers.

2.1 Quick Recap of Lecture 1

Before building anything new, let's fix the vocabulary we'll be using constantly today.

Term	One-line meaning
Ring $(R, +, \times)$	Two operations; addition invertible, multiplication need not be
$\mathbb{Z}_q$	Integers $\{0, \ldots, q-1\}$, wraparound ("clock") arithmetic
Vector space, basis	Every point = a unique combination of basis directions
Lattice L	*Integer-only* combinations of a basis; a discrete grid in $\mathbb{R}^n$
$\det(L)$	Volume of the lattice's fundamental cell; basis-independent
$\lambda_1(L)$	Length of the shortest nonzero vector in L
Good basis vs. bad basis	Short, near-orthogonal vs. long, skewed — same lattice, different difficulty

Remark: The One Idea Everything Today Builds On

Lecture 1 ended with this sentence: *"the secret key is typically a good (short) basis, while the public key describes the same lattice using a bad basis."* Today we make that idea concrete and computational. Every hard problem in this lecture is, underneath, a question of the form: *"given a bad description of a lattice, recover something a good description would make trivial."*

2.2 One-Way Functions: The Idea Behind All of Cryptography

2.2.1 What Makes a Function "One-Way"?

Every cryptographic scheme — classical or post-quantum — is ultimately an engineering answer to the same question: is there a function that is **easy to compute in one direction and computationally infeasible to invert in the other**? Mixing two paint colors is easy; recovering the exact two starting colors from the mixture is not, even though nothing about the mixing itself was complicated. A **one-way function** is the mathematical version of that asymmetry: given x, computing $f(x)$ takes a fraction of a second; given only $f(x)$, finding an x that produces it is believed to take longer than the lifetime of the universe, once the numbers involved are large enough.

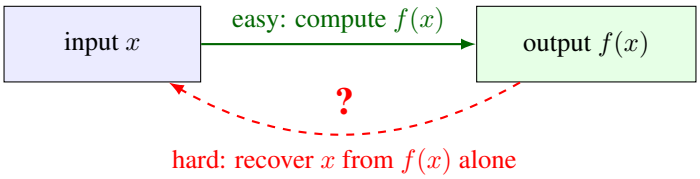


Figure 2.1: The shape every cryptographic hard problem shares: an easy forward direction, and a backward direction with no known efficient shortcut. Everything in this lecture — SVP, CVP, SIS, LWE — is one more attempt to build a trustworthy instance of this exact picture.

2.2.2 Two Classical Examples: Factoring and the Discrete Log Problem

Long before lattices entered cryptography, two number-theoretic one-way functions carried almost the entire field. Both are worth seeing now, briefly and without proof, purely so that the lattice-based version you're about to meet has something familiar to stand next to; a later lecture in this series covers RSA and elliptic-curve cryptography in full.

- **Integer factorization.** Multiplying two large primes p and q together to get $n = p \times q$ takes a computer a fraction of a second, no matter how large p and q are. Going backward — given only n, recovering the two primes p and q that were multiplied — has no known efficient algorithm once n is a few hundred digits long. This asymmetry is the entire foundation of **RSA**.
- **The discrete logarithm problem.** Fix a group (for instance, integers modulo a large prime p, under multiplication) and a fixed element g in it. Computing g^a for a given exponent a is fast (repeated squaring). Going backward — given g and g^a, recovering a — is again believed to have no efficient general algorithm. This asymmetry underlies **Diffie–Hellman key exchange**, **ElGamal encryption**, and, in a version that swaps the group for points on an elliptic curve, **elliptic-curve cryptography (ECC)**.

Remark: Why Both Are Now on Borrowed Time

Factoring and the discrete log problem are exactly the two problems a large-scale quantum computer, running **Shor's algorithm**, can solve efficiently — which is precisely why RSA and ECC are the schemes post-quantum cryptography needs to replace, and precisely why this course exists. Lattice problems are one of the leading replacement candidates because no known quantum algorithm solves them efficiently.

2.2.3 A Lattice-Flavored Preview

Lattice-based cryptography needs its own one-way function, built from its own hard problem, with the same easy-forward/hard-backward shape as factoring and the discrete log problem — just grown out of the geometry Lecture 1 built rather than out of number theory. The next section states that hard problem formally, in two versions.

2.3 Two Canonical Hard Problems on a Lattice

2.3.1 The Shortest Vector Problem (SVP)

Definition: Shortest Vector Problem (SVP)

Given a basis $B = \{b_1, \ldots, b_n\}$ for a lattice $L = \mathcal{L}(B) \subseteq \mathbb{R}^n$, find a nonzero vector $v \in L$ with $\|v\| = \lambda_1(L)$ (the minimum distance defined in Lecture 1). The **decision** version, GapSVP$_\gamma$, only asks: is $\lambda_1(L) \leq d$, or is $\lambda_1(L) > \gamma \cdot d$, for a given distance d and approximation factor $\gamma \geq 1$? (One of these two must hold; the problem promises no "messy middle" case.)

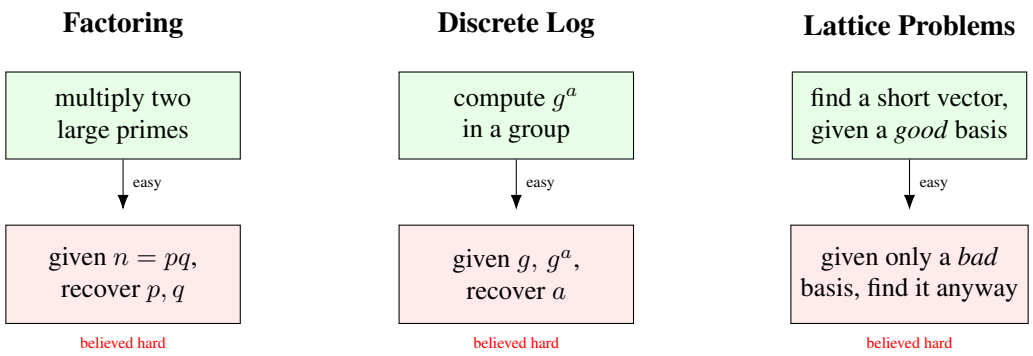


Figure 2.2: Three unrelated-looking sources of one-way hardness, side by side. Factoring underlies RSA; the discrete log problem underlies Diffie–Hellman and elliptic-curve cryptography. Lattice problems are this course's family — "good basis" and "bad basis" are exactly Lecture 1's terms, and the next section pins down precisely what "find it anyway" means.

Remark: Why the "Approximation Factor" γ Matters

Finding the *exact* shortest vector ($\gamma = 1$) is extremely hard even for classical computers once n is a few hundred. But so is finding a vector even *polynomially longer* than the shortest one, as long as γ stays modest and n is large — and that weaker, "approximate" version is all cryptography actually needs to be hard. This is why you will see phrases like "SVP is hard to approximate within polynomial factors" rather than just "SVP is hard."

2.3.2 The Closest Vector Problem (CVP)

Definition: Closest Vector Problem (CVP)

Given a basis B for a lattice $L \subseteq \mathbb{R}^n$ and a **target point** $t \in \mathbb{R}^n$ (not necessarily in L), find the lattice point $v \in L$ minimizing $\|t - v\|$. The approximate decision version GapCVP_γ again just distinguishes "some lattice point within distance d of t" from "every lattice point is farther than $\gamma \cdot d$ from t."

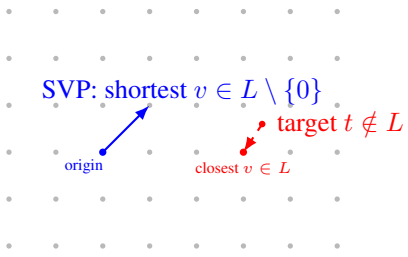


Figure 2.3: SVP (left, blue) asks for the shortest nonzero vector reachable inside the lattice itself. CVP (right, red) asks, for an arbitrary target point that is *not* on the grid, which grid point is nearest. Both are easy to state and easy to solve by eye in 2D — and famously hard in high dimensions with only a bad basis to work with.

Example: CVP by Eye vs. CVP by Formula

In Figure 2.3 you can *see* the closest grid point instantly. That's because your eye is effectively using a very good, orthogonal basis (the obvious horizontal/vertical grid directions). Algorithmically, CVP is solved by writing $t = \sum c_i b_i$ (allowing real, non-integer c_i) and **rounding each c_i to the nearest integer** — but this shortcut only gives the *true* closest point when the basis is good (short, near-orthogonal). Hand the exact same lattice to an algorithm with a bad basis instead, and naive rounding can miss the true closest point by a wide margin. This gap — correct-with-a-good-basis, unreliable-with-a-bad-basis — is precisely what CVP-based cryptography exploits.

2.3.3 The Big Picture: From Worst-Case Geometry to a Cryptographic Tool

Section 2.2 listed three things a raw hard problem needs before it becomes a usable one-way function: an efficiently **sampleable random instance**, a **proof** tying that instance's hardness back to the worst case, and an **easy forward direction**. SVP and CVP, exactly as just stated, don't hand you any of the three for free — "given a basis, find X" says nothing about how to generate a random hard instance, let alone prove anything about it.

SIS and LWE, met later in this lecture, are exactly this fix. Each one starts from a **uniformly random matrix** A — trivial to generate — and each comes with a theorem (Ajtai, 1996 for SIS; Regev, 2005 for LWE) tying average-case hardness of that random instance back to worst-case SVP/CVP-style hardness. In fact, SIS and LWE are not new ideas grafted onto SVP and CVP: **they are SVP and CVP**, restricted to one specific, algebraically-generated, easy-to-sample family of lattices built from A.

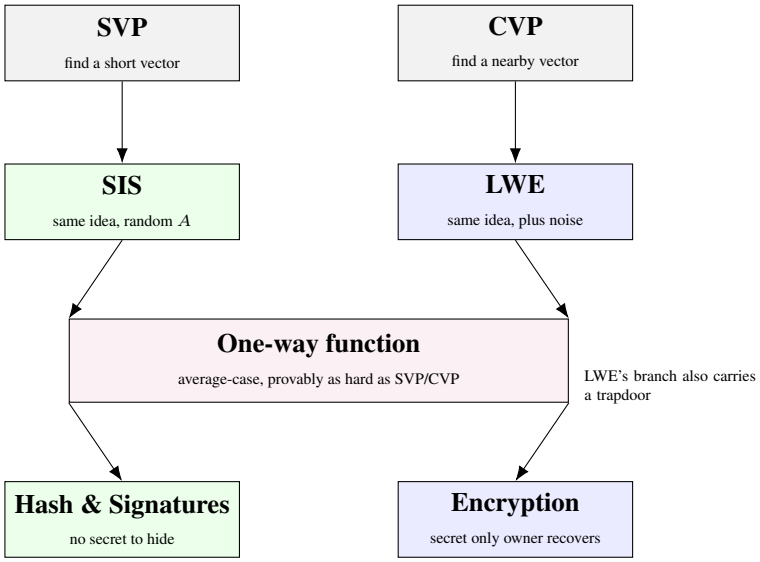


Figure 2.4: SIS and LWE are not a detour from SVP and CVP — they *are* SVP and CVP, restricted to a random, easy-to-sample family of lattices built from a matrix A. SIS lives on the kernel lattice $\Lambda^{\perp}(A)$ (Section 2.4) and keeps SVP's "find a short vector" shape; LWE lives on the image lattice $\Lambda_q(A)$ (Section 2.5.5) and keeps CVP's "find a nearby point" shape. That structural difference is exactly why the two problems power different cryptographic jobs: SIS's kernel/short-vector shape gives you "no two things collide" (hash functions, signatures — nothing to hide), while LWE's image/nearby-point shape gives you "there's a hidden secret only the trapdoor-holder can recover" (encryption).

Remark: A Trapdoor Is a Fourth, Optional Ingredient — Not Part of the Three Requirements

It's tempting to read the merged box above as claiming every one-way function built this way needs a trapdoor — it doesn't. The three requirements from Section 2.2 (a sampleable random instance, a worst-case reduction, an easy forward direction) are exactly what a plain one-way function needs, and that's already enough for **SIS-based hashing**: nobody — not even whoever generated A — can efficiently find a collision. There is no secret held by any party, and none is needed; that's the entire point of a hash function. A **trapdoor** is a separate, optional fourth ingredient, needed only when some designated party must be able to invert efficiently while everyone else can't: exactly what **LWE-based encryption** needs (the key-owner decrypts; nobody else can), and what Falcon's GPV sampling (Lecture 4) needs on the signing side. SIS's branch of Figure 2.4 never uses a trapdoor at all; only LWE's does.

Remark: Where the Two Halves of This Figure Get Made Rigorous

Section 2.4's "Why This Is Secretly a Lattice Problem" box shows SIS's solutions living inside the kernel lattice $\Lambda^{\perp}(A)$ — literally an SVP instance. Section 2.5.5's "LWE Is Bounded-Distance Decoding" box shows LWE's target b sitting near the image lattice $\Lambda_q(A)$ — literally a (promise-bearing) CVP instance. Both boxes cash in the claim made by Figure 2.4 above.

2.4 The Short Integer Solution (SIS) Problem

Remark: Motivating SIS With a Question

Here's a question that sounds almost too easy: if I hand you a random matrix A (think: rows and columns of random numbers mod q), can you find a *short* nonzero vector x so that $Ax \equiv \vec{0} \pmod{q}$? If x could be any size at all, this would be trivial linear algebra — as long as A has more columns than rows, there are always *some* nonzero solutions (more unknowns than equations). The catch, and the entire reason this is a cryptographic problem rather than a homework exercise, is the word **short**: among the (many) solutions that exist, finding one whose entries are all small turns out to be hard once the matrix is big enough. This is SIS.

Definition: Short Integer Solution (SIS)

Fix positive integers n, $m > n$, a modulus q, and a bound β. Given a uniformly random matrix $A \in \mathbb{Z}_q^{n \times m}$, find a nonzero vector $x \in \mathbb{Z}^m$ with

$$Ax \equiv \vec{0} \pmod{q} \qquad \text{and} \qquad \|x\| \leq \beta.$$

Remark: Why This Is Secretly a Lattice Problem

Define $\Lambda^{\perp}(A) = \{\, x \in \mathbb{Z}^m : Ax \equiv \vec{0} \pmod{q} \,\}$. The name "$\perp$" is exactly what it looks like: $Ax \equiv \vec{0}$ means x pairs to zero (mod q) with *every* row of A, so $\Lambda^{\perp}(A)$ is the orthogonal complement of A's row space — the same idea as $V^{\perp}$ for ordinary vector spaces, just computed mod q instead of over $\mathbb{R}$.

A quick check confirms $\Lambda^{\perp}(A)$ is a subgroup of $\mathbb{Z}^m$ (closed under addition and negation, contains $\vec{0}$) and it is discrete (it's a subset of $\mathbb{Z}^m$) — so by Lecture 1's subgroup definition, $\Lambda^{\perp}(A)$ **is itself a lattice**. Solving SIS is exactly finding a short nonzero vector in $\Lambda^{\perp}(A)$ — i.e., **SIS is SVP**, restricted to this specific, algebraically-generated family of lattices. The reason cryptographers like this family is that generating a *random* instance is as easy as generating a random matrix A, yet the resulting lattice is (conjecturally) exactly as hard to solve SVP on as the worst-case lattices Lecture 1 discussed abstractly.

Example: A Tiny SIS Instance by Hand

Let $n = 1$, $m = 4$, $q = 17$, and $A = \begin{pmatrix} 3 & 5 & 8 & 1 \end{pmatrix} \in \mathbb{Z}_{17}^{1\times 4}$. We want a short, nonzero, *integer* vector $x = (x_1, x_2, x_3, x_4)$ with $3x_1 + 5x_2 + 8x_3 + x_4 \equiv 0 \pmod{17}$. Try $x = (1, 1, -1, -16) \to$ too long. Instead try $x = (1, -1, 0, 2)$: $3(1) + 5(-1) + 8(0) + 1(2) = 3 - 5 + 0 + 2 = 0 \equiv 0 \pmod{17}$. Success, and $\|x\|$ is small (each entry is ≤ 1 in absolute value except the last, which is 2). This x is a valid, short solution — with $m = 4$ columns squeezed against only $n = 1$ modular equation, short solutions are guaranteed to exist by a pigeonhole argument (more columns than rows means lots of redundancy); the point of SIS being *hard* is that finding one is nontrivial once n, m are cryptographically large (hundreds to thousands), not that one fails to exist.

Remark: What SIS Is Used For

SIS is the workhorse behind lattice-based **hash functions** and was historically the basis of early lattice signature schemes. Section 2.4.1 below builds the hash-function connection explicitly, right now, while the SIS instance above is still fresh. You will meet SIS's signature-adjacent side again in Session 3 (Dilithium/ML-DSA) and again in Session 5, since Dilithium's fault-attack surface is closely tied to how it uses a SIS-flavored rejection-sampling step.

2.4.1 A Concrete Application: SIS-Based Hash Functions

Remark: What a Hash Function Needs

A cryptographic hash function takes an input of some size and **compresses** it down to a fixed, shorter output, with one crucial security property: **collision resistance** — it should be hard to find two *different* inputs $x_1 \neq x_2$ that hash to the *same* output. Everything below is aimed at exactly one goal: building a compressing function whose collision resistance follows directly, and provably, from SIS being hard — no separate, from-scratch security assumption required.

Definition: The Ajtai-Style SIS Hash Function

Fix a uniformly random matrix $A \in \mathbb{Z}_q^{n \times m}$ (exactly the SIS matrix above), with $m > n \log_2 q$ so the map below is genuinely compressing. Define $h_A : \{0,1\}^m \to \mathbb{Z}_q^n$ by

$$h_A(x) = Ax \bmod q.$$

The input x is an m-bit string (so there are 2^m possible inputs); the output lives in $\mathbb{Z}_q^n$ (only q^n possible outputs). Since $m > n \log_2 q$ forces $2^m > q^n$, the function is squeezing a strictly bigger set down into a strictly smaller one — exactly the compression a hash function needs, and exactly why some collision is guaranteed to exist by the pigeonhole principle alone (finding one is the hard part).

Remark: Why Collision Resistance *Is* SIS, Not Just *Like* SIS

Suppose an attacker finds a collision: two distinct inputs $x_1 \neq x_2 \in \{0,1\}^m$ with $h_A(x_1) = h_A(x_2)$, i.e. $Ax_1 \equiv Ax_2 \pmod q$. By linearity, $A(x_1 - x_2) \equiv \vec{0} \pmod q$. Set $x = x_1 - x_2$. Then:

- $x \neq \vec{0}$, since $x_1 \neq x_2$.
- $Ax \equiv \vec{0} \pmod q$ — exactly the SIS equation.
- x is automatically **short**: each coordinate of $x_1, x_2 \in \{0,1\}^m$ is 0 or 1, so each coordinate of $x = x_1 - x_2$ is in $\{-1, 0, 1\}$, giving $\|x\| \leq \sqrt{m}$ — comfortably within any reasonable SIS bound β.

So **a collision in h_A is, verbatim, a solution to the SIS instance defined by the same matrix A.** Turning this around: if SIS is hard for A, then finding a collision in h_A is exactly as hard — there is no weaker link elsewhere in the construction. This is the entire appeal of building a hash function this way: security reduces to a single, already-studied lattice problem, with nothing extra to trust.

Remark: Why the Domain Restriction Isn't Optional

It's worth being explicit about why h_A is defined on bit strings $\{0,1\}^m$ specifically, rather than on all of $\mathbb{Z}_q^m$. Without a shortness bound, $Ax \equiv \vec{0} \pmod q$ is *always* trivially solvable, for *any* matrix A at all: just take $x = q \cdot e_i$ (i.e. q in one coordinate, 0 elsewhere) for any i. Then $Ax = q \cdot (Ae_i) \equiv \vec{0} \pmod q$ automatically, no matter what A is — every entry is a multiple of q. So "find a nonzero x with $Ax \equiv \vec{0}$," with no bound on $\|x\|$, carries *zero* cryptographic content; this is exactly why the SIS defbox above required $\|x\| \leq \beta$ as part of the problem statement, not as an afterthought.

This is precisely what the domain restriction to $\{0,1\}^m$ is doing, and it's doing double duty: $m > n \log_2 q$ forces a compression (Section 2.4.1's defbox), *and*, because both inputs come from $\{0,1\}^m$, their difference is *automatically* short — $\|x\| \leq \sqrt{m}$ — no matter which collision an attacker happens to find. If the hash's domain were unrestricted instead, a "collision" could hand back the trivial $x = q \cdot e_i$ above, which satisfies the SIS equation but is not a hard instance of anything. The reduction only means something because the construction *forces* every possible collision, without exception, into the short/hard regime.

Example: A Collision, Found and Verified by Hand

Reuse $n = 1$, $m = 4$, $q = 17$, $A = \begin{pmatrix} 3 & 5 & 8 & 1 \end{pmatrix}$ from the SIS worked example above, and the short SIS solution already found there, $x = (1, -1, 0, 2)$. That x has entries outside $\{-1, 0, 1\}$ in one coordinate (the 2), so it isn't itself a difference of two bit-strings — but it illustrates the mechanism cleanly, and a genuine bit-string collision is built the same way. Take

$$x_1 = (1, 0, 0, 1), \qquad x_2 = (0, 1, 0, 0) \qquad \text{(both in } \{0,1\}^4\text{)}.$$

Compute both hashes mod 17:

$$h_A(x_1) = 3(1) + 5(0) + 8(0) + 1(1) = 4, \qquad h_A(x_2) = 3(0) + 5(1) + 8(0) + 1(0) = 5.$$

These don't collide ($4 \neq 5$) — so x_1, x_2 is *not* a collision, only a candidate pair, exactly as most randomly-chosen pairs will not collide. This is the expected, normal outcome: with $q = 17$ this toy instance barely compresses at all ($m = 4$ bits down to one symbol in $\{0, \ldots, 16\}$), so collisions are not hard to stumble on by search, but the *mechanism* — take any actual collision, subtract, get a short nonzero SIS solution — is exactly what the remark above proves in general, regardless of how easy or hard this particular toy-sized search happens to be.

Remark: Why This Only Works Because of Compression

It's worth being precise about where $m > n \log_2 q$ was used. Without that condition, h_A might be injective (no collisions could exist at all, even in principle), and the whole security question would be vacuous. With it, collisions are *guaranteed to exist* (pigeonhole, as noted in the def-box) — the hardness claim is never "no collision exists," only "no one can *find* one efficiently." This mirrors ordinary cryptographic hash functions like SHA-256 exactly: collisions must exist (infinite inputs, finite outputs), and security only ever means "hard to find," never "impossible."

Fact: $h_A(x) = Ax$ vs. SHA-2/SHA-3: Two Different Kinds of Confidence

- **SHA-2/SHA-3:** security is *empirical* — decades of public cryptanalysis have failed to find a collision. Fast, compact, no known quantum weakness worth designing around (Grover only gives a quadratic speedup). The default choice for hashing in practice.
- **h_A:** security is a *theorem* — a collision provably yields a solution to worst-case SIS/lattice problems, not just this one instance. The cost is size and speed: a large public matrix A and a matrix–vector multiply per hash, versus a few dozen fast bitwise operations.
- **Why bother, then:** h_A is *linear* in x, so it composes algebraically with other lattice constructions — homomorphic operations, commitments, zero-knowledge proofs — in ways SHA-2/SHA-3's deliberately nonlinear internals cannot. Its role is as a building block *inside* lattice-based protocols, not as a drop-in replacement for general-purpose hashing.

Exercise: Practice

Using the same $A = (3, 5, 8, 1) \in \mathbb{Z}_{17}^{1\times 4}$: (a) compute $h_A(x)$ for $x_1 = (1, 1, 0, 0)$ and $x_2 = (0, 0, 1, 1)$; do they collide? (b) Find, by inspection or trial, a genuine colliding pair $x_1 \neq x_2 \in \{0,1\}^4$ with $h_A(x_1) = h_A(x_2)$. (c) For your pair from part (b), compute $x = x_1 - x_2$ and confirm directly that $Ax \equiv 0 \pmod{17}$ — i.e., confirm your collision really does hand you a SIS solution, exactly as the argument above claims it must.

Remark: SIS vs. LWE: Why We Need Both

SIS and LWE are built from the same raw material — a random matrix A — but they are hard for different reasons and do different jobs. SIS's hardness is about *collisions*: it's hard to find a short x with $Ax \equiv \vec{0}$, which is exactly the guarantee **hash functions** (Section 2.4.1 above) and (lattice) **signatures** need — there's no secret to hide, only "no two short inputs collide." LWE's hardness is about *concealment*: without noise, $b = As$ is solved instantly by ordinary linear algebra (Section 2.5.1's box below); the noise e is precisely what breaks that shortcut, and it creates an asymmetry — whoever knows s can cancel/tolerate the noise, while anyone else just sees (A, b) that looks uniformly random. That asymmetry is what **encryption** needs, and SIS alone can't give it. The two problems are even dual views of the same A: SIS lives on the kernel lattice $\Lambda^{\perp}(A)$ (Section 2.4 above), LWE lives on the image lattice $\Lambda_q(A)$ (Section 2.5.5 below) — one random matrix, two lattices, two different cryptographic jobs.

2.5 Learning With Errors (LWE)

2.5.1 The Idea, in Plain Language

Imagine someone hands you a stack of linear equations in an unknown secret vector s — but each equation's right-hand side has been nudged by a small, random amount of noise before you see it. Solving *noise-free* linear equations is easy (that's just linear algebra — Gaussian elimination). Solving them once every equation is slightly wrong, and you don't know by how much, turns out to be extremely hard. That is the entire idea of LWE.

Remark: Why Not Just Solve It With Linear Algebra?

This is the single most common point of confusion about LWE, so it's worth working through a tiny concrete case. Suppose $n = 2$ and, with no noise at all, we're given:

$$1 \cdot s_1 + 2 \cdot s_2 = 8, \qquad 3 \cdot s_1 + 1 \cdot s_2 = 9.$$

Two equations, two unknowns — solve normally (e.g. elimination) and you get $s_1 = 2,\ s_2 = 3$ exactly, every time, no matter how large n gets, as long as you have n independent equations. **Linear algebra doesn't care how big the system is** — Gaussian elimination scales smoothly to thousands of unknowns. Now add noise: suppose the equations you're actually given are

$$1 \cdot s_1 + 2 \cdot s_2 = 8 + e_1, \qquad 3 \cdot s_1 + 1 \cdot s_2 = 9 + e_2,$$

for some small, unknown $e_1, e_2 \in \{-1, 0, 1\}$ that you are never told. Now elimination doesn't work: any attempt to "solve exactly" is solving for the *wrong* thing, since the right-hand sides are off by an unknown amount. You could try every combination of e_1, e_2 and re-solve each time — for two small error values that's manageable, but for hundreds of equations, each with its own small unknown error, the number of combinations to check explodes exponentially. That explosion, not any deep mystery, is where LWE's hardness lives: **noise breaks the one tool (linear algebra) that would otherwise make this trivial**, and no comparably efficient tool has been found to replace it.

2.5.2 Formal Definition

Definition: Learning With Errors (LWE)

Fix a secret dimension n, a number of samples m (typically $m \gg n$ in practice), a modulus q, and an error distribution χ over $\mathbb{Z}_q$ (Section 2.5.3 pins down the usual choice of χ). Exactly as with SIS (Section 2.4), the problem starts from a **uniformly random matrix** — fix $A \in \mathbb{Z}_q^{m \times n}$ chosen uniformly at random. Let $s \in \mathbb{Z}_q^n$ be a fixed, unknown **secret vector**, and let $e \in \mathbb{Z}_q^m$ be an **error vector** whose m entries are drawn independently from χ. Define

$$b \;=\; As + e \pmod{q} \;\in \mathbb{Z}_q^m.$$

- **Search-LWE:** given (A, b), find s.
- **Decision-LWE:** given (A, b), decide whether b was built this way, or b is instead a *uniformly random* vector in $\mathbb{Z}_q^m$, independent of A.

Row by row, this is exactly the "many samples" description. Write $a_i \in \mathbb{Z}_q^n$ for the i-th row of A, and b_i for the i-th entry of b. Then $b_i = \langle a_i, s \rangle + e_i \pmod{q}$ is one LWE sample, for $i = 1, \ldots, m$ — and A is nothing more than these m row-vectors $a_1, \ldots, a_m$ stacked on top of each other. "Many noisy inner-product samples" and "one noisy matrix-vector equation" are the *same object*, just described two ways; every worked example below moves freely between the two descriptions.

Remark: Yes — LWE Has a Matrix Too, Just Playing the Opposite Role From SIS

It's easy to walk away from Section 2.4 thinking "SIS has a matrix, LWE doesn't" — especially since LWE is usually introduced sample-by-sample first. That impression is wrong: **both problems start from the exact same raw ingredient, a uniformly random matrix over $\mathbb{Z}_q$.** What differs is the *shape* of the matrix and what you're asked to do with it:

- **SIS:** $A \in \mathbb{Z}_q^{n \times m}$ is *wide* (more columns than rows, $m > n$). You search for a short nonzero x in A's *kernel*: $Ax \equiv \vec{0}$. Nothing is hidden — the matrix is entirely public, and the whole difficulty is finding one particular short vector among the many that satisfy the equation.
- **LWE:** $A \in \mathbb{Z}_q^{m \times n}$ is *tall* (at least as many rows as columns, $m \geq n$, usually $m \gg n$). You're handed a point b that sits *near* A's *image* (near some As), nudged off by small noise e, and asked to recover which s produced it, or even just to tell that b isn't uniform.

Same raw material, opposite structural question: SIS asks "what short vector does A kill?"; LWE asks "what secret does A almost reveal?" This is also exactly why Lecture 1's subgroup/lattice machinery applies to *both*: SIS's matrix generates the kernel lattice $\Lambda^{\perp}(A)$ (Section 2.4), while LWE's matrix generates an image lattice $\Lambda_q(A)$ (Section 2.5.5 below) — two different lattices built from the same kind of object.

A notational warning. Some sources — including Chris Peikert's *Lattices in Cryptography* lecture notes (Lecture 13, "Learning With Errors"), which Lecture 1's bibliography already points to — state LWE with the *transposed* convention: a *wide* matrix $A \in \mathbb{Z}_q^{n \times m}$ (columns, not rows, are the samples), a secret *row* vector s, and $b^T = s^T A + e^T$. This is mathematically identical to the defbox above — just transpose everything, and "columns of A are samples" becomes "rows of A are samples." We fix the row-convention above throughout this course because it matches how Lecture 3 writes Kyber's $\vec{t} = A\vec{s} + \vec{e}$ and Lecture 4's ML-DSA $\vec{t} = A\vec{s}_1 + \vec{s}_2$; just be aware that flipping between sources' conventions is normal in this literature, and always check which axis a given source calls "n" and which it calls "m" before comparing formulas.

2.5.3 What the Error Distribution χ Actually Is

We've been saying "a small error" without pinning down what "small" means or where e_i actually comes from. Time to fix that.

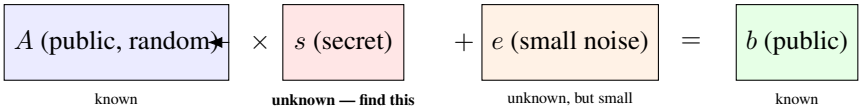
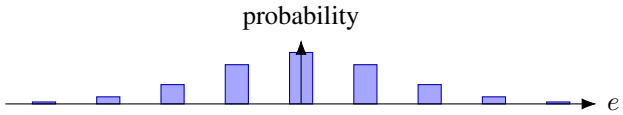


Figure 2.5: The anatomy of one LWE equation $b = As + e \pmod{q}$. An attacker sees A and b (both public/known) and must recover s, without ever being told e — only that e is small. Without the noise term e, this would be ordinary linear algebra and trivial to invert; the noise is precisely what makes it hard.

Definition: Discrete Gaussian Distribution (the usual choice of χ)

For a parameter $\sigma > 0$ (roughly, how spread out the noise is), the discrete Gaussian distribution over $\mathbb{Z}$ assigns to each integer x a probability proportional to $\exp(-x^2/2\sigma^2)$ — the same bell-curve shape as the ordinary (continuous) Gaussian, just evaluated only at integers and re-normalized so the probabilities sum to 1. In practice this means: $e = 0$ is the single most likely outcome, $e = \pm 1$ is somewhat less likely, $e = \pm 2$ less likely still, and values far from 0 (say $|e| > 6\sigma$) are so improbable they essentially never occur. χ is this distribution, and each e_i in an LWE sample is drawn independently from it.



Remark: Why a Bell Curve, and Not (Say) a Coin Flip $\{-1, 0, 1\}$?

Our worked examples below use a crude stand-in ($e \in \{-1, 0, 1\}$, each roughly equally likely) purely because it's easy to compute by hand. Real schemes use something bell-curve-shaped (a true discrete Gaussian, or in Kyber's case a closely related and cheaper-to-sample *centered binomial distribution*) for a precise mathematical reason: the hardness proofs connecting LWE back to worst-case lattice problems (Regev's reduction, Section 2.5.5) require the noise to behave like a Gaussian. Swap in a different-shaped distribution and those proofs may simply no longer apply — "small errors" alone isn't enough; their *shape* matters too. This is a recurring theme you'll meet again in Session 5: real implementations must sample from exactly the right distribution, and subtle sampling mistakes (or side-channel leakage *during* sampling) can quietly undermine security guarantees that look airtight on paper.

2.5.4 A Fully Worked Numeric Example

Example: Search-LWE by Hand, $n = 3$, $q = 17$

Let the (unknown, to an attacker) secret be $s = (2, 9, 1) \in \mathbb{Z}_{17}^3$, and let the error distribution χ produce values in $\{-1, 0, 1\}$ only (a toy stand-in for a discrete Gaussian). Suppose four samples are generated:

a_i	$\langle a_i, s \rangle \bmod 17$	e_i	$b_i = \langle a_i, s \rangle + e_i \bmod 17$
$(1, 0, 0)$	2	$+0$	2
$(0, 1, 0)$	9	-1	8
$(1, 1, 1)$	12	$+1$	13
$(3, 2, 1)$	$6 + 18 + 1 = 25 \equiv 8$	0	8

An attacker only ever sees the a_i (left column) and b_i (right column) — *not* the middle two columns. Notice that sample 1 alone, $b_1 = 2$, looks like it might reveal $s_1 = 2$ directly — and here it does, since e_1 happened to be 0. But sample 2 gives $b_2 = 8$, while the true $s_2 = 9$: the noise $e_2 = -1$ has already shifted the visible answer away from the truth by one full step, with no way for an outside observer to know whether the "off-by-one" is coming from s_2 or from e_2. Multiply that ambiguity across hundreds of dimensions and thousands of samples, and brute-force guessing collapses — this is the essence of why LWE is hard even though each individual equation looks almost solvable.

Remark: The Exact Same Example, Written as One Matrix Equation

Stack the four a_i rows from the table above into a matrix, exactly as the defbox's "row by row" remark describes — this is the $A \in \mathbb{Z}_{17}^{4\times 3}$ of the formal definition, with $m = 4$ samples and secret dimension $n = 3$:

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 1 & 1 \\ 3 & 2 & 1 \end{pmatrix}, \qquad s = \begin{pmatrix} 2 \\ 9 \\ 1 \end{pmatrix}, \qquad e = \begin{pmatrix} 0 \\ -1 \\ 1 \\ 0 \end{pmatrix}, \qquad b = As + e \bmod 17 = \begin{pmatrix} 2 \\ 8 \\ 13 \\ 8 \end{pmatrix}.$$

Multiplying out As row by row reproduces exactly the middle column of the table above $(2, 9, 12, 8 \bmod 17)$, and adding e then reduces mod 17 reproduces the last column $(2, 8, 13, 8)$ — the two views really are the same computation. An attacker in the search-LWE game is handed only A and b (the matrix and the right-hand column) and must recover s; in the decision-LWE game, they're handed A and *either* this b *or* four independent uniform values in $\mathbb{Z}_{17}$, and must say which.

Exercise: Try This Before Next Lecture

Using the same A rows as above but a *different* secret $s' = (2, 8, 1)$ (differs from s only in the middle coordinate), recompute $b_1, \ldots, b_4$ with the same errors e_i. Compare the resulting b_i values to the table above. How many of the four samples end up identical to the original table, purely by coincidence of the noise? What does this tell you about how many samples an attacker needs before guessing becomes hopeless?

Remark: Making "Brute Force Collapses" Concrete

In the toy example above, each error e_i took one of only 3 values, and there were 4 samples. A brute-force attacker who doesn't know the errors could, in principle, try every combination of errors, undo them, and check whether the resulting system solves consistently. The number of combinations to try is $(\text{number of error values})^{(\text{number of samples})}$. The table below shows how fast this explodes as the problem scales toward realistic sizes (real schemes use hundreds of samples, and a Gaussian χ with far more than 3 realistic values — this table deliberately keeps 3 values fixed just to isolate the effect of *sample count* growing):

Samples	Combinations (3^{samples})	Roughly
4 (our example)	81	trivial by hand
20	≈ 3.5 billion	a laptop, seconds
100	$\approx 5 \times 10^{47}$	far beyond any computer on Earth
500 (realistic scale)	astronomically larger still	meaningless to even state

This is only a crude illustration (real attacks are smarter than raw brute force — that's what lattice-reduction algorithms like LLL and BKZ are for), but it captures the right intuition: **the ambiguity introduced by unknown noise compounds multiplicatively with every additional equation**, which is exactly the opposite of ordinary linear algebra, where more equations make life *easier*, not harder.

Example: Decision-LWE: Can You Spot the Real Samples?

Here are two sets of (a_i, b_i) pairs over $\mathbb{Z}_{17}$. One set was generated as genuine LWE samples $b_i = \langle a_i, s \rangle + e_i$ for some fixed secret s; the other set has every b_i replaced by an independent uniformly random value in $\mathbb{Z}_{17}$, with no relationship to a_i at all.

Set X		Set Y	
a_i	b_i	a_i	b_i
$(1,0,0)$	2	$(1,0,0)$	11
$(0,1,0)$	8	$(0,1,0)$	3
$(1,1,1)$	13	$(1,1,1)$	6

Try to decide, just by looking, which set is "real" LWE and which is uniform noise. (Set X is, in fact, the real one — it's the same data as our search-LWE example above.) **The point of this exercise is that you can't tell, and neither can a computer, efficiently** — both sets look like an unremarkable list of numbers in $\mathbb{Z}_{17}$. That indistinguishability, formalized, *is* decision-LWE. It's also exactly the property real encryption schemes lean on: a ciphertext under LWE-based encryption should be computationally indistinguishable from random noise to anyone without the secret key.

2.5.5 Why LWE Is Believed Hard: the Lattice Connection

Remark: LWE Is Bounded-Distance Decoding (BDD), a Cousin of CVP

Define the lattice $\Lambda_q(A) = \{\, y \in \mathbb{Z}_q^m : y \equiv As \pmod q \text{ for some } s \in \mathbb{Z}_q^n \,\}$ (all vectors reachable as A times *some* secret, viewed as a lattice via the ring structure of $\mathbb{Z}_q$ from Lecture 1). The vector $b = As + e$ is, by construction, a point $As \in \Lambda_q(A)$ that has been nudged by a *small* vector e. Recovering s from b is therefore exactly **Bounded-Distance Decoding**: given a target b that is guaranteed to be *unusually close* to some lattice point (closer than a typical random point would be), find that lattice point. This is CVP's easier, promise-bearing cousin — and, exactly as with CVP in Section 2.3.2, solving it reliably requires something like a good basis for $\Lambda_q(A)$. The whole point of choosing A uniformly at random is that no such good basis is visible to an attacker; only whoever *generated* s (or knows a matching trapdoor) has an edge.

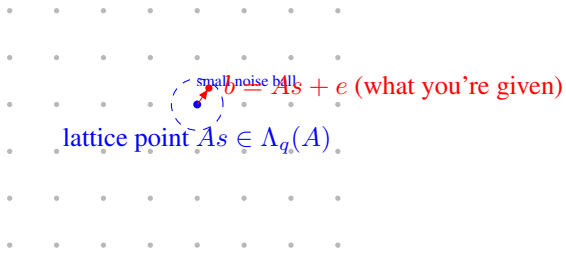


Figure 2.6: LWE as Bounded-Distance Decoding: b is guaranteed to land inside a small ball around some lattice point As — unlike general CVP (Figure 2.3), where the target could be anywhere. That extra promise ("close, not just closest") is what makes BDD tractable *with a good basis* and still believed hard *without one*.

Fact: Regev's Worst-Case-to-Average-Case Reduction (Statement Only)

A landmark 2005 result (Oded Regev) shows: if there existed an efficient algorithm that solves *average-case, randomly-generated* decision-LWE instances, that algorithm could be turned into an efficient algorithm for *worst-case* GapSVP and other hard lattice problems (via a quantum reduction; later work gave classical reductions for related problems). In plain terms: **LWE's hardness is not a bet made up from nothing** — it inherits its credibility from the same worst-case lattice hardness Lecture 1 built toward. This is exactly why cryptographers trust random LWE instances, generated fresh for every key, to be hard: hardness on average is tied by proof to hardness in the worst case. We take this as given; the proof itself is well beyond this course.

Looking Ahead: Search vs. Decision — Also Provably Equivalent

It's also a known theorem (not proven here) that search-LWE and decision-LWE are polynomially equivalent: an efficient solver for one can be converted into an efficient solver for the other. This matters practically because security *proofs* for encryption schemes are usually stated against decision-LWE ("ciphertexts are indistinguishable from random"), while *attacks* usually aim at search-LWE ("recover the actual secret key"). Session 5 will use both framings, so it's worth knowing they're two views of the same underlying hardness.

2.6 From Plain LWE to Ring-LWE

2.6.1 The Problem With Plain LWE: Size

A single LWE public key with security parameter n requires an $m \times n$ matrix A over $\mathbb{Z}_q$ — roughly n^2 numbers. For cryptographically secure parameters (n in the many hundreds), that's an enormous public key, and every encryption/decryption operation costs a full matrix-vector multiplication, $O(n^2)$ work. Real deployed schemes need public keys measured in kilobytes and operations fast enough for a TLS handshake or a smartcard, not this.

Example: Putting Real Numbers On "Too Big"

Take a security-relevant dimension, $n = 1024$, with q a few thousand (so each number needs about 12 bits ≈ 1.5 bytes to store). A plain-LWE public key needs roughly $n^2 = 1{,}048{,}576$ such numbers — over 1.5 megabytes, just for one public key, before even discussing ciphertexts. Compare that to a real ML-KEM-1024 public key, which is about 1,568 *bytes* in total — roughly a thousand times smaller. That gap is exactly what Ring-LWE and Module-LWE close, by replacing an $n \times n$ grid of independent numbers with a much smaller number of *structured* ring elements that each silently represent n numbers at once (Section 2.6.3's remark makes precise how).

2.6.2 The Fix: Move Into a Polynomial Ring

Recall from Lecture 1's "Ring" section and its preview box: Kyber and Dilithium do arithmetic inside $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ — polynomials of degree $< n$ with coefficients in $\mathbb{Z}_q$, where powers of x wrap around using the rule $x^n \equiv -1$.

Definition: The Ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$

Elements of R_q are polynomials $a(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{n-1} x^{n-1}$ with each coefficient $a_i \in \mathbb{Z}_q$. Addition is ordinary coefficient-wise addition mod q. Multiplication is ordinary polynomial multiplication, *followed by reduction*: whenever a term x^n appears, replace it with -1 (and more generally $x^{n+k} \to -x^k$), then reduce every coefficient mod q. This makes R_q a finite ring with exactly q^n elements — and, crucially, a single element of R_q packs n numbers into one object that a computer can multiply in roughly $O(n \log n)$ time using the Number-Theoretic Transform (NTT, a finite-field cousin of the FFT) — we will not need NTT details in this course, just the fact that it's the reason Ring-LWE is fast in practice.

Example: Multiplying Two Small Polynomials in R_q, $n = 4$, $q = 17$

Let $n = 4$, so we reduce using $x^4 \equiv -1$, and work mod $q = 17$. Take $a(x) = 1 + 2x$ and $c(x) = 3 + x^3$ in R_{17}. Ordinary polynomial multiplication first:

$$a(x)\, c(x) = (1 + 2x)(3 + x^3) = 3 + x^3 + 6x + 2x^4.$$

Now reduce using $x^4 \equiv -1$: the term $2x^4$ becomes $2 \cdot (-1) = -2$. So we're left with

$$3 + 6x + x^3 - 2 \;=\; 1 + 6x + 0x^2 + x^3.$$

Finally reduce coefficients mod 17 (here every coefficient is already in $\{0, \ldots, 16\}$, so nothing changes): the product is $1 + 6x + x^3 \in R_{17}$. Notice the "wraparound" step ($x^4 \to -1$) is exactly what folds the high-degree term back down to degree < 4 — this is the polynomial-ring analogue of "carrying" in ordinary mod-q arithmetic.

2.6.3 Ring-LWE, Formally

Definition: Ring Learning With Errors (RLWE)

Fix $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ and an error distribution χ over R_q (coefficients drawn independently from a small distribution). Let $s \in R_q$ be a fixed secret ring element. A search-RLWE sample is a pair $(a, b) \in R_q \times R_q$ where a is uniform random and

$$b = a \cdot s + e \pmod{q}, \qquad e \leftarrow \chi.$$

Search-RLWE asks for s given many such pairs; decision-RLWE asks to distinguish (a, b) from a uniformly random pair in $R_q \times R_q$.

Remark: One Ring-LWE Sample $\approx n$ Plain-LWE Samples

Compare the two definitions side by side: plain LWE needs an entire matrix $A \in \mathbb{Z}_q^{m \times n}$ to state m equations. A *single* Ring-LWE pair (a, b), because a and s are each secretly a length-n coefficient vector multiplied together via the structured polynomial product above, is algebraically equivalent to about n ordinary LWE-style equations bundled into one ring multiplication. This is exactly where the size and speed win comes from — and exactly why the ring's extra structure is a trade-off: it buys efficiency at the cost of a stronger (though still, so far, unbroken) hardness assumption than plain LWE, since the extra algebraic structure is, in principle, more surface area for a future attack to exploit.

2.7 Module-LWE: the Middle Ground Kyber and Dilithium Actually Use

Definition: Module Learning With Errors (MLWE)

Fix R_q as above and a small integer k (the **module rank**). The secret is now a length-k vector of ring elements, $\vec{s} \in R_q^k$. A sample is $(\vec{a}, b)$ with $\vec{a} \in R_q^k$ uniform random and

$$b = \langle \vec{a}, \vec{s} \rangle + e \pmod{q} = \sum_{i=1}^{k} a_i s_i + e, \qquad e \leftarrow \chi.$$

Remark: MLWE Sits Exactly Between Plain LWE and Ring-LWE

Set $k = 1$ and MLWE collapses to Ring-LWE exactly. Let $n = 1$ (no ring structure at all — just plain integers) and MLWE collapses toward plain LWE. Choosing $k > 1$ with a modest per-ring dimension n gives designers a dial: more of the speed benefit of rings, without committing to the single largest, most algebraically rigid ring dimension. This is precisely why Kyber and Dilithium use MLWE with a moderate fixed $n = 256$ and vary k (Kyber uses $k \in \{2, 3, 4\}$ for its three security levels) instead of varying n directly — a design choice we will see in full in Session 3.

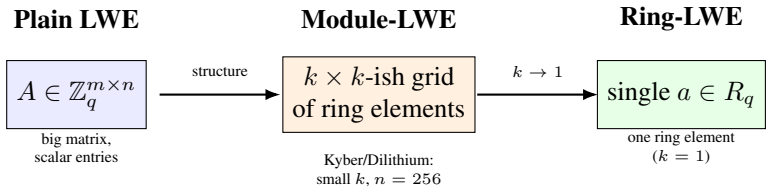


Figure 2.7: Module-LWE is a size/speed dial between the two extremes: no ring structure at all (plain LWE, left) and maximal ring structure (Ring-LWE, right). Kyber and Dilithium sit in the middle deliberately.

2.8 Bridge to Lecture 3: A Real Public Key *Is* an (M)LWE Sample

When we open up Kyber (ML-KEM) in Lecture 3, its key generation will look almost exactly like the MLWE definition above: pick a random public matrix (of ring elements) $\mathbf{A}$, pick a small secret $\vec{s}$ and small error $\vec{e}$, and publish $\mathbf{t} = \mathbf{A}\vec{s} + \vec{e}$ as the public key — literally an MLWE sample, with $\vec{s}$ kept as the secret key. Encryption uses a *second*, independent MLWE-style sample to hide the message inside more noise. Once you can point at a real Kyber public key and say "that's $b = As + e$," the scheme stops looking like magic and starts looking like an application of exactly the object defined in Section 2.7.

2.9 Summary Table

Problem	Given	Find / Decide
SVP	Basis B of lattice L	Shortest nonzero $v \in L$
CVP	Basis B, target $t \notin L$	Closest $v \in L$ to t
SIS	Random $A \in \mathbb{Z}_q^{n \times m}$	Short nonzero x: $Ax \equiv 0$
SIS hash $h_A(x) = Ax$	Same A, input $x \in \{0,1\}^m$	Compressed output in $\mathbb{Z}_q^n$; collisions $\Rightarrow$ SIS solutions
Search-LWE	$(A, b = As + e)$	Recover secret s
Decision-LWE	(A, b)	Is b "$As + e$" or uniform random?
Ring-LWE	$(a, b = as + e) \in R_q^2$	Recover $s \in R_q$
Module-LWE	$(\vec{a}, b) \in R_q^k \times R_q$	Recover $\vec{s} \in R_q^k$

2.10 Full List of Exercises (Assign as Homework Before Lecture 3)

Exercise: Homework Set

1. State, in your own words (2–3 sentences each), the difference between SVP and CVP, and the difference between the search and decision versions of a problem in general.
2. For $n = 1$, $m = 3$, $q = 13$, $A = \begin{pmatrix} 4 & 6 & 9 \end{pmatrix}$: find a nonzero, short $x \in \mathbb{Z}^3$ solving the SIS instance $Ax \equiv 0 \pmod{13}$ (Section 2.4 worked example, above). Confirm your answer by direct computation.
3. Complete the SIS-hash practice exercise in Section 2.4.1, if you have not already: find a genuine colliding pair for h_A and confirm it yields a valid SIS solution.
4. Redo the search-LWE worked example (Section 2.5.4) with secret $s = (2, 9, 1)$ but a *new* sample $a_5 = (2, 0, 1)$ and error $e_5 = 1$. Compute b_5. If you were only given a_5 and b_5 (not s or e_5), could you determine s from this single equation alone? Why not?
5. Explain in your own words why LWE would become *easy* (not hard) if the error e were removed entirely (i.e., $b = As$ exactly). Connect your answer to ordinary linear algebra.
6. In the ring $R_{17} = \mathbb{Z}_{17}[x]/(x^4 + 1)$: compute the product $(2 + x^2)(1 + 3x)$, showing the wraparound step explicitly (Section 2.6.2 worked example, above).
7. In 3–4 sentences: why does moving from plain LWE to Ring-LWE reduce key size roughly from $O(n^2)$ numbers to $O(n)$ numbers? Use the "one Ring-LWE sample $\approx n$ plain-LWE samples" remark (Section 2.6.3) in your explanation.
8. Module-LWE with $k = 1$ is exactly Ring-LWE, and (informally) R_q with $n = 1$ is exactly $\mathbb{Z}_q$. Using these two facts, explain why Module-LWE is fairly described as "a generalization that contains both plain LWE and Ring-LWE as special cases."
9. **Looking ahead:** Kyber-768 uses module rank $k = 3$ with per-ring dimension $n = 256$ and $q = 3329$. Without looking anything up yet, estimate how many total $\mathbb{Z}_q$-coefficients are packed into one Kyber-768 secret key $\vec{s} \in R_q^3$. (You'll check this answer in Session 3.)

Lesson 3: ML-KEM (Kyber) — A Real Scheme, End to End

Parameters, Compression, CPA-Secure Encryption, and the Route to CCA Security

Purpose of this lecture. Lecture 2 ended with a promise: a real Kyber public key would turn out to be "just an MLWE sample." This lecture cashes that promise in. We fix concrete parameters, define the compression functions real ciphertexts use, write out Kyber's core encryption scheme (KeyGen / Enc / Dec) algorithm-by-algorithm, prove — with real numbers — why decryption actually recovers the message, and then explain the one extra transformation (Fujisaki–Okamoto) that turns this into the standardized, chosen-ciphertext-secure **ML-KEM (FIPS 203)**. That last step deserves special attention: it is exactly the part of the scheme most often targeted by the physical attacks in Section 2 of the proposal.

3.1 Recap: What You Already Have

From Lecture 2: Module-LWE says that, for a small integer k and $R_q = \mathbb{Z}_q[x]/(x^n + 1)$, the pair $(\vec{a}, b = \langle \vec{a}, \vec{s} \rangle + e)$ is computationally indistinguishable from uniform random, for secret $\vec{s} \in R_q^k$ and small error e. Today, $\vec{a}$ becomes a whole *matrix* A of ring elements (one MLWE sample per row), $\vec{s}$ becomes the actual Kyber secret key, and b's vector analogue $\vec{t} = A\vec{s} + \vec{e}$ becomes the actual Kyber public key. Nothing new needs to be invented — we are about to watch Lecture 2's abstract object become a working encryption scheme.

3.2 Concrete Parameters

Definition: ML-KEM Parameter Sets

All three standardized parameter sets share $n = 256$ (ring dimension) and $q = 3329$ (modulus, a prime chosen so $q \equiv 1 \pmod{256}$, which makes the NTT fast — we will not need the NTT itself, just this reason for q's odd-looking value). They differ only in the module rank k and the noise parameters η_1, η_2 (which control how "wide" the centered binomial noise distribution — Lecture 2, Section 2.5.3 — is):

Parameter set	k	η_1	η_2	Public-key size
ML-KEM-512	2	3	2	800 bytes
ML-KEM-768	3	2	2	1,184 bytes
ML-KEM-1024	4	2	2	1,568 bytes

Remark: Reading This Table With Lecture 2's Eyes

k is exactly the module rank from Lecture 2, Section 2.7 — the secret key is a length-k vector of ring elements, $\vec{s} \in R_q^k$, and the public matrix A is $k \times k$ (ring elements). Larger k means more MLWE "dimensions" to search over, hence higher security — and a bigger public key, since you're now storing k ring elements (each 256 coefficients) instead of one. This is the size/security dial promised back in Lecture 2's Figure 2.7.

3.3 Compression: Trading Precision for Size

A ciphertext, as defined so far, would consist of full elements of R_q — every one of 256 coefficients stored with full precision mod $q = 3329$ (about 12 bits each). Kyber shrinks ciphertexts by deliberately throwing away low-order bits of precision, in a controlled way.

Definition: Compression and Decompression

For an integer $d < \lceil \log_2 q \rceil$, define, for $x \in \mathbb{Z}_q$:

$$\text{Compress}_d(x) = \left\lceil \frac{2^d}{q} \cdot x \right\rfloor \bmod 2^d, \qquad \text{Decompress}_d(y) = \left\lceil \frac{q}{2^d} \cdot y \right\rfloor,$$

where $\lceil \cdot \rfloor$ denotes rounding to the nearest integer. Compression maps a $\mathbb{Z}_q$ value down to a d-bit value; decompression maps it back up to (approximately) the original value. Applied coefficient-by-coefficient, these extend to compressing/decompressing an entire ring element.

Example: Compressing and Decompressing by Hand, $q = 17$, $d = 2$

With $q = 17$ and $d = 2$ (so values get squeezed down to 2 bits $\in \{0, 1, 2, 3\}$): take $x = 9$. Then $\text{Compress}_2(9) = \lceil \frac{4}{17} \cdot 9 \rfloor = \lceil 2.12 \rfloor = 2$. Decompressing: $\text{Decompress}_2(2) = \lceil \frac{17}{4} \cdot 2 \rfloor = \lceil 8.5 \rfloor = 9$ (or 8, depending on rounding convention for the exact half-way case) — in this instance we land back exactly on 9. In general you do *not* get back the original value exactly — that's the entire point: Compress then Decompress introduces a small, bounded rounding error, in exchange for needing only $d = 2$ bits instead of $\lceil \log_2 17 \rceil = 5$ bits to store or transmit x.

Remark: Why Deliberately Add More Error?

This seems backwards — Lecture 2 spent an entire lecture explaining that noise is what makes LWE hard, and now we're adding *more* noise on purpose. The resolution: encryption already tolerates a noise budget (Section 3.5 below shows exactly how much). Compression consumes a small, carefully bounded slice of that budget in exchange for a large reduction in ciphertext size — as long as the total noise (compression error *plus* the original LWE error) stays under the budget, decryption still succeeds. Push d too low, though, and decryption starts failing — this is a real, quantifiable trade-off the standard's parameters were chosen to balance, and it's also precisely the kind of margin that a fault attack (Session 5) tries to push past its limit on purpose.

3.4 The CPA-Secure Core: Kyber.CPAPKE

We now write out the actual scheme. To keep hand-computation possible later, we describe it at the module level using Lecture 2's notation directly; A^T denotes the transpose of the matrix of ring elements, and all arithmetic is in R_q.

Algorithm 1 Kyber.CPAPKE.KeyGen

1: Generate $A \in R_q^{k \times k}$ uniformly at random (in practice: expanded deterministically from a short public seed ρ via a hash-based expander, so only ρ — not all of A — needs to be stored or sent)
2: Sample $\vec{s} \in R_q^k$, each coefficient from the small noise distribution (centered binomial, parameter η_1)
3: Sample $\vec{e} \in R_q^k$ the same way
4: Compute $\vec{t} = A\vec{s} + \vec{e} \in R_q^k$
5: **return** public key $pk = (A, \vec{t})$, secret key $sk = \vec{s}$

Algorithm 2 Kyber.CPAPKE.Enc$(pk = (A, \vec{t}),\ m \in \{0,1\}^n)$

1: Sample $\vec{r} \in R_q^k$, $\vec{e}_1 \in R_q^k$ from the small noise distribution (parameter η_1), and $e_2 \in R_q$ (parameter η_2)
2: Compute $\vec{u} = A^T\vec{r} + \vec{e}_1 \in R_q^k$
3: Encode the message: $\mathrm{Encode}(m) \in R_q$ has i-th coefficient $\lceil q/2 \rceil \cdot m_i$ (each bit becomes either 0 or "as far from 0 as possible mod q")
4: Compute $v = \vec{t}^T\vec{r} + e_2 + \mathrm{Encode}(m) \in R_q$
5: **return** ciphertext $c = \big(\mathrm{Compress}_{d_u}(\vec{u}),\ \mathrm{Compress}_{d_v}(v)\big)$

Algorithm 3 Kyber.CPAPKE.Dec$(sk = \vec{s},\ c = (c_u, c_v))$

1: Recover $\vec{u} = \mathrm{Decompress}_{d_u}(c_u)$, $v = \mathrm{Decompress}_{d_v}(c_v)$
2: Compute $m' = v - \vec{s}^T\vec{u} \in R_q$
3: **return** $\mathrm{Decode}(m')$: for each coefficient of m', output bit 1 if it is closer to $\lceil q/2 \rceil$ than to 0 (mod q), else bit 0

Remark: Recognizing Lecture 2 Inside This Algorithm

Line 4 of KeyGen, $\vec{t} = A\vec{s} + \vec{e}$, is exactly an MLWE sample, verbatim, from Lecture 2 Section 2.7. Encryption generates a *second*, independent MLWE-style sample ($\vec{u}$) using a fresh secret $\vec{r}$, and hides the message inside a third quantity v that mixes the public key with that same $\vec{r}$. Nothing about the shape of these equations is new — only the bookkeeping (which noise term uses which name, and the extra Encode/Compress steps) is specific to Kyber.

3.5 Why Decryption Works: the Noise-Cancellation Argument

Remark: The Algebra Behind Line 2 of Decryption (Ignoring Compression for a Moment)

Substitute the definitions of $\vec{u}$ and v into $m' = v - \vec{s}^T\vec{u}$:

$$\begin{aligned} m' &= \big(\vec{t}^T\vec{r} + e_2 + \mathrm{Encode}(m)\big) - \vec{s}^T\big(A^T\vec{r} + \vec{e}_1\big) \\ &= \vec{t}^T\vec{r} - \vec{s}^T A^T\vec{r} + e_2 - \vec{s}^T\vec{e}_1 + \mathrm{Encode}(m) \\ &= (A\vec{s} + \vec{e})^T\vec{r} - \vec{s}^T A^T\vec{r} + e_2 - \vec{s}^T\vec{e}_1 + \mathrm{Encode}(m) \qquad \text{(substituting } \vec{t} = A\vec{s} + \vec{e}\text{)} \\ &= \vec{s}^T A^T\vec{r} + \vec{e}^T\vec{r} - \vec{s}^T A^T\vec{r} + e_2 - \vec{s}^T\vec{e}_1 + \mathrm{Encode}(m) \\ &= \mathrm{Encode}(m) + \underbrace{\big(\vec{e}^T\vec{r} + e_2 - \vec{s}^T\vec{e}_1\big)}_{\text{total noise, call it } \varepsilon}. \end{aligned}$$

The two copies of $\vec{s}^T A^T\vec{r}$ **cancel exactly** — this is the whole trick, and it's why the scheme is designed around A appearing on both sides (once via $\vec{t}$, once via $\vec{u}$). What's left is the message, plus a total noise term ε built entirely from small quantities ($\vec{e}, \vec{r}, e_2, \vec{s}, \vec{e}_1$ are all "small" by construction). As long as every coefficient of ε stays smaller than $q/4$ in absolute value, m''s coefficients land close enough to 0 or $\lceil q/2 \rceil$ that Decode recovers m exactly.

Remark: Decryption Failure: a Real, Quantifiable Possibility

If ε's coefficients happen to be unusually large (all the small noise terms landing far from 0 by chance, or compression eating too much of the remaining budget), decryption can output the *wrong* bit. This isn't a flaw — it's an accepted, carefully bounded probability (ML-KEM's parameters keep it astronomically small, far below 2^{-100} per ciphertext) baked into the design. It matters for your project for a specific reason: some physical attacks work by *deliberately inducing* decryption failures (via faults, or via chosen ciphertexts that sit right at the edge of the noise budget) and then use the pattern of failures to leak information about $\vec{s}$ — Session 5's material on Kyber's central-reduction leak and CCA-style attacks builds directly on this margin.

3.6 A Fully Worked Example, By Hand

Example: Kyber-Style Encryption/Decryption, $k = 1$, $n = 4$, $q = 17$ (Verified by Hand)

Take $R_{17} = \mathbb{Z}_{17}[x]/(x^4+1)$ and module rank $k = 1$ (so everything below is a single ring element, not a vector — this is really Ring-LWE-flavored Kyber, kept at $k = 1$ purely so the arithmetic stays hand-tractable).

KeyGen. Let $A = 3 + 5x + 2x^2 + x^3$ (public). Let $s = 1 - x$ (secret, small coefficients) and $e = 1$ (small error). Multiplying in R_{17} (using $x^4 \equiv -1$, exactly as in Lecture 2's polynomial-multiplication example):

$$A \cdot s = 4 + 2x + 14x^2 + 16x^3, \qquad t = A \cdot s + e = 5 + 2x + 14x^2 + 16x^3.$$

Public key: (A, t). Secret key: $s = 1 - x$.

Enc. Message $m = (1, 0, 1, 0)$ (four bits, one per coefficient). Encode with threshold value $9 \approx \lceil 17/2 \rceil$: $\text{Encode}(m) = 9 + 0x + 9x^2 + 0x^3$. Sample small randomness $r = x$, $e_1 = 1$, $e_2 = -1$. Compute:

$$u = A \cdot r + e_1 = 3x + 5x^2 + 2x^3 + x^4 + 1 = 0 + 3x + 5x^2 + 2x^3,$$

$$\begin{aligned} v &= t \cdot r + e_2 + \text{Encode}(m) \\ &= (1 + 5x + 2x^2 + 14x^3) + (-1) + (9 + 0x + 9x^2 + 0x^3) \\ &= 9 + 5x + 11x^2 + 14x^3. \end{aligned}$$

(We skip Compress/Decompress here to keep the arithmetic focused — Section 3.3's example already showed compression separately.) Ciphertext: $c = (u, v)$.

Dec. Compute $s \cdot u = 2 + 3x + 2x^2 + 14x^3$, then

$$m' = v - s \cdot u = (9 - 2) + (5 - 3)x + (11 - 2)x^2 + (14 - 14)x^3 = 7 + 2x + 9x^2 + 0x^3.$$

Decode each coefficient (closer to 9 than to 0, mod 17, means bit 1): $7 \to 1$, $2 \to 0$, $9 \to 1$, $0 \to 0$. **Recovered message: $(1, 0, 1, 0)$ — exactly the original.** Every step above is real Kyber structure, just at toy scale; nothing was simplified away except using $k = 1$ instead of $k \in \{2, 3, 4\}$ and full 256-coefficient rings.

3.7 From CPA-Secure Encryption to a CCA-Secure KEM

Kyber.CPAPKE, as just described, is only secure against an attacker who never gets to submit chosen ciphertexts for decryption (a **CPA** — chosen-plaintext-attack — guarantee). Real protocols (TLS, VPNs) need more: an attacker who *can* submit arbitrary ciphertexts and observe what happens (timing, error

messages, side channels) still shouldn't learn anything. That stronger notion is **CCA security**, and it's what ML-KEM actually provides — via a generic transformation, not a redesign of the algebra above.

Definition: Key Encapsulation Mechanism (KEM)

A KEM replaces "encrypt an arbitrary message" with a narrower, safer interface: $\mathrm{Encaps}(pk)$ outputs *both* a ciphertext c *and* a fresh shared secret K; $\mathrm{Decaps}(sk, c)$ recovers the same K. Neither party ever chooses K directly — it's generated as a by-product of encapsulation. K is then used as a symmetric key for the rest of the session (e.g., to key AES inside TLS).

Remark: The Fujisaki–Okamoto (FO) Transform, Informally

The key idea: instead of encrypting a message chosen by an attacker, **encrypt a message that only the honest sender ever knows**, and make the ciphertext itself depend on that message in a way the receiver can double-check. Concretely, ML-KEM's Encaps generates a fresh random message m, derives both the encryption randomness *and* the final shared secret K from m (via hashing), and encrypts m using Kyber.CPAPKE as above. On the receiving end, Decaps does something distinctive: it decrypts c to recover a candidate m', then **re-encrypts m' using the exact same (deterministic, derived-from-m') randomness** and checks whether it gets back the *same* ciphertext c. If yes, K is derived from m' normally. If the re-encryption check fails — meaning c wasn't a ciphertext that any honest sender would have produced — **Decaps does not error out**. Instead it returns a *different*, pseudorandom-looking K (derived from the secret key and c together, "implicit rejection"), indistinguishable to an outside observer from a legitimate key.

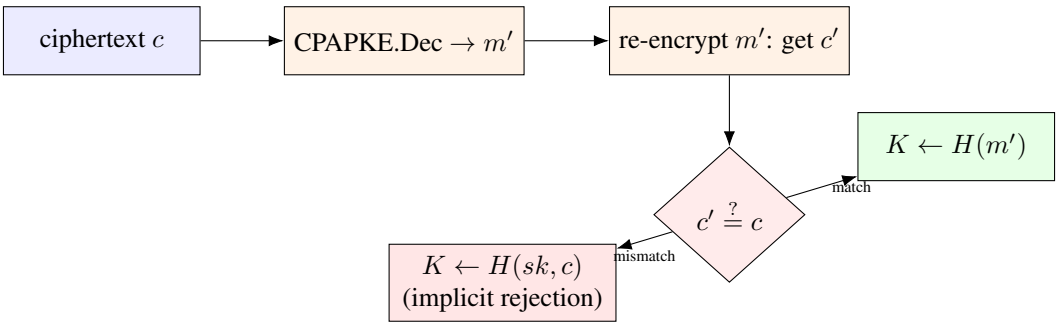


Figure 3.1: ML-KEM's Decaps, schematically. The re-encryption check ($c' \stackrel{?}{=} c$) is the step the Fujisaki–Okamoto transform adds on top of plain Kyber.CPAPKE decryption — and, because both branches must be computed in a way that reveals nothing about *which* branch was taken (equal time, no data-dependent memory access), it is also one of the most scrutinized few lines of code in the entire scheme from a side-channel perspective.

Looking Ahead: Why This One Step Gets an Entire Attack Literature (Preview of Session 5)

Two properties make the FO re-encryption check a magnet for physical attacks. First, it is a *comparison* whose outcome (match vs. mismatch) must never leak — if an attacker can learn, via timing or power consumption, whether a chosen malformed ciphertext passed or failed this check, they can often turn many such queries into a full key-recovery attack (this is the shape of several published Kyber side-channel results). Second, the intermediate value m' computed on line 1 (*before* the check) briefly exists in memory even for a ciphertext that will ultimately be rejected — if that intermediate computation itself leaks (e.g. through the CPAPKE decryption arithmetic touching the secret $\vec{s}$), the rejection branch doesn't help you, because the damage already happened one step earlier. Keep both of these observations in mind; Session 5 names specific published attacks that exploit exactly this structure.

3.8 Deployment Status

ML-KEM is not experimental. It was finalized by NIST as **FIPS 203** in August 2024, alongside ML-DSA (FIPS 204) and SLH-DSA (FIPS 205).[1] It is already deployed at scale: Chrome enabled hybrid ML-KEM by default in late 2024, Apple's iMessage uses it as part of PQ3, and the major cloud providers (AWS, Google Cloud, Microsoft Azure) support PQC-enabled TLS endpoints. In these hybrid deployments, ML-KEM typically runs *alongside* a classical algorithm like X25519 rather than replacing it outright, so that security holds even if one of the two turns out to have an unforeseen weakness.

3.9 Summary Table

Object	**What it is, in Lecture 2's language**
Public key $(A, \vec{t})$	An MLWE sample: $\vec{t} = A\vec{s} + \vec{e}$
Secret key $\vec{s}$	The MLWE secret
Ciphertext part $\vec{u}$	A second, independent MLWE sample using fresh randomness $\vec{r}$
Ciphertext part v	Public key mixed with $\vec{r}$, plus the message, plus noise
Decryption	Noise cancels algebraically, leaving message + small residual noise
CPAPKE $\rightarrow$ ML-KEM	Fujisaki–Okamoto transform: re-encrypt and check, or implicitly reject

[1] NIST, *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, FIPS 203, August 2024.

3.10 Exercises (Assign Before Lecture 4)

Exercise: Homework Set

1. Redo the compression worked example (Section 3.3) with $d = 3$ instead of $d = 2$ (still $q = 17$). Is the recovered value after decompression closer to the original 9 than it was with $d = 2$? Explain why, in general, larger d means smaller compression error.
2. In the noise-cancellation derivation (Section 3.5), identify by name every term that ends up inside ε. Which of these terms come from key generation, and which come from encryption?
3. Using the worked example in Section 3.6, redo encryption with a *different* message $m = (0, 1, 0, 1)$, keeping the same A, s, e, r, e_1, e_2. Compute u, v, and confirm decryption recovers $(0, 1, 0, 1)$.
4. In your own words (4–5 sentences): why is it not enough for ML-KEM's Decaps to simply "return an error" when the re-encryption check fails, instead of the implicit-rejection mechanism in Section 3.7? Think about what an attacker could learn from the mere presence or timing of an error message.
5. **Looking ahead:** skim the abstract of one published Kyber side-channel paper of your choosing (we will assign specific ones in Session 5) and identify which single line of the algorithms in Section 3.4 of this lecture it targets — KeyGen, Enc, Dec, or the FO check in Section 3.7.

Lesson 4: ML-DSA (Dilithium) and FN-DSA (Falcon)

Signing With Lattices: Fiat–Shamir-with-Aborts and Trapdoor Sampling

Purpose of this lecture. Lecture 3 built an encryption scheme out of MLWE. Today we build *signature* schemes — a different goal (proving you hold a secret, not hiding a message) that leads to genuinely different designs. We cover **ML-DSA (Dilithium)**, which reuses MLWE directly inside a technique called Fiat–Shamir-with-Aborts, and **FN-DSA (Falcon)**, which uses a different lattice (NTRU) and a different technique (GPV trapdoor sampling) entirely. Both sections end by naming the exact step in each algorithm that Session 5's physical attacks target — that mapping is the main deliverable of this lecture.

4.1 What a Signature Scheme Needs to Do

Encryption (Lecture 3) hides information; a signature does the opposite — it *proves* something publicly, without hiding anything. A signature scheme has three algorithms: KeyGen (produces a public/secret key pair), $\mathrm{Sign}(sk, \text{message})$ (produces a signature), and $\mathrm{Verify}(pk, \text{message}, \text{signature})$ (accepts or rejects). The security goal is **unforgeability**: without sk, no one should be able to produce a signature that Verify accepts, even after seeing many genuine signatures on messages of their choosing.

Remark: The Core Design Tension

A signature must depend on the secret key (otherwise anyone could forge it) — but it must not depend on the secret key in a way that *leaks* it, even after an attacker collects thousands of signatures. Both schemes in this lecture are, at heart, different engineering answers to that one tension. Keep this framing in mind; it's the thread connecting today's material straight through to Session 5.

4.2 A Primer on Fiat–Shamir (Needed Before Dilithium Makes Sense)

Definition: Fiat–Shamir Transform, Informally

Start with a three-move *interactive* protocol between a Prover (who knows a secret) and a Verifier: (1) Prover sends a **commitment** w (built from fresh randomness, independent of the secret); (2) Verifier sends a random **challenge** c; (3) Prover computes a **response** z that combines w's randomness with the secret and c, and the Verifier checks a consistency equation relating w, c, z, and the public key. The Fiat–Shamir transform removes the Verifier: the Prover computes c themself, as a hash of w (and the message being signed), $c = H(w, \text{message})$. Because H behaves unpredictably, the Prover still can't predict c before committing to w — so the security argument survives, but now there's no interaction: the whole transcript (w, c, z) can be published as a non-interactive signature.

Remark: Why This Detour Was Necessary

Every part of Dilithium's Sign algorithm below — the commitment w, the challenge c, the response z — is exactly this three-move shape, with the interaction removed via hashing. Without seeing the general pattern first, Dilithium's algorithm looks like an arbitrary pile of steps; with it, each line has an obvious job.

4.3 ML-DSA (Dilithium): Parameters

Definition: ML-DSA Parameter Sets

All parameter sets share the familiar ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ with $n = 256$, this time with $q = 8{,}380{,}417$ (a different prime from Kyber's — still chosen for NTT-friendliness). The secret is a pair of small vectors $(\vec{s}_1, \vec{s}_2) \in R_q^\ell \times R_q^k$; the public key is built from an MLWE-style sample using them.

Parameter set	(k, ℓ)	Signature size	Public-key size
ML-DSA-44	(4,4)	2,420 bytes	1,312 bytes
ML-DSA-65	(6,5)	3,309 bytes	1,952 bytes
ML-DSA-87	(8,7)	4,627 bytes	2,592 bytes

4.4 ML-DSA: KeyGen, Sign, Verify

Algorithm 4 ML-DSA.KeyGen

1: Generate $A \in R_q^{k \times \ell}$ uniformly at random (again, from a short public seed)
2: Sample small $\vec{s}_1 \in R_q^\ell$, $\vec{s}_2 \in R_q^k$ (coefficients bounded by a small η)
3: Compute $\vec{t} = A\vec{s}_1 + \vec{s}_2 \in R_q^k$
4: **return** $pk = (A, \vec{t})$, $sk = (\vec{s}_1, \vec{s}_2)$

Remark: Yes, This Is an MLWE Sample Too

Line 3 is identical in shape to Kyber's KeyGen (Lecture 3). The two schemes start from the *same* hard problem; what differs entirely is what you do with the key afterward.

Algorithm 5 ML-DSA.Sign$(sk = (\vec{s}_1, \vec{s}_2),\ \text{msg})$ — Simplified

1: Sample a masking vector $\vec{y} \in R_q^\ell$ with somewhat larger small coefficients than $\vec{s}_1, \vec{s}_2$ (derived deterministically from sk and msg via a PRF, not from fresh randomness — see remark below)
2: Compute the commitment $\vec{w} = A\vec{y} \in R_q^k$
3: Compute the challenge $c = H(\vec{w}, \text{msg})$ — a specially-shaped "sparse" small polynomial
4: Compute the response $\vec{z} = \vec{y} + c\vec{s}_1$
5: **Rejection check:** if any coefficient of $\vec{z}$ (or of a related quantity built from $\vec{w} - c\vec{s}_2$) is too large in absolute value, **discard everything and go back to step 1** with fresh $\vec{y}$
6: Once the check passes, **return** signature $(\vec{z}, c)$ (in the real scheme, a compact *hint* $\vec{h}$ is also included, letting the verifier reconstruct $\vec{w}$'s high-order bits exactly — omitted here for simplicity)

Algorithm 6 ML-DSA.Verify$(pk = (A, \vec{t}),\ \text{msg},\ (\vec{z}, c))$ — Simplified

1: Check that every coefficient of $\vec{z}$ is within the allowed bound (reject if not)
2: Recompute $\vec{w}' = A\vec{z} - c\vec{t}$
3: Accept if $c = H(\vec{w}', \text{msg})$; reject otherwise

Remark: Why Verification Works: the Algebra

Substitute: $\vec{w}' = A\vec{z} - c\vec{t} = A(\vec{y} + c\vec{s}_1) - c(A\vec{s}_1 + \vec{s}_2) = A\vec{y} + cA\vec{s}_1 - cA\vec{s}_1 - c\vec{s}_2 = A\vec{y} - c\vec{s}_2 = \vec{w} - c\vec{s}_2$. The two copies of $cA\vec{s}_1$ cancel — the same style of cancellation as Kyber's decryption in Lecture 3. $\vec{w}'$ isn't *exactly* $\vec{w}$; it's $\vec{w}$ shifted by a small quantity $c\vec{s}_2$. Real ML-DSA's "high bits" / hint mechanism (omitted above) is specifically engineered so that this small shift doesn't change the *high-order* bits of $\vec{w}$, so $H(\vec{w}', \text{msg})$ still reproduces the same challenge c the signer used. Our simplified version above sidesteps this by just checking exact equality of the full hash input in a toy example (Section 4.6) small enough that no shift occurs.

4.5 Why Rejection Sampling Exists (This Is the Important Part)

Remark: The Problem Rejection Sampling Solves

Look at line 4 above: $\vec{z} = \vec{y} + c\vec{s}_1$. If we *always* output $\vec{z}$, its distribution would depend — ever so slightly — on the secret $\vec{s}_1$, because $c\vec{s}_1$ is added on every single signature. Collect enough signatures on enough messages, and that slight dependence becomes a statistical attack surface: an attacker who sees many $(\vec{z}, c)$ pairs could, in principle, average out the masking $\vec{y}$ and recover information about $\vec{s}_1$. **Rejection sampling closes this gap**: $\vec{y}$ is deliberately sampled from a range wide enough that, *conditioned on being accepted*, $\vec{z}$'s distribution is statistically independent of $\vec{s}_1$ — indistinguishable from $\vec{z}$ having been sampled uniformly from the accepted range, with no trace of the secret at all. The rejected attempts simply never get published; only "safe-looking" signatures ever leave the signer.

Fact: Deterministic Signing, and Why It's a Double-Edged Sword

Step 1 above notes that $\vec{y}$ is derived *deterministically* from (sk, msg) via a pseudorandom function, rather than from fresh random bits each time. This mirrors a well-known fix (RFC 6979-style) for a classical signature failure mode: if two different messages were ever signed with the *same* $\vec{y}$ by accident (e.g. a broken random-number generator), the two resulting equations $\vec{z}_1 = \vec{y} + c_1\vec{s}_1$ and $\vec{z}_2 = \vec{y} + c_2\vec{s}_1$ can be subtracted to isolate $\vec{s}_1$ directly — a catastrophic, complete key-recovery, and a real failure class in classical ECDSA history (e.g. the Sony PlayStation 3 signing-key incident). Determinism eliminates the accidental version of this bug. But it introduces a new one: **if a fault attack can force the same $\vec{y}$ to be reused across two different signing operations** (by corrupting the PRF's internal state, or replaying an earlier internal value), the exact same subtraction attack becomes available again — this time caused deliberately rather than by accident. This is precisely the "fault attacks on determinism" line item flagged for Session 5: determinism is a defense against one failure mode and an attack surface for another, and real published Dilithium fault attacks exploit exactly this tension.

4.6 A Fully Worked Toy Example

Example: ML-DSA-Style Signing, $k = \ell = 1$, $n = 4$, $q = 17$ (Verified by Hand)

Reusing $R_{17} = \mathbb{Z}_{17}[x]/(x^4 + 1)$ and $A = 3 + 5x + 2x^2 + x^3$ from Lecture 3.
KeyGen. Let $s_1 = 1$ (constant polynomial) and $s_2 = 1 - x$ (small). Then

$$t = A \cdot s_1 + s_2 = A + (1 - x) = (3 + 1) + (5 - 1)x + 2x^2 + x^3 = 4 + 4x + 2x^2 + x^3.$$

Sign. Mask $y = x$. Commitment: $w = A \cdot y = A \cdot x = 16 + 3x + 5x^2 + 2x^3$ (computed exactly as in Lecture 2's ring-multiplication example). For this toy example, take the challenge to be the simplest possible sparse polynomial, $c = 1$ (constant). Response: $z = y + c \cdot s_1 = x + 1 = 1 + x$. (No rejection triggered — coefficients of z are small.) Signature: $(z, c) = (1 + x,\ 1)$.
Verify. Recompute $w' = A \cdot z - c \cdot t$:

$$\begin{aligned} A \cdot z &= A \cdot (1 + x) = A + A \cdot x = (3, 5, 2, 1) + (16, 3, 5, 2) \\ &= (19, 8, 7, 3) \equiv (2, 8, 7, 3) \bmod 17, \\ c \cdot t &= 1 \cdot t = (4, 4, 2, 1), \\ w' &= (2 - 4,\ 8 - 4,\ 7 - 2,\ 3 - 1) = (-2, 4, 5, 2) \equiv (15, 4, 5, 2) \bmod 17. \end{aligned}$$

Compare to the original $w = (16, 3, 5, 2)$: they are *not* identical — $w' = w - c \cdot s_2 = w - (1, -1, 0, 0)$, matching the algebra of Section 4.4's remark exactly (subtract $s_2 = 1 - x = (1, -1, 0, 0)$ from $w = (16, 3, 5, 2)$: $(16 - 1, 3 - (-1), 5, 2) = (15, 4, 5, 2) = w'$, confirmed). In this toy example we accept the signature by checking this exact algebraic identity holds (the real scheme's hint mechanism is what lets a verifier who does *not* know s_2 still confirm this identity, via the high-order bits of w instead of w itself — omitted here for tractability, as flagged in Section 4.4).

Exercise: Practice

Using the same A, s_1, s_2: sign the same message again, but this time with $y = 1$ (instead of $y = x$) and the same challenge $c = 1$. Compute the new z and confirm $w' = w - c \cdot s_2$ still holds with the new $w = A \cdot y$. Then, pretending you are an attacker who somehow learned that the *same* $y = x$ was used for two different messages with challenges $c_1 = 1$ and $c_2 = 2$ respectively, write down the two equations for z_1 and z_2 and show algebraically how subtracting them isolates s_1 — this is the calculation behind the factbox in Section 4.5.

4.7 FN-DSA (Falcon): a Different Lattice, a Different Technique

Falcon abandons MLWE entirely and uses a different hard problem on a different kind of lattice — worth seeing, precisely because your project needs to compare attack surfaces *across* schemes, not just understand one.

Definition: NTRU Lattices

Fix $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ as before. An NTRU key pair consists of two *short* polynomials $f, g \in R_q$ (the secret), from which the public key $h = g \cdot f^{-1} \bmod q$ is computed (assuming f is invertible mod q). The **NTRU lattice** associated to h is

$$\Lambda_h = \{\, (u, v) \in R_q^2 : u + v \cdot h \equiv 0 \pmod{q} \,\}.$$

(f, g) (suitably arranged) is a *short vector* in Λ_h — in fact, a short enough one to serve as a trapdoor *basis* for the whole lattice, not just a single short vector: this is the "all vectors short together" property Lecture 1's optional Minkowski's Second Theorem box was pointing toward.

Definition: The GPV Framework (Gentry–Peikert–Vaikuntanathan)

Given a trapdoor (short basis) for a lattice Λ, and a target point $t = H(\text{msg})$ (hashed into the same space as Λ), **sample** a lattice point $v \in \Lambda$ close to t, using the trapdoor to guide a randomized search — specifically, sampling from a discrete Gaussian distribution (Lecture 2, Section 2.5.3) centered near t. The signature is (essentially) the short offset $s = t - v$. Verification recomputes t from the message, checks s is short, and checks $s + v'$ (reconstructed from the public key and s) lands back on a valid target. Unlike Dilithium's commit-challenge-response shape, this is a single sampling step against a target — no interactive protocol being flattened, just a direct trapdoor-guided search.

Remark: Falcon's KeyGen/Sign/Verify, at a Glance

KeyGen: generate short f, g (this step itself is delicate number theory — specific polynomial sampling techniques — which we do not detail here); compute $h = gf^{-1} \bmod q$; the trapdoor is (f, g) (or an equivalent short basis derived from them), the public key is h. **Sign:** hash the message to a target point, then use the trapdoor with GPV sampling (Falcon's specific method is called *fast Fourier sampling*, chosen for speed) to find a short vector (s_1, s_2) with $s_1 + s_2 h \equiv H(\text{msg}) \pmod{q}$; output (s_1, s_2) (compressed) as the signature. **Verify:** recompute $s_1 = H(\text{msg}) - s_2 h \bmod q$ from the public h and the received s_2, and check that (s_1, s_2) is short enough to plausibly come from the trapdoor.

Looking Ahead: Why Falcon's Sampler Is the Star of Session 5

Two things make Falcon's signing step uniquely attack-prone compared to Dilithium's. First, GPV sampling needs to sample from a discrete Gaussian *exactly* (or very close to it) — an approximate or biased sampler can leak information about the trapdoor (f, g) through the statistical shape of many signatures, even without any single signature looking wrong. Second, Falcon's fast Fourier sampling is implemented using **floating-point arithmetic**, for speed — and floating-point operations have historically been far harder to make constant-time (free of secret-dependent timing or power variation) than the simple integer comparisons Dilithium's rejection sampling relies on. Put together: Dilithium's main defense (rejection sampling) is a young technique but algorithmically simple to implement safely; Falcon's main mechanism (Gaussian trapdoor sampling) is a mathematically older and better-understood technique, but a much harder *implementation* target to make side-channel-resistant. This is exactly the gap that produced the published single-trace and sampler-leakage attacks on Falcon that Session 5 will walk through.

4.8 Standardization Status

ML-DSA was finalized as **FIPS 204** alongside ML-KEM and SLH-DSA in August 2024, and is NIST's recommended general-purpose signature standard today.[1] FN-DSA (Falcon), by contrast, remains a **draft** standard (to become FIPS 206): as of mid-2026 it has not yet been finalized, with publication expected in late 2026 or early 2027, and major certificate authorities have stated they will not deploy it in production until it is.[2] Worth stating plainly in the proposal's related-work section: attacking a finalized federal standard (ML-DSA) and attacking a still-draft one (FN-DSA) carry different framing, even though both are technically live research targets today.

4.9 Comparing the Three Schemes So Far

	ML-KEM (Kyber)	**ML-DSA (Dilithium)**	**FN-DSA (Falcon)**
Hard problem	Module-LWE	Module-LWE	NTRU lattice (SIS/GPV-flavored)
Purpose	Key encapsulation	Signature	Signature
Core technique	MLWE encryption + FO transform	Fiat–Shamir-with-Aborts	GPV trapdoor sampling
Key defense mechanism	Implicit rejection (Decaps)	Rejection sampling	Exact Gaussian sampling
Riskiest implementation step	Re-encryption check	$\vec{y}$ generation / bound check	Floating-point sampler
Standard status (mid-2026)	FIPS 203, finalized	FIPS 204, finalized	FIPS 206, draft

Looking Ahead: Where This Leads: Session 4 and Session 5

You now have, for all three schemes, both the correct algorithm *and* a named "riskiest step." Session 4 will formalize the general distinction between attacking the math (everything in Lectures 1–4) versus attacking an implementation of it. Session 5 will then walk through real, published attacks on exactly the six risky steps identified across Lectures 3 and 4's summary tables — at that point, every attack should read as "targeting a specific line of an algorithm you can already write out from memory," not as an isolated trick.

[1] NIST, *Module-Lattice-Based Digital Signature Standard*, FIPS 204, August 2024.
[2] NIST Initial Public Draft, FIPS 206, submitted for public review in late 2025.

4.10 Exercises

Exercise: Homework Set

1. In your own words, explain the Fiat–Shamir transform to someone who has never seen an interactive proof — use the commit/challenge/response structure from Section 4.2, but invent your own non-cryptographic analogy (e.g. a game show, a magic trick) to illustrate why removing the interactive Verifier and replacing it with a hash still works.
2. Complete the practice exercise in Section 4.6 (two-signature key-recovery calculation) if you have not already.
3. Compare Kyber's re-encryption check (Lecture 3, Section 3.7) and Dilithium's rejection sampling (Section 4.5 above): both exist to prevent a secret-dependent quantity from becoming visible to an attacker. In 4–5 sentences, describe what each mechanism is protecting against, and why the two mechanisms look so different from each other despite that similarity of purpose.
4. Falcon's signature is a pair (s_1, s_2) with $s_1 + s_2 h \equiv H(\text{msg}) \pmod{q}$. Suppose $h = 5$ (a toy scalar stand-in, not a real ring element) and $H(\text{msg}) = 12 \pmod{17}$. If $s_2 = 2$, what must s_1 be for the equation to hold? Is this pair unique, or could other (s_1, s_2) pairs also satisfy the equation — and if so, what property (from Section 4.7) picks out the one Falcon actually outputs?
5. **Looking ahead:** using the comparison table in Section 4.9, predict — before Session 5 reveals the real answer — which of the three schemes' riskiest steps you would guess is *easiest* to defend against with constant-time coding practices alone, and which would need dedicated hardware countermeasures. Briefly justify your guess; we will revisit it directly in Session 5.